

실전 가이드

Claude Code로 팀 자동화 완성

에이전트와 원격 제어 실전 가이드



6명짜리 AI 개발팀을 내 컴퓨터에 만들고
어디서든 스마트폰으로 지휘한다

Claude Code

TMUX

Remote-Control

gstack

superpowers

RTK

클로드 코드(Claude Code)로 팀 자동화 완성 : 에이전트와 원격 제어 실전 가이드

이 책은 무엇인가요?

이 책은 **Claude AI**를 혼자 쓰는 것에서 벗어나, 여러 명의 **AI**를 팀처럼 동시에 운용하는 방법을 다룬 실전 가이드입니다.

보통 AI를 이렇게 씁니다.

나 → Claude에게 질문 → 답변 → 또 질문 → ...

이 책은 이걸 가능하게 합니다.

나 (스마트폰으로 지시)
↓
팀장 AI → 기획자 AI, 개발자 AI, 리뷰어 AI에게 동시에 배분

핵심 기술 3가지:

- **Claude Code** — 터미널에서 파일을 직접 읽고 쓰는 AI
- **TMUX** — 화면을 여러 칸으로 나눠 AI를 여러 명 실행
- **Remote-Control** — 스마트폰에서 내 컴퓨터의 AI를 원격 지휘

한 줄 요약: 6명짜리 AI 개발팀을 내 컴퓨터에 만들고, 어디서든 스마트폰으로 지휘한다.

이런 것들이 가능해집니다

출근길 지하철에서 일을 시킨다

스마트폰으로 "결제 모듈 버그 수정해줘"라고 보내면, 사무실에 도착했을 때 AI가 이미 코드를 고쳐놓고 있습니다.

6가지 일이 동시에 진행된다

기획자 AI가 설계 문서를 쓰는 동안, 개발자 AI가 코드를 짜고, 리뷰어 AI가 품질을 검토합니다.

순서를 기다릴 필요가 없습니다.

대화가 끊겨도 작업이 이어진다

새 세션을 시작해도 AI가 이전 작업 맥락을 파일에서 불러옵니다. "아까 어디까지 했더라"가 사라
집니다.

배포까지 **AI**가 챙긴다

코드 작성 → 테스트 → 리뷰 → PR 생성까지 한 흐름으로 이어집니다. 사람은 최종 승인만 합니
다.

어떻게 가능한가요?

세 가지 기술을 조합합니다.

TMUX — **AI**를 여러 명으로 만든다

터미널 화면을 6칸으로 나누고, 각 칸에서 Claude AI를 독립적으로 실행합니다. 각자 다른 역할을
맡아 동시에 일합니다.

CLAUDE.md — **AI**에게 역할을 부여한다

각 AI가 시작할 때 읽는 설정 파일입니다. "너는 개발자야, 설계는 네 역할이 아니야"처럼 역할 경
계를 명확히 합니다.

Remote-Control — 스마트폰을 리모컨으로 만든다

내 컴퓨터에서 실행 중인 AI에게 스마트폰으로 연결합니다. 코드와 파일은 절대 클라우드로 나가
지 않고, 통신 경로만 인터넷을 씁니다.

책 구성

장	제목	배우는 것
1장	들어가며	왜 팀 에이전트가 필요한가, 전체 구조 미리 보기
2장	환경 설치	Ubuntu/WSL2, Claude Code, TMUX 설치
3장	팀 구성	TMUX로 6칸 만들기, CLAUDE.md로 역할 부여
4장	Remote-Control	원격 제어 활성화, 서버 모드 설정
5장	모바일 연결	스마트폰 앱으로 세션 접속, 푸시 알림
6장	핵심 도구	gstack, superpowers, gsd, RTK 설치·활용
7장	실전 운용	팀 지시 흐름, Bot Mode, Triple Crown 전략

장	제목	배우는 것
8장	문제 해결	인증 오류, 세션 복구, 충돌 방지
9장	마치며	앞으로의 발전 방향

각 장은 이전 장 위에 쌓이는 구조입니다. 2장에서 기반을 만들고, 3~5장에서 팀을 구성하고, 6~7장에서 실전 수준으로 끌어올립니다.

이 책을 마치면

기술적으로 갖추게 되는 것

- Ubuntu + TMUX + Claude Code로 구성된 6인 AI 팀 환경
- 스마트폰에서 언제든지 접속할 수 있는 원격 제어 시스템
- gstack · superpowers · gsd를 조합한 Triple Crown 워크플로우

일하는 방식이 바뀌는 것

- "AI에게 물어보는" 사람에서 "AI 팀을 지휘하는" 사람으로
- 자리를 비워도 일이 진행되는 환경
- 혼자서 기획·개발·리뷰를 동시에 돌리는 1인 개발 체계

이 책이 진짜 가르치는 것

도구는 계속 바뀝니다. TMUX 대신 다른 것이 나올 수 있고, Claude 다음에 더 좋은 AI가 나올 수 있습니다. 하지만 역할 분리, 병렬 실행, 원격 제어, 맥락 유지라는 개념은 어떤 시대에도 유효합니다. 이 책은 그 개념을 가장 현실적인 도구로 직접 구현해보는 과정입니다.

Claude Code 팀 에이전트 운용 가이드

Remote-Control로 멀티에이전트를 원격에서 지휘하다

1장. 들어가며

- 1-1. 이 책의 목적과 대상 독자
 - 1-2. Remote-Control 팀 에이전트란 무엇인가
 - 1-3. 전체 아키텍처 미리 보기
-

2장. Ubuntu 환경에서 Claude & TMUX 설치

- 2-1. Ubuntu 설치: WSL2 또는 네이티브
 - 2-2. 필수 패키지 설치
 - 2-3. Claude Code 설치 및 인증
 - 2-4. TMUX 설치 및 기본 명령어
-

3장. Claude Code와 TMUX로 멀티에이전트 구성

- 3-1. TMUX 세션·윈도우·파인 구조
 - 3-2. 팀 에이전트 레이아웃 설계
 - 3-3. 각 파인에 Claude Code 자동 실행
 - 3-4. CLAUDE.md로 팀원 역할 정의
 - 3-5. 팀 셋업 스크립트 작성
-

4장. Claude Code Remote-Control 기능 설정 및 사용

- 4-1. Remote-Control 개요
- 4-2. Remote Control 활성화 방법 3가지
- 4-3. 서버 모드 및 Spawn 모드
- 4-4. 세션 이름 설정
- 4-5. Stream JSON 제어

- 4-6. 보안 설정 및 인증 요구사항
-

5장. 휴대폰 Claude 앱에서 Remote-Control 사용

- 5-1. Claude 모바일 앱 설치 및 계정 연결
 - 5-2. QR 코드로 세션 연결하기
 - 5-3. 세션 목록에서 팀 에이전트 선택
 - 5-4. 모바일에서 지시 전달 및 도구 승인
 - 5-5. 푸시 알림으로 작업 완료 확인
-

6장. 핵심 도구 설치 및 활용

- 6-1. gstack — Claude Code 플러그인 스택 관리
 - 설치 방법
 - 플러그인 추가/제거
 - 주요 명령어
 - 6-2. superpowers — 스킬 기반 워크플로우 자동화
 - 설치 방법
 - 스킬 목록 확인 및 호출
 - 커스텀 스킬 작성
 - 6-3. gsd — Get Stuff Done 프로젝트 관리
 - 설치 방법
 - 프로젝트 초기화 및 마일스톤 관리
 - 페이지 계획 및 실행
 - 6-4. RTK — Rust Token Killer 토큰 최적화
 - 설치 방법
 - 토큰 절약 분석 (`rtk gain`)
 - Hook 기반 자동 적용
-

7장. 실전 팀 에이전트 운용

- 7-1. 팀 지시 흐름 설계

- 7-2. Bot Mode 활용
 - 7-3. 업무 분담 전략
 - 7-4. Triple Crown 전략 — gstack + GSD + Superpowers 통합 워크플로우
 - 7-5. 자동화 워크플로우 예시
-

8장. 운영 및 문제 해결

- 8-1. Remote-Control 인증 오류 해결
 - 8-2. TMUX 세션 복구 및 재연결
 - 8-3. 토큰 최적화 심화
 - 8-4. 멀티에이전트 충돌 방지
-

9장. 마치며

- 9-1. 앞으로의 발전 방향
- 9-2. 커뮤니티 참여

1-1. 이 책의 목적과 대상 독자

왜 이 책을 쓰게 되었는가

2026년 현재, AI 코딩 어시스턴트는 더 이상 신기한 도구가 아니다. 개발자라면 누구나 한 번쯤 AI에게 코드를 요청해본 경험이 있을 것이다. 그러나 대부분의 사용자는 여전히 하나의 창에서 하나의 **AI**와 일대일 대화를 나누는 방식에 머물러 있다.

만약 AI 에이전트를 여러 명 동시에 운용할 수 있다면 어떨까? PM이 설계를 하는 동안 개발자가 코드를 짜고, 리뷰어가 품질을 검토하는 — 마치 실제 개발팀처럼 AI를 구성할 수 있다면?

이 책은 바로 그 질문에서 시작되었다. **Claude Code**와 **TMUX**를 결합하여 멀티에이전트 팀을 구성하고, **Remote-Control** 기능으로 어디서든 이 팀을 지휘하는 실전 방법을 다룬다.

이 책에서 다루는 내용

이 책은 크게 세 가지 영역을 다룬다.

1. 환경 구축

Ubuntu(WSL2 포함) 위에 Claude Code와 TMUX를 설치하고, 6개의 파인(Pane)으로 구성된 팀 에이전트 환경을 만드는 과정을 안내한다.

Pane	이름	역할
0	팀장	총괄·라우터
1	민준	PM·아키텍트
2	지훈	리서처
3	수아	디자이너
4	서연	개발자
5	태양	리뷰어

AI 팀 에이전트 구성

2. Remote-Control 연동

Claude Code의 Remote-Control 기능을 활성화하여 모바일 앱이나 웹 브라우저에서 팀 에이전트를 원격 제어하는 방법을 상세히 설명한다. 출퇴근길 지하철에서도, 카페에서도 팀에게 지시를 내릴 수 있다.

3. 실전 운용

Remote-Control, Bot Mode, 업무 분담 전략 등 실제 팀을 운용하면서 축적된 노하우를 공유한다. 이론이 아닌 실전 경험에 기반한 내용이다.

대상 독자

이 책은 다음과 같은 독자를 위해 작성되었다.

대상	설명
AI 활용에 관심 있는 개발자	Claude Code를 사용해본 적 있으나 멀티에이전트 구성은 처음인 분
팀 생산성을 높이고 싶은 리더	AI를 팀 단위로 활용하여 개발 속도를 높이려는 테크 리더
1인 개발자	혼자서 여러 역할(PM, 개발, 리뷰)을 AI에게 분담시키고 싶은 분
DevOps/자동화 엔지니어	CI/CD 파이프라인에 AI 에이전트를 통합하려는 분

사전 지식

다음 정도의 기본 지식이 있으면 이 책을 따라가는 데 어려움이 없다.

- Linux 터미널 기본 조작 — `cd`, `ls`, `cat` 같은 기본 명령어
- Git 기본 워크플로우 — `commit`, `push`, `branch` 정도
- 마크다운 문법 — CLAUDE.md 파일을 작성해야 하므로

TMUX를 처음 접하더라도 괜찮다. 2장에서 설치부터 기본 명령어까지 차근차근 안내한다.

이 책을 읽는 방법

순서대로 읽는 것을 권장하지만, 이미 Claude Code와 TMUX에 익숙하다면 **4장(Remote-Control)**이나 **7장(실전 운용)**부터 시작해도 된다.

각 장의 코드 예시는 복사하여 바로 실행할 수 있도록 작성했다. 읽기만 하지 말고, 직접 터미널을 열어 따라 해보길 권한다. AI 팀을 직접 만들어보는 순간, "아, 이게 되는구나" 하는 감각이 올 것이다.

한 줄 요약: 이 책은 Claude Code + TMUX + Remote-Control로 AI 팀 에이전트를 구성하고 원격으로 지휘하는 실전 가이드다.

1-2. Remote-Control 팀 에이전트란 무엇인가

기존 AI 사용 방식의 한계

대부분의 개발자가 AI 코딩 어시스턴트를 사용하는 방식은 이렇다.

[개발자] → [AI 챗봇] → [응답]

하나의 터미널(또는 웹 인터페이스)에서 하나의 AI와 대화한다. 질문을 던지고, 답을 받고, 다시 질문한다. 이 방식에는 몇 가지 근본적인 한계가 있다.

1. 단일 컨텍스트 — 하나의 대화창에서 설계, 구현, 리뷰를 모두 처리하면 컨텍스트가 뒤섞인다
2. 순차 처리 — 한 번에 하나의 작업만 진행할 수 있다. 리서치가 끝나야 설계를, 설계가 끝나야 구현을 시작한다
3. 디바이스 종속 — 터미널을 닫으면 세션이 끊긴다. 자리를 비우면 작업도 멈춘다

Remote-Control 팀 에이전트의 개념

이 책에서 제안하는 방식은 완전히 다르다.

팀 에이전트 개념도

핵심은 세 가지 기술의 결합이다.

1. TMUX — 다중 에이전트의 물리적 공간

TMUX는 하나의 터미널 안에 여러 개의 독립된 창(파인)을 만들 수 있는 터미널 멀티플렉서다. 각 파인에서 별도의 Claude Code 인스턴스를 실행하면, 물리적으로 분리된 여러 AI 에이전트가 탄생한다.

```
# 6개의 파인으로 팀 구성
tmux new-session -s team
tmux split-window -h
tmux split-window -v
# ... 각 파인에 Claude Code 실행
```

각 에이전트는 독립된 컨텍스트를 가진다. PM의 대화가 개발자의 코드 작성을 방해하지 않는다.

2. CLAUDE.md — 역할과 규칙의 정의

CLAUDE.md 파일은 Claude Code의 행동 규칙을 정의하는 설정 파일이다. 각 에이전트에게 서로 다른 역할을 부여할 수 있다.

```
# 서연 (개발자)
## 역할
- 코드 구현 담당
- 팀장의 지시에 따라 기능 개발
- 커밋 전 자체 테스트 수행

## 규칙
- 직접 판단으로 아키텍처 변경 금지
- 구현 완료 시 팀장에게 보고
```

3. Remote-Control — 어디서든 팀 지휘

Claude Code의 Remote-Control 기능은 로컬에서 실행 중인 세션을 다른 기기에서 제어할 수 있게 한다.

```
# 로컬에서 Remote-Control 활성화
claude --remote-control "팀 에이전트"
```

이후 **claude.ai/code** 웹사이트나 **Claude** 모바일 앱에서 해당 세션에 접속하여 지시를 내릴 수 있다. 세션은 로컬 머신에서 계속 실행되므로, 모바일에서 접속해도 로컬 파일시스템과 모든 도구에 접근 가능하다.

전통적 팀 vs. AI 팀 에이전트

비교 항목	인간 팀	AI 팀 에이전트
가용 시간	업무 시간	24시간
역할 전환	채용·교육 필요	CLAUDE.md 수정으로 즉시
동시 작업	인원 수에 비례	TMUX 파인 수만큼
커뮤니케이션	회의·슬랙	TMUX <code>send-keys</code>
원격 지휘	메신저·이메일	Remote-Control

비교 항목	인간 팀	AI 팀 에이전트
비용	인건비	API 토큰 비용

실제 활용 시나리오

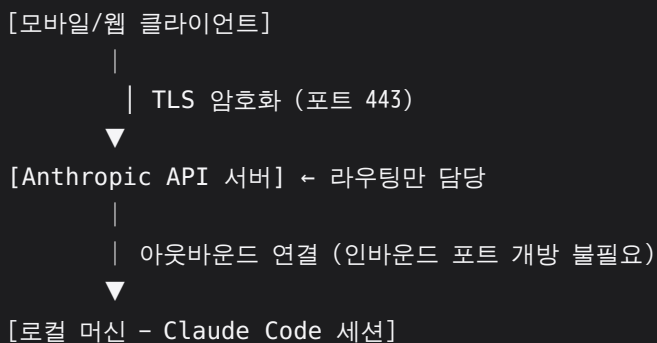
시나리오: 새 기능 개발

1. 출근길 지하철에서 — 모바일 앱으로 팀장에게 지시: "결제 모듈 리팩토링 시작해줘"
2. 팀장이 분석 — 작업을 분해하여 팀원에게 분배
3. **PM(민준)** — 기존 결제 흐름 분석, 새 아키텍처 설계
4. 개발자(서연) — PM의 설계에 따라 코드 구현
5. 리뷰어(태양) — 구현된 코드 리뷰 및 피드백
6. 사무실 도착 — 터미널에서 진행 상황 확인, 필요한 부분만 수정

이 모든 과정이 하나의 **TMUX** 세션 안에서 이루어지며, **Remote-Control**을 통해 어디서든 개입할 수 있다.

Remote-Control의 기술적 특징

Remote-Control이 단순한 원격 접속과 다른 점을 짚어두자.



- 로컬 실행: 세션은 항상 로컬 머신에서 실행된다. 클라우드로 이전되지 않는다
- 인바운드 포트 없음: 로컬 머신에 포트를 열 필요가 없다. 보안에 유리하다
- 자동 재연결: 네트워크가 끊겨도 10분 이내에 복구되면 자동으로 재연결된다
- 실시간 동기화: 모바일에서 보내는 메시지가 즉시 로컬 세션에 반영된다

핵심 정리: Remote-Control 팀 에이전트란, TMUX로 구성된 다중 Claude Code 인스턴스를 Remote-Control을 통해 어디서든 원격 지휘하는 시스템이다.

1-3. 전체 아키텍처 미리 보기

시스템 구성도

이 책에서 구축하는 시스템의 전체 아키텍처를 미리 살펴보자. 각 구성 요소가 어떻게 연결되는지 큰 그림을 먼저 파악하면, 이후 장들을 따라가기가 훨씬 수월하다.

전체 시스템 아키텍처

계층별 상세 설명

1. 제어 레이어 (Control Layer)

사용자가 팀에게 지시를 내리는 인터페이스 계층이다. 두 가지 경로를 지원한다.

경로	용도	프로토콜
claude.ai/code	웹 브라우저에서 세션 제어	Remote-Control (TLS)
Claude 모바일 앱	스마트폰에서 원격 지시	Remote-Control (TLS)

Remote-Control은 Anthropic API 서버를 경유하여 로컬 세션과 통신한다. 세션은 항상 로컬 머신에서만 실행되며 로컬 파일시스템과 MCP 서버는 외부로 전송되지 않는다.

2. 오케스트레이션 레이어 (Orchestration Layer)

팀장(Pane 0)이 사용자의 지시를 수신하고, 분석하여 적절한 팀원에게 분배하는 계층이다.

```
# 사용자의 지시가 팀장에게 도달하는 흐름
[사용자] "결제 모듈 리팩토링해줘"
|
▼
[팀장 - Pane 0]
| 분석: 리팩토링 → PM에게 설계, 개발자에게 구현, 리뷰어에게 검토
|
|→ tmux send-keys -t team:0.1 "설계 작업..." Enter
|→ tmux send-keys -t team:0.4 "구현 작업..." Enter
|→ tmux send-keys -t team:0.5 "리뷰 준비..." Enter
```

팀장은 직접 코드를 작성하거나 파일을 수정하지 않는다. 지시를 수령하고, 작업을 분해하고, 팀원에게 배분하고, 결과를 종합하여 보고하는 역할만 수행한다.

3. 실행 레이어 (Execution Layer)

각 팀원(Pane 1~5)이 할당받은 작업을 실제로 수행하는 계층이다.

파인	역할	주요 작업
Pane 1 (민준)	PM·아키텍트	프로젝트 계획, 시스템 설계, 기술 문서 작성
Pane 2 (지훈)	리서처	기술 조사, 라이브러리 비교, 레퍼런스 수집
Pane 3 (수아)	디자이너	UI/UX 설계, 사용자 경험 기획, 프로토타입
Pane 4 (서연)	개발자	코드 구현, 테스트 작성, 버그 수정
Pane 5 (태양)	리뷰어	코드 리뷰, 품질 검토, 보안 점검

모든 팀원은 동일한 로컬 파일시스템에 접근하므로, 한 팀원이 작성한 파일을 다른 팀원이 바로 읽을 수 있다.

4. 인프라 레이어 (Infrastructure Layer)

팀 에이전트가 작업하는 환경의 기반이다.

- 파일시스템: 프로젝트 소스 코드, 설정 파일, 문서 등
- **Git** 저장소: 버전 관리 및 협업. 팀원이 작성한 코드를 커밋·푸시
- **MCP** 서버: Claude Code의 도구 확장. 외부 서비스 연동

데이터 흐름 상세

지시 전달 흐름

1. 사용자가 모바일 앱 또는 `claude.ai/code`에서 지시 전송
"결제 API 리팩토링해줘"
2. `Remote-Control`이 메시지를 로컬 세션(Pane 0)으로 전달
3. 팀장(Pane 0)이 지시 분석
→ 작업 분해: [설계] [구현] [리뷰]
4. 팀장이 각 팀원에게 지시 전달
→ `tmux send-keys -t team:0.1 "..."` (PM에게)
→ `tmux send-keys -t team:0.4 "..."` (개발자에게)

5. 팀원이 작업 수행 후 결과 보고
→ 각 팀원이 팀장에게 완료 알림

6. 팀장이 결과를 종합하여 Remote-Control을 통해 사용자에게 보고

핵심 기술 스택 요약

기술	버전	역할
Ubuntu	22.04+ (WSL2 가능)	운영 체제
Claude Code	2.x	AI 에이전트 엔진
TMUX	3.x	터미널 멀티플렉서 (다중 파인)
Remote-Control	Claude Code 내장	원격 세션 제어
Git	2.x	버전 관리

이후 강의 로드맵

2장: Ubuntu 환경 구축 → 인프라 레이어 셋업
3장: TMUX 멀티에이전트 → 실행 레이어 구성
4장: Remote-Control 설정 → 제어 레이어 (Remote-Control)
5장: 모바일 원격 제어 → 제어 레이어 (모바일)
6장: 핵심 도구 설치 → 도구 확장
7장: 실전 팀 운용 → 오케스트레이션 + 운용 노하우
8장: 운영 및 문제 해결 → 유지보수
9장: 마치며 → 향후 방향

각 장은 아키텍처의 한 계층씩 쌓아올리는 구조다. 2장에서 기반을 만들고, 3~5장에서 팀을 구성하고, 6~7장에서 실전 운용하고, 8장에서 안정적으로 유지하는 방법을 다룬다.

핵심 정리: 전체 아키텍처는 제어(Remote-Control) → 오케스트레이션(팀장) → 실행(팀원) → 인프라(파일/Git/MCP)의 4개 계층으로 구성된다.

2-1. Ubuntu 설치: WSL2 또는 네이티브

이 책에서 구성하는 Claude 멀티에이전트 환경은 **Ubuntu Linux** 위에서 동작합니다. Windows 사용자라면 WSL2(Windows Subsystem for Linux 2)를 통해, 리눅스/Mac 사용자라면 네이티브 환경을 그대로 사용할 수 있습니다.

WSL2로 Ubuntu 설치하기 (Windows)

WSL2는 Windows 10/11에서 완전한 Linux 커널을 실행할 수 있는 Microsoft 공식 솔루션입니다. 성능과 호환성 모두 네이티브에 가깝습니다.

1단계: WSL2 활성화

PowerShell을 관리자 권한으로 열고 아래 명령을 실행합니다.

```
wsl --install
```

이 명령 하나로 WSL2 기능 활성화, 가상 머신 플랫폼 설치, Ubuntu 최신 LTS 다운로드까지 자동으로 처리됩니다. 설치 완료 후 PC를 재시작하세요.

재시작 후 Ubuntu 초기 설정 창이 자동으로 열립니다.

```
Enter new UNIX username: allen      # 원하는 사용자명 입력
New password: ****                  # 비밀번호 설정
```

2단계: WSL 버전 확인

```
wsl --list --verbose
```

출력 결과에서 Ubuntu 행의 VERSION이 2 인지 확인합니다.

NAME	STATE	VERSION
* Ubuntu	Running	2

VERSION이 1 이라면 다음 명령으로 업그레이드합니다.

```
wsl --set-version Ubuntu 2
```

3단계: Ubuntu 터미널 열기

시작 메뉴에서 **Ubuntu** 앱을 실행하거나, Windows Terminal에서 Ubuntu 탭을 선택합니다. 이후 모든 작업은 이 Ubuntu 터미널에서 진행합니다.

네이티브 Ubuntu 설치 (리눅스/Mac)

이미 Ubuntu 22.04 이상 또는 Debian 계열 배포판을 사용 중이라면 별도 설치 없이 다음 챕터로 넘어가도 됩니다. macOS는 Homebrew 환경을 그대로 활용할 수 있습니다.

패키지 목록 최신화

설치 후 반드시 패키지 목록을 업데이트합니다.

```
sudo apt update && sudo apt upgrade -y
```

WSL2 네트워크 정보 확인

WSL2는 Windows 호스트와 별도의 가상 IP를 가집니다. 나중에 SSH나 포트 접근이 필요할 때 유용합니다.

```
# WSL2 IP 확인
ip addr show eth0 | grep 'inet '

# Windows 호스트 IP 확인 (WSL 내부에서)
cat /etc/resolv.conf | grep nameserver
```

요약

환경	방법
Windows	<code>wsl --install</code> → 재부팅 → Ubuntu 앱 실행
기존 Ubuntu/Debian	그대로 진행
macOS	Homebrew 환경 유지, bash/zsh 터미널 사용

Ubuntu 환경이 준비되었으면 다음 챕터에서 Claude Code 실행에 필요한 패키지들을 설치합니다.

2-2. 필수 패키지 설치

Claude Code와 TMUX 기반 멀티에이전트 환경을 구성하려면 Node.js, npm, 그리고 몇 가지 유틸리티가 필요합니다. 이 챕터에서는 순서대로 설치합니다.

Node.js 설치

Claude Code는 npm 패키지로 배포됩니다. **Node.js 18** 이상이 필요하며, 최신 LTS 버전을 권장합니다.

nvm으로 설치 (권장)

nvm(Node Version Manager)을 사용하면 Node.js 버전을 유연하게 관리할 수 있습니다.

```
# nvm 설치
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.40.1/install.sh | bash

# 셸 재로드
source ~/.bashrc

# nvm 설치 확인
nvm --version

# Node.js LTS 설치
nvm install --lts

# 버전 확인
node --version
npm --version
```

출력 예시:

```
v22.14.0
10.9.0
```

apt로 직접 설치 (대안)

```
# NodeSource 저장소 추가 (Node.js 22.x)
curl -fsSL https://deb.nodesource.com/setup_22.x | sudo -E bash -

# 설치
```

```
sudo apt install -y nodejs
```

```
# 확인  
node --version  
npm --version
```

Git 설치

Git은 Claude Code의 프로젝트 관리와 설정 파일 버전 관리에 활용됩니다.

```
sudo apt install -y git
```

```
# 설치 확인  
git --version
```

초기 사용자 설정도 함께 진행합니다.

```
git config --global user.name "Your Name"  
git config --global user.email "your@email.com"
```

curl, wget, unzip

네트워크 도구와 압축 해제 유틸리티입니다.

```
sudo apt install -y curl wget unzip
```

build-essential (선택)

일부 npm 패키지가 네이티브 빌드를 요구할 수 있습니다.

```
sudo apt install -y build-essential
```

전체 설치 요약 스크립트

위 단계를 한 번에 실행하려면 아래 스크립트를 사용하세요.

```
#!/bin/bash
set -e

# 시스템 업데이트
sudo apt update && sudo apt upgrade -y

# 기본 도구
sudo apt install -y git curl wget unzip build-essential

# nvm & Node.js LTS
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.40.1/install.sh | bash
source ~/.bashrc
nvm install --lts

echo "✅ 설치 완료"
echo " node: $(node --version)"
echo " npm:  $(npm --version)"
echo " git:  $(git --version)"
```

설치 확인 체크리스트

```
node --version    # v18.0.0 이상
npm --version     # 9.0.0 이상
git --version     # 2.x.x
curl --version    # 설치 확인
```

모든 명령이 오류 없이 버전을 출력하면 다음 단계로 넘어갑니다.

2-3. Claude Code 설치 및 인증

Claude Code는 Anthropic이 만든 AI 코딩 어시스턴트 CLI 도구입니다. 터미널에서 직접 Claude와 대화하며 코드를 작성하고, 파일을 편집하고, 명령을 실행할 수 있습니다. 멀티에이전트 환경에서는 각 TMUX 파인마다 Claude Code 인스턴스가 독립적으로 동작합니다.

설치

npm을 통해 전역 설치합니다.

```
npm install -g @anthropic-ai/claude-code
```

설치 후 버전을 확인합니다.

```
claude --version
```

출력 예시:

```
claude-code 2.1.71
```

Claude Code 설치 확인

인증

Claude Code를 처음 실행하면 Anthropic 계정 인증이 필요합니다.

```
claude
```

처음 실행 시 아래 두 가지 다이얼로그가 순서대로 나타납니다.

다이얼로그 1: 폴더 신뢰

```
Do you trust the files in this folder?  
> Yes, I trust this folder (proceed)  
No, exit
```


Enter를 눌러 신뢰를 승인합니다.

다이얼로그 2: 이용 약관

```
Do you accept the terms of service?  
No, exit  
> Yes, I accept
```

아래 방향으로 "Yes, I accept"를 선택 후 Enter를 누릅니다.

로그인 방식 선택

```
How would you like to authenticate?  
> Login with claude.ai (recommended)  
Use an API key
```

`Login with claude.ai`를 선택하면 브라우저가 열리고 Anthropic 계정으로 OAuth 로그인을 진행합니다.

중요: Remote Control 기능을 사용하려면 반드시 `claude.ai`를 통한 OAuth 로그인이 필요합니다. API 키 방식으로는 Remote Control이 활성화되지 않습니다.

인증 확인

로그인 후 Claude 프롬프트(>)가 나타나면 인증이 완료된 것입니다.

```
> Hello! What can I help you with?
```

현재 인증 상태는 `/status` 명령으로 확인할 수 있습니다.

```
> /status
```

모델 설정

기본 모델은 Claude Sonnet입니다. 다른 모델로 시작하려면 `--model` 플래그를 사용합니다.

```
# Sonnet (기본, 속도·비용 균형)
claude --model claude-sonnet-4-6

# Opus (고성능, 복잡한 작업)
claude --model claude-opus-4-6

# Haiku (경량, 빠른 응답)
claude --model claude-haiku-4-5-20251001
```

권한 설정 (멀티에이전트용)

여러 파인에서 자동으로 실행할 때는 권한 다이얼로그를 건너뛰는 플래그를 사용합니다.

```
claude --dangerously-skip-permissions
```

이 플래그는 파일 읽기/쓰기, 명령 실행 등의 권한 승인 절차를 자동으로 허용합니다. 신뢰할 수 있는 환경에서만 사용하세요.

업데이트

새 버전이 출시되면 동일한 npm 명령으로 업데이트합니다.

```
npm update -g @anthropic-ai/claude-code

# 또는
npm install -g @anthropic-ai/claude-code@latest
```

요약

```
# 설치
npm install -g @anthropic-ai/claude-code

# 최초 실행 및 인증
claude

# 인증 후 정상 동작 확인
claude --version # 버전 출력
claude           # > 프롬프트 확인
```

다음 챕터에서는 멀티에이전트 환경의 핵심인 TMUX를 설치하고 기본 사용법을 익힙니다.

2-4. TMUX 설치 및 기본 명령어

TMUX(Terminal Multiplexer)는 하나의 터미널 창 안에서 여러 개의 터미널을 동시에 실행하고 관리할 수 있는 강력한 도구입니다. Claude 멀티에이전트 환경에서는 TMUX의 파인(Pane) 기능을 활용해 6명의 에이전트를 동시에 운영합니다.

설치

```
sudo apt install -y tmux
```

```
# 설치 확인  
tmux -V
```

출력 예시:

```
tmux 3.4
```

TMUX 버전 확인

핵심 개념

TMUX는 세 가지 계층 구조로 구성됩니다.

```
세션(Session)  
├── 윈도우(Window)  
│   └── 파인(Pane)
```

- 세션: TMUX의 최상위 단위. 여러 윈도우를 포함. 터미널을 닫아도 세션은 백그라운드에서 유지됩니다.
 - 윈도우: 세션 내의 탭. 하나의 전체 화면을 차지합니다.
 - 파인: 윈도우를 분할한 공간. 각 파인은 독립적인 셸을 실행합니다.
-

기본 명령어

세션 관리

```
# 새 세션 생성
tmux new-session -s team

# 세션 목록 확인
tmux ls

# 세션 접속 (attach)
tmux attach -t team
tmux a -t team      # 단축 형식

# 세션 분리 (detach, 세션은 백그라운드 유지)
# 단축키: Ctrl+B, D

# 세션 종료
tmux kill-session -t team
```

윈도우 관리

TMUX 세션 안에서 사용하는 단축키입니다. 모든 단축키는 **Ctrl+B** 를 먼저 누른 후 키를 입력합니다.

단축키	동작
Ctrl+B c	새 윈도우 생성
Ctrl+B n	다음 윈도우로 이동
Ctrl+B p	이전 윈도우로 이동
Ctrl+B 0~9	번호로 윈도우 이동
Ctrl+B &	현재 윈도우 종료

파인 관리

단축키	동작
Ctrl+B %	세로로 분할 (좌우)

단축키	동작
Ctrl+B "	가로로 분할 (상하)
Ctrl+B 방향키	파인 간 이동
Ctrl+B x	현재 파인 종료
Ctrl+B z	현재 파인 전체 화면 토글
Ctrl+B Ctrl+방향키	파인 크기 조절

명령으로 파인 제어하기

스크립트에서 특정 파인에 명령을 전송하는 방법입니다.

```
# 특정 파인에 텍스트 전송 (Enter 없이)
tmux send-keys -t team:0.1 "echo hello"

# 특정 파인에 명령 실행 (Enter 포함)
tmux send-keys -t team:0.1 "echo hello" Enter

# 파인 내용 캡처 (현재 화면 텍스트 읽기)
tmux capture-pane -t team:0.1 -p
```

형식: `세션이름:윈도우번호.파인번호`

레이아웃 설정

TMUX는 파인 배치를 자동으로 정렬하는 레이아웃 기능을 제공합니다.

```
# 레이아웃 적용
tmux select-layout -t team:0 even-horizontal # 좌우 균등
tmux select-layout -t team:0 even-vertical   # 상하 균등
tmux select-layout -t team:0 main-vertical   # 왼쪽 크게, 나머지 세로 배열
tmux select-layout -t team:0 main-horizontal # 위 크게, 나머지 가로 배열
tmux select-layout -t team:0 tiled           # 격자 배열
```

파일 제목 설정

각 파일에 이름(제목)을 붙여 누가 어떤 파일인지 한눈에 알 수 있게 합니다.

```
# 파일 상단 제목 표시 활성화
tmux set-option -t team pane-border-status top
tmux set-option -t team pane-border-format " #{pane_title} "

# 파일 이름 설정
tmux select-pane -t team:0.0 -T "쥔 (팀장)"
tmux select-pane -t team:0.1 -T "민준 아키텍트"
```

실습: 간단한 멀티파일 세션

```
# 1. 새 세션 생성 (백그라운드)
tmux new-session -d -s practice

# 2. 파일 분할 (좌우 2개)
tmux split-window -t practice:0.0 -h

# 3. 각 파일에 명령 실행
tmux send-keys -t practice:0.0 "echo 'Pane 0 작동 중'" Enter
tmux send-keys -t practice:0.1 "echo 'Pane 1 작동 중'" Enter

# 4. 세션 접속하여 확인
tmux attach -t practice
```

요약

TMUX의 핵심 흐름은 세션 생성 → 파일 분할 → 각 파일에 명령 전송입니다. 다음 챕터에서는 이 구조를 활용해 6명의 Claude 에이전트가 동시에 동작하는 팀 환경을 구성합니다.

3-1. TMUX 세션·윈도우·파인 구조

멀티에이전트 환경을 설계하기 전에 TMUX의 계층 구조를 완전히 이해해야 합니다. 에이전트 간 통신, 레이아웃 구성, 자동화 스크립트 모두 이 구조를 기반으로 작동합니다.

계층 구조 전체 그림

```
tmux 서버 (백그라운드 프로세스)
|
├── 세션: team                                ← tmux new-session -s team
|   ├── 윈도우 0: 메인 작업공간              ← Window 0 (기본)
|   |   ├── 파인 0: 쥔 (팀장)                ← Pane 0
|   |   ├── 파인 1: 민준                    ← Pane 1
|   |   ├── 파인 2: 지훈                    ← Pane 2
|   |   ├── 파인 3: 수아                    ← Pane 3
|   |   ├── 파인 4: 서연                    ← Pane 4
|   |   └── 파인 5: 태양                    ← Pane 5
|   └── 윈도우 1: 로그/모니터링              ← Window 1 (선택)
|
└── 세션: work                                ← 별도 작업용 세션 (선택)
```

세션(Session)

세션은 TMUX의 최상위 단위입니다. 터미널을 닫거나 SSH 연결이 끊겨도 세션은 서버에서 계속 실행됩니다.

```
# 세션 생성 (백그라운드)
tmux new-session -d -s team -x 317 -y 85
#                  ^백그라운드 ^이름 ^너비 ^높이

# 세션 목록 확인
tmux ls
# team: 1 windows (created ...) (attached)

# 세션 이름 변경
tmux rename-session -t team newname
```

`-x` 와 `-y` 는 세션의 가상 터미널 크기입니다. 실제 터미널 크기보다 크게 설정하면 파인이 잘리지 않습니다.

윈도우(Window)

윈도우는 세션 안의 탭입니다. 한 번에 하나의 윈도우만 화면에 표시됩니다.

```
# 현재 세션에 새 윈도우 추가
tmux new-window -t team

# 윈도우 이름 설정
tmux rename-window -t team:0 "agents"
tmux rename-window -t team:1 "monitor"

# 특정 윈도우로 전환 (세션 내부에서)
# Ctrl+B 0 → 윈도우 0
# Ctrl+B 1 → 윈도우 1
```

멀티에이전트 환경에서는 보통 윈도우 0 하나만 사용합니다. 6개 파인이 모두 윈도우 0에 배치됩니다.

파인(Pane)

파인은 윈도우를 분할한 공간으로, 각각 독립적인 쉘 프로세스를 실행합니다. Claude 에이전트 하나당 파인 하나를 할당합니다.

파인 분할

```
# 세로 분할 (좌우로 나누기, -h = horizontal split)
tmux split-window -t team:0.0 -h

# 가로 분할 (상하로 나누기, -v = vertical split)
tmux split-window -t team:0.0 -v
```

파인 번호 체계

파인 번호는 분할 순서에 따라 자동으로 부여됩니다. 6개 파인을 생성하는 표준 순서는 다음과 같습니다.

```
# 세션 생성 → Pane 0 자동 생성
tmux new-session -d -s team
```

```
# 5번 분할 → Pane 1~5 생성
tmux split-window -t team:0.0 -h # Pane 1
tmux split-window -t team:0.1 -h # Pane 2
tmux split-window -t team:0.2 -h # Pane 3
tmux split-window -t team:0.3 -h # Pane 4
tmux split-window -t team:0.4 -h # Pane 5

# 균등 배분
tmux select-layout -t team:0 even-horizontal
```

파일 주소 지정

스크립트에서 특정 파일을 지정할 때는 **세션:윈도우.파일** 형식을 사용합니다.

```
team:0.0 # team 세션, 윈도우 0, 파일 0
team:0.3 # team 세션, 윈도우 0, 파일 3
```

파일 정보 조회

```
# 모든 파일 목록과 정보
tmux list-panes -t team:0

# 출력 예시:
# 0: [158x84] [history 500/50000, 1234 bytes] %0
# 1: [38x84] [history 100/50000, 567 bytes] %1

# 특정 파일의 현재 화면 내용 캡처
tmux capture-pane -t team:0.2 -p

# 특정 파일에서 실행 중인 프로세스 확인
tmux list-panes -t team:0 -F "#{pane_index}: #{pane_current_command}"
```

main-vertical 레이아웃

실제 팀 환경에서 자주 사용하는 레이아웃입니다. 파일 0이 왼쪽에 넓게 자리잡고, 나머지 파일이 오른쪽에 세로로 쌓입니다.

```
tmux select-layout -t team:0 main-vertical
tmux set-option -t team main-pane-width 158
```

Pane	팀원	위치
0	쥔 (팀장)	왼쪽 메인 (너비 158)
1	민준	오른쪽 상단
2	지훈	오른쪽 2번째
3	수아	오른쪽 3번째
4	서연	오른쪽 4번째
5	태양	오른쪽 하단

요약

계층	생성 명령	주소 형식
세션	<code>tmux new-session -s 이름</code>	<code>team</code>
윈도우	<code>tmux new-window -t 세션</code>	<code>team:0</code>
파인	<code>tmux split-window -t 세션:윈도우</code>	<code>team:0.2</code>

이 구조를 이해했으면 다음 챕터에서 6명의 에이전트가 배치되는 실제 레이아웃을 설계합니다.

3-2. 팀 에이전트 레이아웃 설계

TMUX 구조를 이해했다면 이제 실제로 6명의 Claude 에이전트가 배치될 팀 레이아웃을 설계합니다. 좋은 레이아웃은 각 에이전트의 역할을 명확하게 하고, 시각적으로 팀 전체 상태를 한눈에 파악할 수 있게 합니다.

팀 구성 설계

이 책에서 사용하는 팀 구성은 실제 소프트웨어 개발팀을 모델로 합니다.

파인	이름	역할	모델
Pane 0	쫘	팀장 — 지시 수령, 작업 배분, 결과 보고	Sonnet
Pane 1	민준	아키텍트 — 시스템 설계, 기술 방향 결정	Opus
Pane 2	지훈	개발자 — 기능 구현, 코드 작성	Sonnet
Pane 3	수아	디자이너 — UI/UX, 사용자 경험 설계	Sonnet
Pane 4	서연	리서처 — 정보 수집, 기술 조사	Sonnet
Pane 5	태양	리뷰어 — 코드 리뷰, 품질 검토	Sonnet

민준(아키텍트)은 복잡한 설계 판단이 많아 Claude Opus를 사용합니다. 나머지는 속도와 비용이 균형 잡힌 Claude Sonnet을 사용합니다.

레이아웃 구성

main-vertical 레이아웃으로 팀장(Pane 0)이 왼쪽 넓은 공간을 차지하고, 팀원 5명(Pane 1~5)이 오른쪽에 세로로 배열됩니다.

Pane	이름	역할	비고
0	쫘	팀장	Claude CLI
1	민준	아키텍트	

Pane	이름	역할	비고
2	지훈	개발자	
3	수아	UI/UX 디자이너	
4	서연	리서처	
5	태양	QA·리뷰어	

실제 TMUX 파인 목록 출력 결과

레이아웃 구성 스크립트

```
#!/bin/bash
SESSION="team"

# 1. 세션 생성 (백그라운드)
TERM_WIDTH=$(tput cols 2>/dev/null || echo 317)
TERM_HEIGHT=$(tput lines 2>/dev/null || echo 85)
tmux new-session -d -s "$SESSION" -x "$TERM_WIDTH" -y "$TERM_HEIGHT"

# 2. 파인 5개 분할 (좌우, even-horizontal 먼저 적용)
tmux split-window -t "$SESSION:0.0" -h
tmux split-window -t "$SESSION:0.1" -h
tmux split-window -t "$SESSION:0.2" -h
tmux split-window -t "$SESSION:0.3" -h
tmux split-window -t "$SESSION:0.4" -h

# 3. 균등 배치 후 main-vertical로 전환
tmux select-layout -t "$SESSION:0" even-horizontal
tmux select-layout -t "$SESSION:0" main-vertical
tmux set-option -t "$SESSION" main-pane-width 158

# 4. 파인 제목 표시
tmux set-option -t "$SESSION" pane-border-status top
tmux set-option -t "$SESSION" pane-border-format " #{pane_title} "
tmux set-option -t "$SESSION" allow-rename off

# 5. 각 파인에 이름 설정
tmux select-pane -t "$SESSION:0.0" -T "쫘"
tmux select-pane -t "$SESSION:0.1" -T "민준 아키텍트"
tmux select-pane -t "$SESSION:0.2" -T "지훈 개발자"
tmux select-pane -t "$SESSION:0.3" -T "수아 UI/UX디자이너"
tmux select-pane -t "$SESSION:0.4" -T "서연 리서처"
```

```
tmux select-pane -t "$SESSION:0.5" -T "태양 QA·리뷰어"

echo "✅ 레이아웃 구성 완료"
```

레이아웃 설계 원칙

파일 크기 배분

팀장(Pane 0)은 지시를 받고 팀에 전달하는 주 통신 창구이므로 화면을 넓게 사용합니다. `main-pane-width 158` 은 전체 화면(약 317 컬럼)의 절반입니다.

```
# 파일 너비 조정
tmux set-option -t team main-pane-width 158

# 더 넓게 (팀장 중심)
tmux set-option -t team main-pane-width 200
```

균등 배분 레이아웃

팀원 간 역할이 비슷하고 모든 파일을 동등하게 모니터링하려면 `even-horizontal` 을 사용합니다.

```
tmux select-layout -t team:0 even-horizontal
```

레이아웃 저장 및 복원

TMUX는 기본적으로 레이아웃을 저장하지 않습니다. 세션을 종료하면 레이아웃이 사라집니다. 셋업 스크립트를 사용해 언제든지 동일한 레이아웃을 재현합니다.

```
# 현재 레이아웃 확인 (저장용)
tmux list-windows -t team -F "#{window_layout}"

# 출력: main-vertical,317x85,158,0[317x16,158,0,0,317x16,158,17,1,...]
```

이 값을 저장해두면 `tmux select-layout -t team:0 "저장된레이아웃값"` 으로 정확한 파일 크기까지 복원할 수 있습니다.

요약

레이아웃 설계의 핵심은 역할에 맞는 공간 배분과 팀 전체를 한눈에 볼 수 있는 배치입니다. 다음
챕터에서는 이 레이아웃 위에 Claude Code를 자동으로 실행하는 방법을 설명합니다.

3-3. 각 파인에 Claude Code 자동 실행

레이아웃이 준비되었다면 이제 각 파인에 Claude Code를 자동으로 실행하는 방법을 구현합니다. 핵심은 Claude Code 시작 시 나타나는 대화 상자를 자동으로 처리하는 것입니다.

Claude Code 시작 시 나타나는 다이얼로그

Claude Code를 처음 실행하면 두 가지 대화 상자가 순서대로 나타납니다.

```
[다이얼로그 1] Do you trust the files in this folder?  
> Yes, I trust this folder (proceed)  
    No, exit  
  
[다이얼로그 2] Do you accept the terms of service?  
    No, exit  
> Yes, I accept
```

- 다이얼로그 1: Enter만 누르면 됩니다 (기본값이 "Yes").
- 다이얼로그 2: 기본값이 "No"이므로 아래 방향키를 한 번 누른 후 Enter가 필요합니다.

자동화할 때는 이 두 단계를 순서대로 처리해야 합니다.

파인 대기 함수

파인에 특정 텍스트가 나타날 때까지 기다리는 헬퍼 함수입니다.

```
# 특정 pane에 패턴이 나타날 때까지 대기  
# $1: pane 주소 (예: team:0.1)  
# $2: 대기할 텍스트 패턴  
# $3: 최대 대기 시간(초, 기본 30)  
wait_for_pane() {  
    local pane="$1"  
    local pattern="$2"  
    local timeout="${3:-30}"  
    local waited=0  
  
    while [ $waited -lt $timeout ]; do  
        if tmux capture-pane -t "$pane" -p 2>/dev/null | grep -q "$pattern"; then  
            return 0  
        fi  
        sleep 1  
        waited=$((waited+1))  
    done  
}
```



```

        waited=$((waited + 1))
    done
    return 1 # 타임아웃
}

```

파일별 Claude 실행 함수

```

# $1: 파일 주소 (예: team:0.2)
# $2: 모델 (기본값: claude-sonnet-4-6)
start_claude_in_pane() {
    local pane="$1"
    local model="${2:-claude-sonnet-4-6}"
    local claude_bin
    claude_bin="$(command -v claude)"

    # 이전 입력 초기화
    tmux send-keys -t "$pane" C-c 2>/dev/null
    sleep 0.3
    tmux send-keys -t "$pane" C-u 2>/dev/null
    sleep 0.2

    # Claude 실행
    tmux send-keys -t "$pane" \
        "cd /home/user && unset CLAUDECODE && $claude_bin --model $model --
dangerously-skip-permissions" \
        Enter

    # 다이얼로그 1: "trust this folder" → Enter
    if wait_for_pane "$pane" "trust this folder" 20; then
        tmux send-keys -t "$pane" Enter
        sleep 1
    fi

    # 다이얼로그 2: "I accept" → Down + Enter
    if wait_for_pane "$pane" "I accept" 20; then
        tmux send-keys -t "$pane" Down
        sleep 0.5
        tmux send-keys -t "$pane" Enter
        sleep 1
    fi

    # Claude 프롬프트(>) 나타날 때까지 대기
    wait_for_pane "$pane" ">" 30
}

```

전체 팀에 Claude 실행

6개 파인 모두에 순서대로 Claude를 실행합니다.

```
SESSION="team"

# 파인별 이름과 모델 배열
MEMBER_NAMES=("쫘" "민준" "지훈" "수아" "서연" "태양")
MEMBER_MODELS=(
    "claude-sonnet-4-6" # 0: 쫘
    "claude-opus-4-6"   # 1: 민준 (복잡한 설계 → Opus)
    "claude-sonnet-4-6" # 2: 지훈
    "claude-sonnet-4-6" # 3: 수아
    "claude-sonnet-4-6" # 4: 서연
    "claude-sonnet-4-6" # 5: 태양
)

for pane in 0 1 2 3 4 5; do
    model="${MEMBER_MODELS[$pane]}"
    echo -n "   Pane $pane (${MEMBER_NAMES[$pane]}): 시작 중..."

    start_claude_in_pane "$SESSION:0.$pane" "$model"

    # 성공 여부 확인
    if tmux capture-pane -t "$SESSION:0.$pane" -p 2>/dev/null | grep -q ">"; then
        echo "   🟢 준비 완료"
    else
        echo "   ⚠️ 타임아웃 - 수동 확인 필요"
    fi
done
```

이미 실행 중인 파인에 메시지 보내기

Claude가 실행 중인 파인에 메시지를 전송하는 방법입니다.

```
# 특정 파인에 메시지 전송 (줄바꿈 = Enter)
tmux send-keys -t team:0.1 "민준, 현재 아키텍처 검토해줘" Enter

# 여러 파인에 동시 전송 (브로드캐스트)
for pane in 0 1 2 3 4 5; do
    tmux send-keys -t "team:0.$pane" "모두 현재 상태 보고해줘" Enter
done
```

파일 상태 확인

각 파일이 Claude 프롬프트 상태인지 확인합니다.

```
for pane in 0 1 2 3 4 5; do
  content=$(tmux capture-pane -t "team:0.$pane" -p 2>/dev/null)
  if echo "$content" | grep -q ">"; then
    echo "  Pane $pane: ✅ Claude 대기 중"
  elif echo "$content" | grep -q "trust this folder"; then
    echo "  Pane $pane: ⚠️ 다이얼로그 1 대기 중"
  else
    echo "  Pane $pane: ❓ 상태 불명"
  fi
done
```

--dangerously-skip-permissions 주의사항

이 플래그는 파일 읽기/쓰기/실행 권한 요청을 자동 승인합니다. 자동화에 편리하지만 다음 사항을 인지하고 사용하세요.

- 신뢰할 수 있는 로컬 환경에서만 사용
- 외부에서 접근 가능한 서버에서는 주의
- 작업 완료 후 제한된 권한 모드로 전환 가능

요약

자동 실행의 핵심은 `wait_for_pane` 함수로 다이얼로그를 감지하고 자동 응답하는 것입니다. 다음 챕터에서는 각 에이전트에게 고유한 역할을 부여하는 `CLAUDE.md` 파일 작성법을 설명합니다.

3-4. CLAUDE.md로 팀원 역할 정의

CLAUDE.md 는 Claude Code가 프로젝트 디렉토리에서 실행될 때 자동으로 읽는 설정 파일입니다. 각 에이전트에게 고유한 역할, 행동 원칙, 커뮤니케이션 방식을 정의하는 핵심 파일입니다.

CLAUDE.md 로딩 우선순위

Claude Code는 시작 시 다음 순서로 **CLAUDE.md** 를 탐색하고 모두 로드합니다.

```
~/ .claude/CLAUDE.md           ← 전역 설정 (모든 프로젝트에 적용)
~/프로젝트/.claude/CLAUDE.md  ← 프로젝트 설정 (해당 디렉토리에만 적용)
~/프로젝트/CLAUDE.md          ← 프로젝트 루트 (전통적 위치)
```

멀티에이전트 팀에서는 전역 **~/ .claude/CLAUDE.md** 에 팀 공통 설정과 역할 라우팅 규칙을 정의합니다.

팀 공통 CLAUDE.md 예시

```
## 팀 공통 설정

## Bot Mode (최우선 규칙)

메시지가 `[CHANNEL]:[ID]` 접두사로 시작하면 다음을 실행합니다.

1. `{CHANNEL}` 과 `{ID}` 추출
2. 지시된 작업 수행
3. 완료 후 반드시 응답 전송:
   ```bash
 ## 응답은 Remote-Control을 통해 자동 전달됨
   ```
4. 모든 응답은  로 시작 (Claude CLI 식별)
5. 전송 완료 후 `Sent` 출력

### 응답 전송은 절대 생략 금지

- 성공/실패/오류 모두 반드시 전송
- 작업 완료 후 반드시 결과를 팀장에게 보고

---

## 브릿지 명령어 (응답 금지)
```

`@cc`, `@ccn`, `@ccu`, `/cc`, `/ccn`, `/ccu` 로 시작하는 메시지는
claude-bridge 플러그인이 처리합니다.

이 접두사를 보면:

- 절대 해석/처리/응답하지 말 것
- 반드시 이 텍스트만 출력:  Delivered to Claude CLI. Reply will arrive shortly.

개별 역할 정의: 팀장(쥔)

나의 역할: 쥔 (팀장)

나는 팀의 총괄 지휘자입니다.

핵심 원칙

- ****직접 작업 금지****: 코드 작성, 파일 수정, 명령 실행은 팀원에게 위임
- 역할: 지시 수령 → 분석 → 팀원 배분 → 결과 통합 → 보고

업무 처리 흐름

1. 사용자의 요청을 분석
2. 적절한 팀원을 선택
3. `tmux send-keys`로 지시 전달
4. 결과를 수집하여 사용자에게 보고

팀원 호출 방법

```bash

## 민준에게 아키텍처 설계 요청

`tmux send-keys -t team:0.1 "민준, [설계 내용]" Enter`

## 서연에게 리서치 요청

`tmux send-keys -t team:0.4 "서연, [조사 내용]" Enter`

<hr>

## 개별 역할 정의: 아키텍트(민준)

```markdown

나의 역할: 민준 (아키텍트)

나는 시스템 설계와 기술 방향을 담당합니다.

전문 영역

- 시스템 아키텍처 설계
- 기술 스택 선정
- API 설계 및 데이터 모델링
- 성능 및 확장성 검토

작업 완료 후

설계 결과를 팀장(쥔)에게 보고합니다.

```
```bash
```

```
tmux send-keys -t team:0.0 "쥔, 아키텍처 설계 완료: [요약]" Enter
```

<hr>

## @include를 통한 파일 참조

`CLAUDE.md`에서 다른 파일을 포함할 수 있습니다.

```
```markdown
```

```
# CLAUDE.md
```

```
@팀원-역할.md
```

```
@프로젝트-설정.md
```

이 기능을 활용하면 공통 설정과 역할별 설정을 파일로 분리해 관리할 수 있습니다.

파일 구조 예시

```
~/.claude/
```

```
|— CLAUDE.md          ← 메인 (공통 + @include)
```

```
|— roles/
```

```
|   |— jun.md          ← 쥔 역할 정의
```

```
|   |— minjun.md       ← 민준 역할 정의
```

```
|   |— ...
```

```
|— team-config.md     ← 팀 설정 공통
```

역할 정의 작성 팁

1. 역할 범위를 명확히

좋은 예

나는 UI/UX 디자이너입니다. 화면 설계, 컴포넌트 구조, 사용자 플로우를 담당합니다. 백엔드 코드는 작성하지 않습니다.

```
# 나쁜 예 (범위 불명확)
나는 뭘든 합니다.
```

2. 결과 보고 방법 포함

각 팀원의 역할 정의에 작업 완료 후 결과를 어떻게 보고하는지 명시합니다.

3. 우선순위 규칙 먼저

Bot Mode, 브릿지 명령어처럼 즉각 반응해야 하는 규칙을 파일 최상단에 배치합니다.

CLAUDE.md 적용 확인

Claude Code를 실행한 후 `/context` 명령으로 현재 로드된 설정을 확인합니다.

```
> /context
```

또는 직접 질문합니다.

```
> 너의 역할이 뭐야?
```

역할 정의가 올바르게 적용되었다면 설정한 역할에 맞는 답변이 나옵니다.

요약

`CLAUDE.md` 는 에이전트의 "직무 기술서"입니다. 역할 범위, 행동 원칙, 보고 방식을 명확히 정의할수록 에이전트가 의도한 대로 동작합니다. 다음 챕터에서는 이 모든 설정을 원클릭으로 실행하는 팀 셋업 스크립트를 완성합니다.

3-5. 팀 셋업 스크립트 작성

지금까지 배운 레이아웃 구성, Claude 자동 실행, 역할 정의를 하나의 스크립트로 통합합니다. 이 스크립트를 한 번 실행하면 팀 전체 환경이 자동으로 구성됩니다.

스크립트 전체 구조

```
setup-team.sh
|
├── [0단계] 사전 의존성 확인 (tmux, claude)
├── [1단계] 기존 세션 정리
├── [2단계] TMUX 세션 & 레이아웃 생성
├── [3단계] 파인 이름 설정
├── [4단계] 각 파인에 Claude 자동 실행
└── [5단계] 세션 접속
```

완성된 셋업 스크립트

```
#!/bin/bash
# setup-team.sh - Claude 멀티에이전트 팀 환경 자동 구성

set -e

GREEN='\033[0;32m'
CYAN='\033[0;36m'
YELLOW='\033[1;33m'
RED='\033[0;31m'
NC='\033[0m'

SESSION="team"

# ─── 유틸: 파인에 패턴이 나타날 때까지 대기 ───────────────────────────────────
wait_for_pane() {
    local pane="$1" pattern="$2" timeout="${3:-30}" waited=0
    while [ $waited -lt $timeout ]; do
        tmux capture-pane -t "$pane" -p 2>/dev/null | grep -q "$pattern" &&
        return 0
        sleep 1; waited=$((waited + 1))
    done
    return 1
}
```



```

# ——— 유틸: Claude 실행 + 다이얼로그 자동 처리 ———
start_claude_in_pane() {
    local pane="$1" model="${2:-claude-sonnet-4-6}"
    local claude_bin; claude_bin="$(command -v claude)"

    tmux send-keys -t "$pane" C-c 2>/dev/null; sleep 0.3
    tmux send-keys -t "$pane" C-u 2>/dev/null; sleep 0.2

    tmux send-keys -t "$pane" \
        "cd /home/user && unset CLAUDECODE && $claude_bin --model $model --
dangerously-skip-permissions" Enter

    # 다이얼로그 1: trust folder → Enter
    wait_for_pane "$pane" "trust this folder" 20 && {
        tmux send-keys -t "$pane" Enter; sleep 1
    }

    # 다이얼로그 2: terms of service → Down + Enter
    wait_for_pane "$pane" "I accept" 20 && {
        tmux send-keys -t "$pane" Down; sleep 0.5
        tmux send-keys -t "$pane" Enter; sleep 1
    }

    wait_for_pane "$pane" ">" 30
}

# ——— [0/4] 사전 요구사항 확인 ———
echo -e "${YELLOW}[0/4] 사전 요구사항 확인...${NC}"

MISSING=()
command -v tmux &>/dev/null || MISSING+=("tmux (sudo apt install -y tmux)")
command -v claude &>/dev/null || MISSING+=("claude (npm install -g @anthropic-ai/
claude-code)")

if [ ${#MISSING[@]} -gt 0 ]; then
    echo -e "${RED}❌ 누락된 의존성:${NC}"
    for m in "${MISSING[@]"; do echo "    - $m"; done
    exit 1
fi

echo "    ✅ tmux $(tmux -V | awk '{print $2}')"
echo "    ✅ claude $(claude --version 2>/dev/null | head -1)"

# ——— [1/4] 기존 세션 정리 ———

echo -e "\n${YELLOW}[1/4] 기존 세션 초기화...${NC}"
tmux has-session -t "$SESSION" 2>/dev/null && {
    tmux kill-session -t "$SESSION"
}

```

```

    echo "   기존 '$SESSION' 세션 종료"
}

# — [2/4] TMUX 세션 & 레이아웃 구성 —————
echo -e "\n${YELLOW}[2/4] TMUX 세션 & 레이아웃 구성...${NC}"

TERM_WIDTH=$(tput cols 2>/dev/null || echo 317)
TERM_HEIGHT=$(tput lines 2>/dev/null || echo 85)

tmux new-session -d -s "$SESSION" -x "$TERM_WIDTH" -y "$TERM_HEIGHT"

# 파인 5개 분할
tmux split-window -t "$SESSION:0.0" -h
tmux split-window -t "$SESSION:0.1" -h
tmux split-window -t "$SESSION:0.2" -h
tmux split-window -t "$SESSION:0.3" -h
tmux split-window -t "$SESSION:0.4" -h

# main-vertical 레이아웃 (팀장 왼쪽 넓게)
tmux select-layout -t "$SESSION:0" even-horizontal
tmux select-layout -t "$SESSION:0" main-vertical
tmux set-option -t "$SESSION" main-pane-width 158

# 파인 제목 표시 설정
tmux set-option -t "$SESSION" pane-border-status top
tmux set-option -t "$SESSION" pane-border-format " #{pane_title} "
tmux set-option -t "$SESSION" allow-rename off

# 파인 이름 설정
tmux select-pane -t "$SESSION:0.0" -T "쭈"
tmux select-pane -t "$SESSION:0.1" -T "민준 아키텍트"
tmux select-pane -t "$SESSION:0.2" -T "지훈 개발자"
tmux select-pane -t "$SESSION:0.3" -T "수아 UI/UX디자이너"
tmux select-pane -t "$SESSION:0.4" -T "서연 리서처"
tmux select-pane -t "$SESSION:0.5" -T "태양 QA·리뷰어"

echo "   ✅ 레이아웃 구성 완료 (6 panes)"

# — [3/4] Claude 자동 실행 —————
echo -e "\n${YELLOW}[3/4] Claude 실행 중... (파인당 최대 1분)${NC}"

MEMBER_NAMES=("쭈" "민준" "지훈" "수아" "서연" "태양")
MEMBER_MODELS=(
    "claude-sonnet-4-6"
    "claude-opus-4-6"
    "claude-sonnet-4-6"
    "claude-sonnet-4-6"
    "claude-sonnet-4-6"

```

```
"claude-sonnet-4-6"
)

for pane in 0 1 2 3 4 5; do
    echo -n "   Pane $pane (${MEMBER_NAMES[$pane]}): "
    start_claude_in_pane "$SESSION:0.$pane" "${MEMBER_MODELS[$pane]}"

    tmux capture-pane -t "$SESSION:0.$pane" -p 2>/dev/null | grep -q ">" \
        && echo -e "${GREEN}✅ 준비 완료${NC}" \
        || echo -e "${RED}⚠️ 타임아웃 - 수동 확인 필요${NC}"
done

# ─── [4/4] 완료

echo -e "\n${GREEN}"
echo "   ┌────────────────────────────────────────────────────────────────────────────────┐"
echo "   │   ✅ 팀 환경 구성 완료!   │"
echo "   └────────────────────────────────────────────────────────────────────────────────┘"
echo -e "${NC}"

# 터미널에서 직접 실행한 경우 자동 attach
[ -t 1 ] && tmux attach -t "$SESSION"
```

스크립트 실행

```
# 실행 권한 부여
chmod +x setup-team.sh

# 실행
bash setup-team.sh
```

실행 결과 확인

스크립트 완료 후 TMUX 세션에 접속합니다.

```
tmux attach -t team
```

각 파인 상단에 이름이 표시되고, 모든 파인에 Claude 프롬프트(>)가 나타나면 성공입니다.

특정 에이전트에 작업 전달

팀 환경이 실행 중이면 터미널 어디서든 아래 명령으로 에이전트에게 작업을 전달할 수 있습니다.

```
# 지훈에게 코드 작성 요청
tmux send-keys -t team:0.2 "지훈, main.py에 사용자 인증 함수 추가해줘" Enter

# 서연에게 리서치 요청
tmux send-keys -t team:0.4 "서연, Rust와 Go의 성능 비교 조사해줘" Enter

# 태양에게 코드 리뷰 요청
tmux send-keys -t team:0.5 "태양, 방금 작성된 auth.py 코드 리뷰해줘" Enter
```

요약

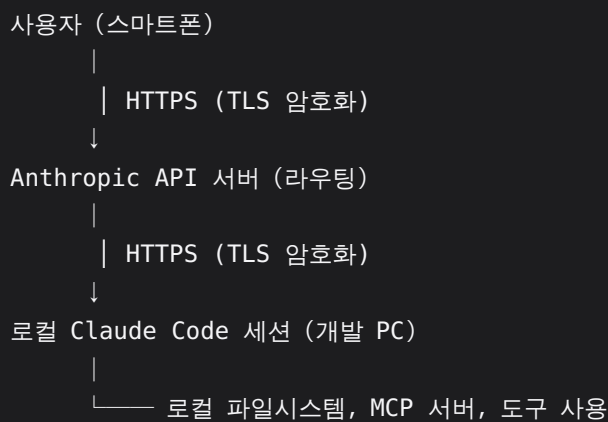
셋업 스크립트 하나로 팀 전체 환경을 재현할 수 있습니다. 다음 4장에서는 이 팀 환경을 스마트폰이나 다른 기기에서 원격으로 제어하는 **Remote-Control** 기능을 설명합니다.

4-1. Remote-Control 개요

Claude Code Remote Control은 로컬에서 실행 중인 Claude Code 세션을 다른 기기에서 실시간으로 제어할 수 있는 기능입니다. 개발 PC를 켜둔 채로 스마트폰이나 태블릿에서 작업을 이어가거나, 실행 중인 작업을 원격으로 감독할 수 있습니다.

핵심 원리

Remote Control의 가장 중요한 특징은 세션이 항상 로컬 머신에서만 실행된다는 점입니다.



클라우드로 이동하는 것은 통신 경로뿐입니다. 코드, 파일, 설정은 모두 로컬에 남아 있습니다.

Remote Control의 주요 특징

실시간 동기화

연결된 모든 기기에서 대화 내용이 실시간으로 동기화됩니다. PC에서 입력한 내용이 스마트폰에 즉시 반영되고, 반대로도 마찬가지입니다.

자동 재연결

네트워크가 끊겨도 재연결 후 자동으로 세션을 복구합니다. 10분 이내의 네트워크 단절은 자동 복구됩니다.

보안 통신

모든 트래픽은 TLS로 암호화됩니다. 로컬 머신에 인바운드 포트를 열 필요가 없어 방화벽 설정이 필요하지 않습니다.

활용 시나리오

시나리오	설명
자리 이동 중 작업 이어하기	데스크톱에서 시작한 작업을 이동 중 폰으로 계속
장시간 작업 모니터링	배포, 빌드, 테스트가 진행 중일 때 다른 기기에서 상태 확인
원격 권한 승인	자리를 비운 동안 Claude가 도구 실행 권한 요청 시 모바일로 승인
멀티 디바이스 협업	폰으로 입력, PC에서 코드 확인, 태블릿에서 최종 승인
푸시 알림	Claude가 입력을 기다릴 때 모바일 앱으로 알림 수신

Remote Control vs 일반 SSH 접속

많은 개발자가 SSH로 원격 서버에 접속하는 방식에 익숙합니다. Remote Control은 어떻게 다를까요?

비교 항목	SSH 접속	Remote Control
연결 방식	직접 TCP (포트 22)	Anthropic API 경유
방화벽 설정	포트 개방 필요	불필요 (HTTPS만 사용)
인증	SSH 키 / 비밀번호	Anthropic 계정 (OAuth)
모바일 앱	별도 SSH 클라이언트	Claude 공식 모바일 앱
알림 기능	없음	푸시 알림 지원

MCP 서버와의 차이

`claude mcp serve` 명령과 Remote Control은 다른 기능입니다.

비교	Remote Control	MCP Serve (<code>claude mcp serve</code>)
목적	세션을 다른 기기에서 제어	Claude Code를 MCP 도구 서버로 노출

비교	Remote Control	MCP Serve (<code>claude mcp serve</code>)
대상	claude.ai/code, 모바일 앱	Claude Desktop 등 MCP 클라이언트
사용 상황	원격에서 Claude와 대화	다른 앱에서 Claude의 도구를 사용

필요 조건

Remote Control을 사용하려면 다음이 필요합니다.

- ✓ Anthropic 계정 (claude.ai 로그인)
- ✓ 구독 플랜: Pro, Max, Team, Enterprise 중 하나
- ✓ claude auth login으로 OAuth 인증 완료 (API 키 방식 불가)
- ✓ 포트 443 아웃바운드 허용 (대부분 환경에서 기본 허용)

접속 방법 요약

Remote Control이 활성화된 세션에 다른 기기에서 접속하는 방법은 세 가지입니다.

1. 직접 URL: 브라우저에서 `https://claude.ai/code?session=<session-id>` 열기
2. QR 코드: 서버 모드에서 스페이스바 → 폰 카메라로 스캔
3. 세션 목록: `claude.ai/code` 에서 세션 이름으로 찾기 (컴퓨터 아이콘 + 초록 점)

요약

Remote Control은 로컬에서 실행, 원격에서 접근이라는 원칙을 유지하면서 어디서든 Claude Code 세션에 연결할 수 있게 해줍니다. 다음 챕터에서는 세 가지 활성화 방법을 상세히 설명합니다.

4-2. Remote Control 활성화 방법 3가지

Remote Control은 상황에 따라 세 가지 방법으로 활성화할 수 있습니다. 항상 켜두고 싶다면 전역 설정을, 특정 세션에만 적용하려면 CLI 플래그나 세션 내 명령을 사용합니다.

방법 1: settings.json 전역 설정

모든 Claude Code 세션에서 Remote Control을 자동 활성화합니다. 한 번만 설정하면 됩니다.

설정 파일 위치

```
~/.claude/settings.json
```

설정 추가

```
# settings.json이 없으면 생성, 있으면 편집
cat ~/.claude/settings.json 2>/dev/null || echo "{}"
```

`~/.claude/settings.json` 파일에 다음 내용을 추가합니다.

```
{
  "remoteControlAtStartup": true
}
```

이 설정이 `true` 이면 `claude` 명령을 실행할 때마다 자동으로 Remote Control이 활성화됩니다.

settings.json Remote Control 설정

/config 메뉴로 설정

직접 파일을 편집하는 대신 Claude Code 내부에서 설정할 수도 있습니다.

```
> /config
```

`/config` 메뉴에서 **Enable Remote Control for all sessions** 항목을 `true` 로 변경합니다.

방법 2: CLI 플래그 (일회성)

특정 세션에만 Remote Control을 활성화하려면 실행 시 플래그를 사용합니다.

CLI 플래그로 Remote Control 실행

인터랙티브 세션 + 원격 접근

```
# 기본 형식
claude --remote-control

# 단축 형식
claude --rc

# 세션 이름 지정
claude --remote-control "백엔드 API 리팩토링"
claude --rc "프론트엔드 작업"
```

세션 이름을 지정하면 claude.ai/code 세션 목록에서 쉽게 찾을 수 있습니다.

서버 모드 실행

```
# 기본 서버 모드 (다중 원격 세션 관리)
claude remote-control

# 세션 이름 접두사 지정
claude remote-control --remote-control-session-name-prefix myproject
```

서버 모드는 4-3 챕터에서 자세히 다룹니다.

방법 3: 실행 중인 세션에서 활성화

이미 실행 중인 Claude Code 세션에서 Remote Control을 켤 수 있습니다.

기본 활성화

```
> /remote-control
```

세션 이름 지정

```
> /remote-control 내 프로젝트 작업
```

명령을 입력하면 Claude가 Remote Control URL과 QR 코드를 제공합니다.

VS Code 확장에서도 동일

VS Code의 Claude 확장 내에서도 동일한 명령이 동작합니다.

```
/remote-control  
/rc
```

활성화 확인

Remote Control이 활성화되면 터미널에 다음과 같은 메시지가 표시됩니다.

```
Remote Control active  
Session: my-project-graceful-unicorn  
URL: https://claude.ai/code?session=abc123...  
  
Press SPACE to show QR code
```

접속 방법

활성화 후 다른 기기에서 세 가지 방법으로 접속합니다.

```
# 방법 1: URL 직접 접속  
https://claude.ai/code?session=<session-id>  
  
# 방법 2: QR 코드 (서버 모드에서 스페이스바)  
# → Claude 모바일 앱으로 스캔  
  
# 방법 3: claude.ai/code 세션 목록  
# → 세션 이름 검색 → 컴퓨터 아이콘 + 초록 점 선택
```

멀티에이전트 팀에서 활용

팀 셋업 스크립트에서 팀장(쥔) 파인만 Remote Control을 활성화하면 모바일에서 팀장에게 지시할 수 있습니다.

```
# 팀장 파인(0)에 Remote Control 포함하여 Claude 실행
tmux send-keys -t team:0.0 \
    "claude --rc '팀장 쥔' --dangerously-skip-permissions" Enter
```

세 가지 방법 비교

방법	적용 범위	사용 시점
<code>settings.json</code>	모든 세션	항상 켜두고 싶을 때
CLI 플래그 (<code>--rc</code>)	특정 세션 1개	그때그때 켜고 싶을 때
<code>/remote-control</code>	현재 실행 중 세션	실행 후 필요해졌을 때

요약

일상적인 개발 환경이라면 `settings.json` 에 `"remoteControlAtStartup": true` 를 추가하는 방법 1이 가장 간편합니다. 다음 챕터에서는 여러 원격 세션을 동시에 관리하는 서버 모드를 설명합니다.

4-3. 서버 모드 및 Spawn 모드

Remote Control의 서버 모드는 하나의 로컬 머신에서 여러 원격 세션을 동시에 관리할 수 있게 해줍니다. 팀원들이 각자 원격으로 연결하거나, 여러 작업을 병렬로 처리할 때 유용합니다.

서버 모드 실행

```
claude remote-control
```

서버 모드가 시작되면 다음과 같은 화면이 나타납니다.

```
Claude Remote Control Server
Session prefix: hostname-
Spawn mode: same-dir
Capacity: 0/32 sessions

Press SPACE to show QR code
Press W to toggle spawn mode (same-dir ↔ worktree)
```

서버 모드 실행 옵션 예시

서버 모드 옵션

```
claude remote-control \
  --name "내 개발 서버" \
  --remote-control-session-name-prefix myproject \
  --spawn worktree \
  --capacity 16 \
  --sandbox \
  --verbose
```

플래그	설명	기본값
<code>--name "제목"</code>	세션 목록 표시 이름	hostname 기반 자동 생성
<code>--remote-control-session-name-prefix</code>	세션 이름 접두사	hostname

플래그	설명	기본값
<code>--spawn</code>	세션 생성 방식	<code>same-dir</code>
<code>--capacity N</code>	최대 동시 세션 수	32
<code>--verbose</code>	상세 연결/세션 로그 출력	비활성
<code>--sandbox</code>	파일시스템·네트워크 격리	비활성
<code>--no-sandbox</code>	격리 명시적 비활성화	—

Spawn 모드

Spawn 모드는 새로운 원격 접속자가 연결될 때 세션을 어떻게 생성할지를 결정합니다.

same-dir (기본)

모든 원격 세션이 서버를 실행한 디렉토리를 공유합니다.

```
claude remote-control --spawn same-dir
```

```
개발자 A ┌
개발자 B ┤ — 같은 디렉토리 (/home/user/project)
개발자 C └
```

- 장점: 모든 세션이 같은 파일을 즉시 공유
- 단점: 동시에 같은 파일을 편집하면 충돌 가능

worktree

접속자마다 별도의 git worktree가 자동으로 생성됩니다.

```
claude remote-control --spawn worktree
```

```
개발자 A — /home/user/project-worktree-abc/ (브랜치: session-abc)
개발자 B — /home/user/project-worktree-def/ (브랜치: session-def)
개발자 C — /home/user/project-worktree-ghi/ (브랜치: session-ghi)
```

- 장점: 각 세션이 완전히 독립적으로 작업 가능
- 단점: git 저장소가 있어야 사용 가능, 디스크 공간 추가 사용

session

단일 세션만 허용합니다. 추가 접속 시도는 거부됩니다.

```
claude remote-control --spawn session
```

혼자만 사용하고 실수로 중복 접속하는 것을 방지할 때 유용합니다.

런타임 단축키

서버 모드 실행 중에 사용할 수 있는 키보드 단축키입니다.

키	동작
스페이스바	QR 코드 표시/숨기기
W	same-dir ↔ worktree 모드 전환
Q 또는 Ctrl+C	서버 종료

실용적인 서버 모드 예시

개인 개발 서버 (**worktree** 격리)

```
cd ~/projects/myapp

claude remote-control \
  --remote-control-session-name-prefix myapp \
  --spawn worktree \
  --capacity 8
```

스마트폰으로 QR 코드를 스캔하면 독립적인 작업 환경이 생성됩니다.

팀 공유 서버 (보안 강화)

```
claude remote-control \  
  --name "팀 공유 AI 서버" \  
  --spawn same-dir \  
  --capacity 16 \  
  --sandbox \  
  --verbose
```

단일 사용자 세션

```
claude remote-control \  
  --spawn session \  
  --name "쥘의 작업"
```

세션 현황 모니터링

서버 모드 실행 중 `--verbose` 플래그를 추가하면 접속/해제 이벤트를 실시간으로 볼 수 있습니다.

```
[2026-04-26 14:00:01] Session connected: myapp-swift-eagle (from mobile)  
[2026-04-26 14:02:15] Message received: "코드 리뷰해줘"  
[2026-04-26 14:05:30] Session disconnected: myapp-swift-eagle
```

TMUX 팀과 서버 모드 결합

멀티에이전트 팀에서 팀장 파인을 서버 모드로 실행하면 외부에서 팀장에게 접속할 수 있습니다.

```
# Pane 0 (팀장)에 서버 모드로 Claude 실행  
tmux send-keys -t team:0.0 \  
  "cd /home/user && claude remote-control --name '쥘-팀장' --spawn session" Enter
```

이제 스마트폰에서 claude.ai/code 에 접속해 `쥘-팀장` 세션을 선택하면 팀장에게 직접 지시를 내릴 수 있습니다.

요약

Spawn 모드	사용 상황
<code>same-dir</code>	같은 파일을 보며 협업
<code>worktree</code>	독립적인 병렬 작업
<code>session</code>	단일 사용자 전용

다음 챕터에서는 원격 세션 목록에서 쉽게 찾을 수 있도록 세션 이름을 설정하는 방법을 자세히 설명합니다.

4-4. 세션 이름 설정

원격에서 여러 Claude Code 세션에 접속할 때 세션 이름이 중요합니다. 이름을 잘 설정하면 claude.ai/code 세션 목록에서 원하는 세션을 빠르게 찾을 수 있습니다.

세션 이름 결정 우선순위

Claude Code는 세션 이름을 다음 순서로 결정합니다.

순위	소스	예시
1	실행 시 명시한 이름	<code>claude --rc "백엔드 API 작업"</code>
2	<code>/rename</code> 으로 변경한 이름	<code>/rename</code> 프론트엔드 리팩토링
3	대화 기록의 마지막 의미 있는 메시지	(자동)
4	<code>{hostname}-{랜덤형용사}-{랜덤명사}</code>	<code>mypc-graceful-unicorn</code>

이름 직접 지정

CLI 플래그로 지정

```
# --remote-control 또는 --rc 다음에 이름 입력
claude --remote-control "백엔드 API 리팩토링"
claude --rc "프론트엔드 로그인 화면"

# 서버 모드에서 --name 플래그
claude remote-control --name "개발 서버"
```

실행 중 세션에서 지정

```
> /remote-control 내 프로젝트 작업
```

`/remote-control` 명령어 뒤에 이름을 입력합니다.

실행 중 세션 이름 변경

```
> /rename 새 세션 이름
```

세션 이름 접두사 설정

`--remote-control-session-name-prefix` 는 자동 생성 이름의 앞부분을 고정합니다. 여러 세션이 같은 프로젝트를 알아보기 쉽게 만들 때 유용합니다.

```
# 플래그로 접두사 지정
claude remote-control --remote-control-session-name-prefix dev-machine
# 결과: dev-machine-graceful-unicorn
# 결과: dev-machine-swift-eagle
# 결과: dev-machine-calm-river
```

환경변수로 설정

매번 플래그를 입력하는 번거로움을 없애려면 환경변수를 사용합니다.

```
# ~/.bashrc 또는 ~/.zshrc에 추가
export CLAUDE_REMOTE_CONTROL_SESSION_NAME_PREFIX=myproject

# 적용
source ~/.bashrc

# 이후 접두사 없이 실행해도 자동 적용
claude remote-control
# 결과: myproject-gentle-forest
```

멀티에이전트 팀에서 세션 이름 관리

팀 환경에서는 각 파인(에이전트)의 세션 이름을 역할로 지정하면 원격에서 누구에게 접속하는지 즉시 알 수 있습니다.

```
# 팀 셋업 스크립트에서 각 파인 시작 시
# 파인 0: 팀장
tmux send-keys -t team:0.0 \
    "claude --rc '팀장-쥔' --dangerously-skip-permissions" Enter

# 파인 1: 아키텍트
```

```
tmux send-keys -t team:0.1 \
    "claude --rc '아키텍트-민준' --dangerously-skip-permissions" Enter

# 파인 2: 개발자
tmux send-keys -t team:0.2 \
    "claude --rc '개발자-지훈' --dangerously-skip-permissions" Enter
```

이렇게 하면 claude.ai/code 에서 세션 목록을 열었을 때 다음과 같이 표시됩니다.

- 팀장-쥔 (컴퓨터 아이콘 + 초록 점 = 활성화)
- 아키텍트-민준
- 개발자-지훈
- 디자이너-수아
- 리서처-서연
- 리뷰어-태양

환경변수 기반 접두사와 역할 이름 결합

```
# .env 또는 셋업 스크립트에서
export CLAUDE_REMOTE_CONTROL_SESSION_NAME_PREFIX="team"

# 각 파인에서 역할 이름으로 시작
claude --rc "pane0-팀장" # → team-pane0-팀장 (접두사 + 이름)
```

세션 이름 모범 사례

프로젝트 + 역할 조합

```
myapp-backend-dev
myapp-frontend-design
myapp-architect
```

날짜 포함 (일시적 작업)

```
claude --rc "$(date +%Y%m%d)-데이터마이그레이션"
# 결과: 20260426-데이터마이그레이션
```

짧고 명확하게

세션 목록에서 잘리지 않도록 이름은 20자 이내로 유지합니다.

세션 이름 확인

현재 세션 이름은 `/status` 명령으로 확인합니다.

```
> /status

Session: 팀장-쫘
Remote Control: Active
Model: claude-sonnet-4-6
```

요약

세션 이름 설정 방법 요약:

```
# 일회성 이름 지정
claude --rc "세션 이름"

# 접두사 설정 (플래그)
claude remote-control --remote-control-session-name-prefix 접두사

# 접두사 설정 (환경변수, 영구)
export CLAUDE_REMOTE_CONTROL_SESSION_NAME_PREFIX=접두사

# 실행 중 이름 변경
/rename 새이름
```

다음 챕터에서는 프로그램 방식으로 Claude를 제어할 수 있는 **Stream JSON** 모드를 설명합니다.

4-5. Stream JSON 제어

Remote Control이 사람이 다른 기기에서 직접 접속하는 방식이라면, **Stream JSON**은 프로그램이 Claude Code를 자동으로 제어하는 방식입니다. 스크립트, 파이프라인, 다른 애플리케이션에서 Claude와 구조화된 방식으로 통신할 수 있습니다.

Stream JSON 모드의 두 가지 방향

방향	플래그	용도
출력 스트리밍	<code>--output-format stream-json</code>	Claude 응답을 JSON 스트림으로 받기
입력 스트리밍	<code>--input-format stream-json</code>	JSON으로 메시지를 Claude에게 전송

출력: Stream JSON

기본 사용법

```
claude -p "파이썬으로 피보나치 함수를 작성해줘" \  
  --output-format stream-json \  
  --verbose \  
  --include-partial-messages
```

`-p` 플래그는 프롬프트를 직접 전달하고 비대화형 모드로 실행합니다.

출력 형식 (NDJSON)

각 줄이 하나의 JSON 객체인 NDJSON(Newline Delimited JSON) 형식입니다.

```
{"type":"stream_event","event":{"type":"message_start","message":  
{"id":"msg_01...","type":"message","role":"assistant","content":  
[]},"session_id":"uuid","uuid":"event-uuid"}  
{"type":"stream_event","event":{"type":"content_block_delta","delta":  
{"type":"text_delta","text":"def "},"session_id":"uuid","uuid":"event-uuid"}  
{"type":"stream_event","event":{"type":"content_block_delta","delta":  
{"type":"text_delta","text":"fibonacci"},"session_id":"uuid","uuid":"event-  
uuid"}  
...
```

```
{"type":"result","subtype":"success","result":"def fibonacci(n):  
\n    ...","session_id":"uuid"}
```

jq로 텍스트만 추출

```
claude -p "간단한 인사말 작성해줘" \  
  --output-format stream-json \  
  --verbose \  
  --include-partial-messages | \  
jq -rj 'select(  
  .type == "stream_event" and  
  .event.delta.type? == "text_delta"  
) | .event.delta.text'
```

출력:

```
안녕하세요! 반갑습니다.
```

최종 결과만 추출

```
claude -p "2 + 2는?" \  
  --output-format stream-json | \  
jq -r 'select(.type == "result") | .result'
```

단일 JSON 출력 (`--output-format json`)

스트리밍이 필요 없고 최종 결과만 필요하다면 단일 JSON 출력을 사용합니다.

```
claude -p "Node.js 버전 확인 방법" --output-format json
```

응답:

```
{  
  "result": "node --version 명령을 실행하면 됩니다.",  
  "session_id": "uuid-abc123",  
  "usage": {  
    "input_tokens": 15,  
    "output_tokens": 12,  
    "cache_read_tokens": 0,  
  }  
}
```

```
    "cache_creation_tokens": 0
  }
}
```

JSON Schema로 구조화된 출력

```
claude -p "다음 텍스트에서 이름 목록을 추출해줘: 앨런, 민준, 서연이 회의에 참석했다" \
  --output-format json \
  --json-schema '{
    "type": "object",
    "properties": {
      "names": {
        "type": "array",
        "items": {"type": "string"}
      }
    }
  }'
```

응답:

```
{
  "result": {"names": ["앨런", "민준", "서연"]},
  "session_id": "uuid",
  "usage": {...}
}
```

입력: Stream JSON

입력도 JSON 형식으로 보낼 수 있습니다. 복잡한 메시지 구조나 자동화 파이프라인에 유용합니다.

```
claude -p \
  --input-format stream-json \
  --output-format stream-json \
  --replay-user-messages
```

이 명령을 실행하면 stdin에서 JSON을 읽습니다. 아래 형식으로 입력합니다.

```
echo '{"type":"user_message","content":"안녕하세요!","uuid":"msg-001"}' | \
  claude -p --input-format stream-json --output-format stream-json
```

입력 메시지 구조:

```
{
  "type": "user_message",
  "content": "메시지 내용",
  "uuid": "고유-이벤트-id"
}
```

플래그	설명
<code>--input-format stream-json</code>	stdin에서 NDJSON으로 입력 받기
<code>--replay-user-messages</code>	수신한 사용자 메시지를 stdout에 에코 (수신 확인용)
<code>--include-partial-messages</code>	부분 토큰 스트림 이벤트 포함

실용 예제: 파이프라인에서 활용

파일 내용을 **Claude**에게 분석 요청

```
cat main.py | \
  claude -p "이 코드의 버그를 찾아줘" \
  --output-format json | \
  jq -r '.result'
```

여러 질문을 순차 처리

```
#!/bin/bash
questions=(
  "파이썬이란 무엇인가?"
  "Node.js의 장점은?"
  "Rust가 인기 있는 이유는?"
)

for q in "${questions[@]}; do
  echo "=== $q ==="
  claude -p "$q" --output-format json | jq -r '.result'
  echo ""
done
```


응답을 파일로 저장

```
claude -p "README.md 초안 작성해줘" \  
  --output-format json | \  
  jq -r '.result' > README.md  
  
echo "README.md 생성 완료"
```

Remote Control과 Stream JSON 비교

비교	Remote Control	Stream JSON
사용자	사람 (모바일/웹)	프로그램/스크립트
입력 방식	인터랙티브 대화	CLI 플래그, stdin
출력 방식	실시간 UI	NDJSON, JSON
세션 유지	영구 세션	요청별 실행
자동화	어려움	쉬움

요약

Stream JSON은 Claude를 자동화 도구로 활용할 때 핵심 기능입니다. 셸 스크립트, CI/CD 파이프라인, 배치 처리에서 Claude의 응답을 구조화된 데이터로 처리할 수 있습니다. 다음 챕터에서는 Remote Control 사용 시 중요한 보안 설정을 설명합니다.

4-6. 보안 설정 및 인증 요구사항

Remote Control은 외부에서 Claude Code를 원격 제어하는 기능이다. 편리한 만큼 보안 설정을 소홀히 하면 의도하지 않은 접근이 발생할 수 있다. 이 절에서는 Remote Control의 보안 모델과 권장 설정을 다룬다.

인증 구조

Remote Control은 Anthropic 계정 인증을 기반으로 동작한다. 연결 흐름은 다음과 같다.

```
모바일/웹 클라이언트
|
▼ (Anthropic 계정 로그인)
Anthropic 인증 서버
|
▼ (세션 토큰 발급)
Claude Code 서버 모드
|
▼ (도구 실행)
로컬 머신
```

핵심 원칙: 동일한 **Anthropic** 계정으로 로그인한 기기만 해당 세션에 접근할 수 있다. 타인이 세션 ID를 알더라도 계정 인증 없이는 연결이 불가능하다.

권한 모드 (Permission Mode)

Claude Code는 도구 실행 시 권한 확인을 거친다. Remote Control 환경에서는 이 권한 모드가 더욱 중요하다.

모드	설명	Remote Control 권장 여부
<code>default</code>	읽기 작업 자동 허용, 쓰기/실행은 승인 필요	[권장]
<code>plan</code>	코드 읽기만 허용, 모든 수정은 승인 필요	[높은 보안]
<code>bypassPermissions</code>	모든 작업 자동 허용	[비권장]

모바일에서의 권한 승인

Remote Control로 접속하면 Claude가 도구를 실행하기 전에 모바일 기기로 승인 요청이 전달된다.

Claude: "git push origin main을 실행하겠습니다. 승인하시겠습니까?"

↓

모바일 화면: [승인] [거부]

↓

승인 → 명령 실행

거부 → 작업 중단

`bypassPermissions` 모드는 이 승인 단계를 건너뛰므로 Remote Control 환경에서는 사용하지 않는 것이 좋다.

settings.json 보안 설정

허용 명령어 제한

`settings.json` 에서 자동 허용할 도구와 명령어를 명시적으로 지정한다.

```
{
  "permissions": {
    "allow": [
      "Bash(git status)",
      "Bash(git diff*)",
      "Bash(git log*)",
      "Bash(npm test)",
      "Bash(npm run lint)",
      "Read"
    ],
    "deny": [
      "Bash(rm -rf*)",
      "Bash(sudo*)",
      "Bash(curl*|*>*)"
    ]
  }
}
```

설정

효과

`allow`

나열된 패턴의 도구는 승인 없이 자동 실행

설정	효과
<code>deny</code>	나열된 패턴은 실행 자체를 차단

읽기 전용 명령어(`git status` , `git log` , `Read`)는 자동 허용하되, 시스템 변경 명령어(`rm` , `sudo`)는 차단하는 것이 기본 원칙이다.

네트워크 보안

SSH 터널을 통한 접속

Remote Control은 Anthropic 서버를 경유하므로 별도의 포트 개방이 필요 없다. 하지만 Claude Code 서버 모드 자체는 로컬에서 실행되므로, 서버가 실행되는 머신의 보안도 중요하다.

```
# WSL2 환경에서 SSH를 통한 접근 제한
sudo ufw allow from 192.168.1.0/24 to any port 22
sudo ufw enable
```

세션 격리

여러 팀원이 같은 머신에서 각자 Claude Code를 실행할 경우, 세션은 Anthropic 계정 단위로 격리된다. A의 계정으로 생성된 세션은 B의 계정으로 접근할 수 없다.

팀 환경 보안 체크리스트

팀 에이전트를 운용할 때 확인해야 할 보안 항목이다.

1. 환경 변수 보호

```
# .env 파일은 Claude가 읽지 못하도록 설정
echo ".env" >> .claudeignore
echo ".env.local" >> .claudeignore
echo "credentials/" >> .claudeignore
```

`.claudeignore` 파일은 `.gitignore` 와 같은 형식으로, Claude Code가 접근하지 않을 파일을 지정한다.

2. API 키 노출 방지

```
{
  "permissions": {
    "deny": [
      "Bash(cat*credentials*)",
      "Bash(cat*.env*)",
      "Bash(echo*KEY*)",
      "Bash(echo*TOKEN*)"
    ]
  }
}
```

3. Git 보호

```
{
  "permissions": {
    "deny": [
      "Bash(git push --force*)",
      "Bash(git reset --hard*)",
      "Bash(git branch -D main)"
    ]
  }
}
```

위험한 Git 명령어를 차단하여 실수로 인한 코드 손실을 방지한다.

4. 파일 시스템 경계

```
# Claude Code가 작업할 디렉토리 명시
claude --project-dir /home/user/my-project
```

`--project-dir` 플래그로 Claude가 접근할 수 있는 디렉토리 범위를 지정한다. 이 범위 밖의 파일은 읽거나 수정할 수 없다.

보안 사고 대응

Remote Control 세션에서 의심스러운 동작이 감지되면 다음과 같이 대응한다.

즉시 차단

```
# 1. Claude Code 프로세스 종료
pkill -f "claude"

# 2. TMUX 세션 종료
tmux kill-session -t team

# 3. Anthropic 계정에서 세션 로그아웃
# (claude.ai → 설정 → 세션 관리)
```

사후 점검

```
# 최근 실행된 명령어 확인
cat ~/.claude/projects/*/conversations/*.jsonl | \
    grep "tool_use" | tail -50

# 파일 변경 이력 확인
git log --oneline -20
git diff HEAD~5
```

요약

Remote Control의 보안은 Anthropic 계정 인증, 권한 모드, `settings.json`의 allow/deny 규칙, 그리고 `.claudeignore` 파일의 조합으로 구성된다. 기본 원칙은 최소 권한: 읽기는 자동 허용하되, 쓰기와 실행은 명시적으로 승인하거나 차단하는 것이다. 다음 장에서는 이 설정을 기반으로 모바일에서 실제로 팀을 제어하는 방법을 다룬다.

5-1. Claude 모바일 앱 설치 및 계정 연결

이 장에서 배울 내용

스마트폰으로 PC의 Claude Code 세션을 제어하려면 먼저 Claude 공식 모바일 앱을 설치하고, 데스크톱과 동일한 계정으로 로그인해야 합니다. 이 장에서는 앱 설치부터 계정 연결까지 단계별로 안내합니다.

준비 사항

Remote Control 기능을 사용하려면 다음 조건을 충족해야 합니다.

항목	요구 사항
Claude 구독	Pro, Max, Team, Enterprise 중 하나
계정	claude.ai 계정 (API 키 계정 불가)
앱 버전	최신 버전의 Claude 모바일 앱
네트워크	인터넷 연결 (Wi-Fi 또는 모바일 데이터)

중요: Remote Control은 API 키 방식 로그인을 지원하지 않습니다. 반드시 claude.ai 계정으로 로그인해야 합니다.

iOS 설치 (iPhone / iPad)

Step 1 — App Store 열기

iPhone 또는 iPad에서 App Store를 실행합니다.

Step 2 — "Claude" 검색

하단 검색 탭을 누르고 검색창에 **Claude**를 입력합니다. Anthropic이 개발한 공식 앱인지 확인하세요. 개발사가 **Anthropic**으로 표시된 앱을 선택합니다.

Step 3 — 다운로드 및 설치

받기 버튼을 탭합니다. Face ID 또는 Touch ID로 인증하면 다운로드가 시작됩니다.

Android 설치

Step 1 — Google Play Store 열기

홈 화면 또는 앱 서랍에서 Play Store를 실행합니다.

Step 2 — "Claude" 검색 및 설치

검색창에 **Claude**를 입력하고, Anthropic의 공식 앱을 찾아 설치 버튼을 탭합니다.

삼성/갤럭시 사용자: Galaxy Store에도 Claude 앱이 있지만, 최신 Remote Control 기능을 보장하려면 Google Play Store 버전을 권장합니다.

계정 로그인

앱 설치 후 계정을 연결합니다.

Step 1 — 앱 실행 및 로그인 화면 진입

Claude 앱을 처음 실행하면 환영 화면이 나타납니다. **Continue** 또는 로그인 버튼을 탭합니다.

Step 2 — 로그인 방식 선택

Google, Apple 계정 또는 이메일로 로그인할 수 있습니다. 반드시 데스크톱 **Claude Code**와 동일한 계정을 선택해야 세션이 연결됩니다.

Step 3 — 인증 완료

선택한 방식으로 인증을 완료하면 Claude 채팅 화면이 나타납니다. 상단 또는 하단 메뉴에 **Code** 탭이 보이면 Remote Control 기능에 접근할 준비가 된 것입니다.

계정 연결 확인

로그인 후 데스크톱과 같은 계정인지 확인합니다.

1. 앱 우측 상단 프로필 아이콘을 탭합니다.
2. **Settings** → **Account** 메뉴에서 이메일 주소를 확인합니다.
3. 데스크톱 터미널에서 `claude /status` 를 입력해 동일한 계정인지 비교합니다.

문제 해결

앱에 **Code** 탭이 보이지 않는 경우

- Pro 이상 구독 여부를 확인합니다. Free 플랜은 Remote Control을 지원하지 않습니다.
- 앱을 최신 버전으로 업데이트합니다.

로그인 오류가 발생하는 경우

- claude.ai 웹사이트에서 먼저 로그인이 되는지 확인합니다.
- 앱을 완전히 종료 후 재시작합니다.

"Full-scope login required" 메시지

- API 키로 로그인을 시도한 경우 이 오류가 발생합니다. Google 또는 이메일 방식으로 다시 로그인하세요.
-

다음 장 미리보기

앱 설치와 로그인이 완료되었습니다. 다음 장에서는 QR 코드를 스캔해 PC의 Claude Code 세션에 즉시 연결하는 방법을 알아봅니다.

모바일 앱으로 스캔하세요

일반 세션에서 **Remote Control**을 켜 경우

`/remote-control` 명령으로 현재 세션을 원격 제어 가능 상태로 만들면, 세션 URL이 표시됩니다. 이 URL을 모바일 브라우저로 직접 열 수도 있지만, QR 코드를 원한다면 다음 방법을 사용합니다.

터미널에서 별도의 서버 모드를 띄운 뒤 스페이스바로 QR 코드를 표시하는 것이 가장 간편합니다.

모바일 앱에서 QR 코드 스캔하기

iOS (iPhone / iPad)

Step 1 — Code 탭 이동

앱 하단 탭 바에서 **Code** 아이콘을 탭합니다.

Step 2 — QR 스캔 버튼 탭

Code 화면 우측 상단에 있는 **QR 코드 스캔** 아이콘(카메라 또는 격자 모양)을 탭합니다.

Step 3 — 카메라로 QR 코드 향하기

카메라 뷰파인더가 열립니다. PC 터미널에 표시된 QR 코드를 화면 중앙에 맞춥니다. 잠시 후 자동으로 인식됩니다.

Step 4 — 연결 확인

인식 즉시 세션 연결 화면으로 전환됩니다. PC에서 실행 중인 Claude Code의 현재 대화 내용이 모바일 화면에 나타나면 연결 성공입니다.

Android

Step 1 — Code 탭으로 이동

앱 하단 내비게이션에서 **Code**를 탭합니다.

Step 2 — 연결 추가

화면 우측 상단의 **+** 버튼 또는 세션 연결 옵션을 탭합니다.

Step 3 — QR 코드 스캔 선택

연결 방법 선택 시트에서 **QR 코드 스캔**을 선택합니다.

Step 4 — 스캔 및 연결

카메라를 PC 터미널의 QR 코드에 맞추면 자동 인식 후 세션이 열립니다.

연결 후 화면 구성

세션 연결에 성공하면 다음 요소가 화면에 나타납니다.

구성 요소	설명
세션 이름	상단 타이틀바 (예: "dev-machine-elegant-fox")
대화 기록	PC에서 진행 중인 대화 내용 실시간 표시
입력창	하단 텍스트 입력 필드
상태 표시	연결 상태 아이콘 (초록색 = 연결됨)

QR 코드가 작동하지 않을 때

카메라가 인식하지 못하는 경우

- 터미널 폰트 크기를 키워 QR 코드를 더 크게 표시합니다.
- 방 조명을 밝히거나 모니터 밝기를 높입니다.
- 카메라와 화면 사이 거리를 15~30cm 정도로 조정합니다.

"세션을 찾을 수 없음" 오류

- PC와 모바일이 동일한 claude.ai 계정으로 로그인되어 있는지 확인합니다.
- PC에서 Remote Control이 여전히 실행 중인지 확인합니다 (터미널 창이 닫히지 않았는지).

스캔은 됐지만 연결이 안 되는 경우

- 인터넷 연결 상태를 확인합니다.
 - PC 방화벽이 443 포트(HTTPS) 아웃바운드를 허용하는지 확인합니다.
-

다음 장 미리보기

QR 코드 연결 외에도, 세션 목록 화면에서 원하는 팀 에이전트 세션을 골라 연결하는 방법이 있습니다. 다음 장에서 세션 목록 탐색 방법을 알아봅니다.

5-3. 세션 목록에서 팀 에이전트 선택

이 장에서 배울 내용

QR 코드 없이도 세션 목록 화면에서 원하는 세션을 직접 찾아 연결할 수 있습니다. 특히 여러 팀 에이전트를 동시에 운영할 때, 세션 목록을 통해 각 에이전트의 상태를 한눈에 확인하고 빠르게 전환하는 방법을 배웁니다.

세션 목록 화면 열기

iOS / Android 공통

Claude 모바일 앱에서 하단 탭의 **Code** 를 탭하면 세션 목록 화면이 열립니다.

세션 목록 화면 구성

세션 목록에는 현재 Remote Control 이 활성화된 모든 세션이 표시됩니다.

각 항목의 구성 요소:

아이콘/요소	의미
[모니터 아이콘]	Remote Control 세션임을 표시
● 초록색 점	세션 활성화 중 (연결 가능)
○ 빈 점	입력 대기 중 (Claude가 답변을 기다림)
회색 / 없음	오프라인 또는 Remote Control 비활성
세션 이름	PC에서 지정한 이름 또는 자동 생성 이름
부제목	현재 작업 내용 (마지막 메시지 요약)
시간	마지막 활동 시각

팀 에이전트 세션 찾기

여러 팀원(에이전트)이 각자의 Claude Code 세션을 Remote Control 로 노출하고 있다면, 세션 이름으로 구분합니다.

세션 이름 규칙 설정 (PC에서)

팀 에이전트별로 알아보기 쉬운 이름을 붙이면 모바일에서 빠르게 찾을 수 있습니다.

```
# 에이전트별 세션 이름 지정 예시
claude remote-control --name "민준-PM-서버"
claude remote-control --name "서연-리서처"
claude remote-control --name "태양-리뷰어"

# 또는 접두사를 사용해 자동 이름 생성
claude remote-control --remote-control-session-name-prefix team-minjun
# → team-minjun-swift-otter 같은 이름 자동 생성
```

모바일에서 세션 검색

세션이 많을 경우, 목록 상단의 검색창을 이용합니다.

세션 이름, 부제목, 호스트명 등 모든 텍스트를 검색할 수 있습니다.

세션 선택 및 연결

원하는 세션을 탭하면 즉시 해당 세션에 연결됩니다.

Step 1 — 목록에서 세션 탭

연결하려는 세션 항목을 탭합니다.

Step 2 — 세션 화면 진입

해당 에이전트의 대화 화면이 열립니다. PC에서 진행 중인 실시간 대화 내용이 표시됩니다.

여러 세션 간 전환

모바일에서는 여러 세션을 동시에 열어두고 빠르게 전환할 수 있습니다.

iOS: 상단 타이틀 영역을 탭하거나 왼쪽 스와이프로 세션 목록으로 돌아갑니다.

Android: 시스템 뒤로가기 버튼 또는 앱 내 뒤로가기 화살표를 탭합니다.

세션 상태 해석 가이드

상태 표시	의미	취할 행동
초록 점 + "실행 중"	Claude가 작업을 처리하는 중	진행 상황 확인
초록 점 + "응답 대기 중"	Claude가 사용자 입력을 기다림	지시 입력
노란 점 + "도구 승인 필요"	실행 권한 승인 대기	5-4장 참고
회색 + "오프라인"	PC 세션이 종료되었거나 네트워크 불량	PC 상태 확인

세션이 목록에 보이지 않을 때

PC에서 확인할 사항:

- Remote Control이 활성화되어 있는지 확인합니다. (`/remote-control` 명령 실행 또는 설정에서 `remoteControlAtStartup: true` 확인)
- 터미널 창이 닫히지 않았는지 확인합니다.

모바일에서 확인할 사항:

- 목록 화면을 아래로 당겨 새로고침합니다.
- 검색창의 텍스트를 지우고 전체 목록을 확인합니다.
- 데스크톱과 동일한 claude.ai 계정으로 로그인되어 있는지 확인합니다.

다음 장 미리보기

세션에 연결했다면, 이제 모바일에서 직접 지시를 입력하거나 Claude가 요청하는 도구 실행을 승인할 수 있습니다. 다음 장에서 모바일 제어의 핵심 기능을 알아봅니다.

5-4. 모바일에서 지시 전달 및 도구 승인

이 장에서 배울 내용

세션에 연결된 후, 모바일에서 할 수 있는 두 가지 핵심 작업을 배웁니다. 첫째, 텍스트나 음성으로 Claude에게 지시를 입력하는 방법. 둘째, Claude가 파일 수정이나 명령어 실행을 요청할 때 스마트폰에서 직접 승인하거나 거부하는 방법입니다.

지시 입력하기

텍스트 입력

세션 화면 하단의 입력창을 탭하고 지시 사항을 입력합니다.

입력 후 전송 버튼(→ 화살표)을 탭하면 PC의 Claude Code에 메시지가 전달됩니다. PC 모니터에서도 실시간으로 같은 메시지가 나타납니다.

```
[모바일 입력] "현재 main.py 파일에 있는 버그 수정해줘"
      ↓
[PC 터미널] 메시지 수신 → Claude가 작업 시작
      ↓
[모바일 화면] Claude 응답 실시간 스트리밍
```

음성 입력 (iOS / Android)

긴 지시를 타이핑하기 번거로울 때 음성 입력을 활용합니다.

iOS:

1. 키보드의 마이크 아이콘을 탭합니다.
2. 지시 사항을 말합니다.
3. 텍스트로 변환된 내용을 확인하고 전송합니다.

Android:

1. 입력창 옆 마이크 아이콘을 탭합니다.
2. 음성 입력 후 자동 변환된 텍스트를 확인합니다.
3. 전송 버튼을 탭합니다.

팁: 한국어, 영어 모두 정확하게 인식됩니다. 명령어나 파일명이 포함된 경우에는 텍스트 입력이 더 정확합니다.

도구 승인 처리

Claude Code는 파일 쓰기, 터미널 명령어 실행 등 중요한 작업을 하기 전에 사용자 승인을 요청합니다. 모바일에서도 이 승인을 처리할 수 있습니다.

승인 요청 알림 확인

Claude가 승인을 기다리는 동안 세션 상태가 "도구 승인 필요"로 변경됩니다. 앱 아이콘에도 배지(숫자)가 표시됩니다.

승인 화면 구성

해당 세션에 진입하면 승인 요청 카드가 표시됩니다.

요소	설명
도구 이름	Bash, Edit, Write 등 Claude가 사용하려는 도구
실행 내용	실행할 명령어 또는 변경할 파일 내용
설명	Claude가 이 작업이 필요한 이유
거부 버튼	해당 작업을 취소하고 Claude에게 알림
승인 버튼	작업 실행 허가

승인하기

승인 버튼을 탭하면 PC에서 즉시 해당 도구가 실행됩니다.

거부하기

거부 버튼을 탭하면 Claude에게 "사용자가 이 작업을 허용하지 않았습니다"라는 메시지가 전달됩니다. Claude는 대안을 제시하거나 작업 방향을 수정합니다.

"이 세션에서 항상 허용" 옵션

반복적으로 같은 도구 유형을 승인하는 경우, 세션 내 항상 허용 옵션을 활성화할 수 있습니다. 이 설정은 현재 세션에만 적용되며, 세션이 종료되면 초기화됩니다.

보안 주의: 항상 허용 옵션은 편의성을 높이지만, Claude가 실행하는 모든 명령어를 검토할 수 없게 됩니다. 프로덕션 환경이나 중요한 파일이 있는 디렉토리에서는 사용하지 않는 것을 권장합니다.

작업 진행 상황 모니터링

지시를 전달하거나 승인을 완료한 후, Claude의 작업 진행 상황을 모바일에서 실시간으로 확인합니다.

응답 스트리밍: Claude의 답변이 토큰 단위로 실시간 표시됩니다.

도구 실행 결과: Bash 명령어 출력, 파일 편집 내용 등이 접이식 카드로 표시됩니다.

도구 실행 결과 카드를 탭하면 전체 내용을 확인할 수 있습니다.

사용 가능한 모바일 명령어

일부 Claude Code 명령어는 모바일에서도 사용할 수 있습니다.

명령어	기능
<code>/compact</code>	대화 컨텍스트 압축
<code>/clear</code>	대화 내용 초기화
<code>/context</code>	현재 컨텍스트 사용량 확인
<code>/usage</code>	토큰 사용량 확인
<code>/exit</code>	세션 종료
<code>/recap</code>	작업 요약 요청

참고: `/mcp` , `/plugin` , `/resume` 처럼 인터랙티브 선택이 필요한 명령어는 모바일에서 사용할 수 없습니다.

문제 해결

메시지를 보냈는데 **PC**에 전달되지 않는 경우

- 세션 상태 아이콘을 확인합니다. 회색이면 세션이 끊겼습니다.
- 모바일 앱을 백그라운드에서 포그라운드로 전환한 뒤 다시 시도합니다.

승인 버튼이 응답하지 않는 경우

- 네트워크 연결을 확인합니다.
 - 앱을 완전히 종료 후 재실행하고 세션에 다시 연결합니다.
-

다음 장 미리보기

이제 모바일에서 지시를 내리고 작업을 승인하는 방법을 알았습니다. 다음 장에서는 Claude가 작업을 완료했을 때 스마트폰으로 알림을 받는 방법, 즉 푸시 알림 설정을 알아봅니다.

5-5. 푸시 알림으로 작업 완료 확인

이 장에서 배울 내용

Claude에게 오래 걸리는 작업을 맡겼을 때, 스마트폰을 계속 들여다볼 필요가 없습니다. Claude가 입력을 기다리거나 작업이 완료되면 푸시 알림으로 알려줍니다. 이 장에서는 알림 설정 방법과 알림을 활용해 효율적으로 작업을 관리하는 방법을 배웁니다.

푸시 알림이 오는 시점

Claude Code는 다음 상황에서 모바일 알림을 전송합니다.

알림 유형	발생 조건
입력 대기	Claude가 다음 지시를 기다릴 때
도구 승인 필요	실행 권한 승인이 필요할 때
작업 완료	긴 작업이 완전히 끝났을 때
오류 발생	Claude가 오류를 만나 중단되었을 때
네트워크 재연결	오프라인 후 세션이 복구되었을 때

이 알림 덕분에 긴 빌드나 테스트 실행, 파일 분석 같은 작업을 맡기고 다른 일을 하다가 완료 시점에만 대응할 수 있습니다.

알림 권한 설정

iOS

처음 앱 실행 시

앱을 처음 설치하고 Code 기능에 접근하면, 알림 권한 요청 팝업이 자동으로 나타납니다. 허용을 탭합니다.

나중에 권한을 켜는 방법

권한 요청을 거부했거나 나중에 활성화하려면:

1. iPhone 설정 앱 열기
2. 아래로 스크롤하여 **Claude** 탭
3. 알림 → 알림 허용 토글 켜기
4. 알림 스타일 에서 배너 또는 알림 선택 권장

Android

처음 앱 실행 시

Android 13 이상에서는 앱 첫 실행 시 알림 권한 요청 팝업이 나타납니다. 허용 을 탭합니다.

나중에 권한을 켜는 방법

1. 기기 설정 → 앱 (또는 애플리케이션)
2. **Claude** 선택
3. 알림 → 알림 허용 토글 켜기

앱 내 알림 설정

Claude 앱 자체에서도 알림 종류를 세밀하게 조정할 수 있습니다.

설정 경로:

- 앱 우측 상단 프로필 아이콘 → **Settings** → **Notifications**

설정 항목	설명
Input required	Claude가 입력을 기다릴 때 알림
Tool approval	도구 승인 필요 시 알림
Task complete	작업 완료 시 알림
Error alerts	오류 발생 시 알림
Sound	알림음 켜기/끄기
Vibration	진동 켜기/끄기

권장 설정: 장시간 작업 중에는 **Input required** 와 **Tool approval** 만 켜두면 불필요한 알림을 줄일 수 있습니다.

알림 받고 신속하게 대응하기

잠금 화면에서 바로 처리

알림이 오면 잠금 화면에 미리보기가 표시됩니다.

```
[Claude] dev-machine-elegant-fox
도구 실행 승인이 필요합니다.
Bash: npm run test
```

iOS: 알림을 왼쪽으로 스와이프하면 보기 버튼이 나타납니다. 탭하면 앱이 열리며 해당 세션으로 바로 이동합니다.

Android: 알림을 탭하면 해당 세션이 바로 열립니다. 알림을 길게 누르면 승인 / 거부 액션 버튼이 나타납니다(지원 기기에 따라 다름).

알림 센터에서 한번에 확인

여러 세션의 알림이 쌓였을 때 알림 센터에서 한눈에 확인합니다.

iOS: 화면 상단에서 아래로 스와이프 → 알림 센터

Android: 화면 상단에서 아래로 스와이프 → 알림 패널

실전 활용 시나리오

시나리오 1 — 빌드 완료 확인

1. 모바일에서 Claude에게 지시:
"프론트엔드 프로젝트 빌드하고 에러 있으면 수정해줘"
2. 스마트폰을 내려놓고 다른 작업
3. 10분 후 알림 수신:
"dev-machine: 빌드 성공, 입력 대기 중"

4. 알림 탭 → 결과 확인 → 다음 지시

시나리오 2 — 이동 중 승인

1. 출근 중 전철 안에서 알림 수신:
"도구 승인 필요 - git push origin main"
2. 알림 탭 → 변경 사항 확인
3. 내용 검토 후 승인 또는 거부
4. Claude가 작업 재개 → 완료 시 재알림

방해 금지 모드와 알림

야간이나 집중 작업 시간에는 방해 금지 모드를 활용합니다.

iOS 집중 모드 설정:

1. 설정 → 집중 모드 → 방해 금지
2. 앱 알림 허용 에서 Claude 추가
3. 설정 시간 동안 Claude 알림만 허용

Android 방해 금지 설정:

1. 설정 → 소리 → 방해 금지
2. 예외 에서 Claude 알림을 허용된 앱 으로 추가

이렇게 하면 야간에도 도구 승인이 필요한 경우에만 알림을 받을 수 있습니다.

알림이 오지 않을 때 확인사항

체크리스트:

- ☐ iOS/Android 시스템 알림 권한이 켜져 있는가
- ☐ Claude 앱 내 알림 설정에서 원하는 항목이 활성화되어 있는가
- ☐ 방해 금지 모드가 Claude를 차단하고 있지 않은가
- ☐ 모바일이 세션에 연결된 상태인가 (앱이 백그라운드에 있어도 알림은 수신됨)
- ☐ PC 터미널이 열린 채 Remote Control이 실행 중인가

5장 마무리

이 장에서는 스마트폰으로 Claude Code를 완전히 제어하는 방법을 단계별로 배웠습니다.

장	핵심 내용
5-1	Claude 모바일 앱 설치 및 계정 연결
5-2	QR 코드로 1초 만에 세션 연결
5-3	세션 목록에서 팀 에이전트 선택
5-4	지시 입력 및 도구 승인 처리
5-5	푸시 알림으로 비동기 작업 관리

Remote Control과 모바일 앱을 함께 활용하면, PC 앞에 있지 않아도 Claude Code의 모든 기능을 온전히 사용할 수 있습니다. 긴 작업을 맡기고 외출하거나, 이동 중에 승인 처리를 하거나, 팀 에이전트 여러 명을 동시에 관리하는 것이 모두 스마트폰 하나로 가능해집니다.

6-1. gstack — Claude Code 플러그인 스택 관리

gstack이란?

gstack은 Claude Code를 위한 전문화 워크플로우 스킬 시스템입니다. 단순한 AI 어시스턴트 사용을 넘어, 소프트웨어 개발 사이클 전반을 체계적으로 지원하는 역할 기반 명령어 모음을 제공합니다. `/ship`, `/qa`, `/review`, `/investigate`, `/health` 등 개발 현장에서 바로 쓸 수 있는 슬래시 커맨드를 Claude Code에 추가해줍니다.

gstack의 핵심 철학은 세 가지입니다.

- 역할 기반 의사결정: 코드를 작성하기 전에 CEO(비즈니스 가치)나 CTO(아키텍처)의 관점에서 먼저 문제를 검토합니다.
- 자동화된 계획 수립: 복잡한 작업을 단계별로 쪼개고 각 단계마다 검증을 거칩니다.
- 그라운드링(**Grounding**): 프로젝트의 현재 상태와 요구사항에 에이전트를 강력하게 고정시켜 맥락을 잃지 않게 합니다.

설치 방법

gstack은 Claude Code의 스킬 시스템을 통해 설치됩니다. 터미널에서 Claude Code를 실행한 뒤 다음 명령어를 입력합니다.

```
/gstack
```

처음 `/gstack` 을 실행하면 설치 과정이 자동으로 진행됩니다. 설치가 완료되면 스킬이 `~/.claude/skills/gstack` 경로에 저장됩니다.

설치 확인:

```
# Claude Code 세션 내에서  
/gstack-upgrade
```

현재 버전과 최신 버전을 비교하여 업데이트가 필요하면 자동으로 수행합니다. 자동 업데이트를 항상 활성화하려면 다음 명령어를 실행합니다.

```
~/.claude/skills/gstack/bin/gstack-config set auto_upgrade true
```

주요 명령어

커맨드	역할	용도
<code>/cso</code>	Founder / CEO	제품 비전, 범위, 우선순위 결정
<code>/autoplan</code>	Planner	요구사항 분석 및 개발 로드맵 자동 생성
<code>/review</code>	Peer Reviewer	코드 논리 오류, 스타일, 잠재 버그 검토
<code>/qa</code>	QA Engineer	구현 기능이 요구사항을 충족하는지 검증
<code>/ship</code>	Release Manager	작업 마무리 및 배포 준비
<code>/investigate</code>	Detective	버그 및 코드베이스 특정 부분 심층 조사
<code>/learn</code>	Learner	새 라이브러리나 스택 도입 전 선행 학습
<code>/careful</code>	Senior Dev	중요한 코드 수정 시 극도로 신중하게 접근
<code>/context-save</code>	Checkpoint	현재 작업 상태 저장
<code>/context-restore</code>	Restorer	저장된 작업 상태 복원

실전 활용 예시

새 기능 개발 흐름

새로운 기능을 개발할 때 gstack 커맨드를 순서대로 활용하면 체계적인 개발이 가능합니다.

1단계: `/cso`

- "사용자 인증 기능을 추가하려 합니다. 비즈니스 관점에서 검토해주세요."
- Claude가 CEO 역할로 기능의 비즈니스 가치와 우선순위를 분석합니다.

2단계: `/autoplan`

- "JWT 기반 사용자 인증 시스템 구현 계획을 작성해주세요."
- 세부 태스크 목록과 예상 구현 순서를 자동 생성합니다.

3단계: (코드 구현 후) `/review`

- "작성한 `auth.py` 파일을 리뷰해주세요."
- 코드 품질, 보안 취약점, 스타일 일관성을 점검합니다.

4단계: `/qa`

→ "인증 기능이 요구사항을 모두 만족하는지 검증해주세요."
→ 테스트 시나리오를 실행하고 결과를 보고합니다.

5단계: /ship

→ "기능 구현이 완료되었습니다. 배포 준비를 해주세요."
→ 브랜치 정리, 변경사항 요약, 배포 체크리스트를 제공합니다.

컨텍스트 저장 및 복원

긴 세션에서 작업 상태를 저장하고 이어서 작업할 때 유용합니다.

```
# 현재 작업 상태 저장
/context-save 인증기능-구현중

# 목록 확인
/context-save list

# 저장된 상태 복원
/context-restore 인증기능-구현중
```

Triple Crown 전략 (권장 워크플로우)

gstack 단독으로도 강력하지만, GSD(프로젝트 관리)와 Superpowers(품질 관리)를 함께 사용하면 더욱 강력한 워크플로우를 구성할 수 있습니다.

gstack (전략-검증) → GSD (구조-실행) → Superpowers (품질-방법론)
"무엇을 왜" "어떤 순서로" "어떻게 잘"

이 세 도구를 파이프라인으로 연결하는 것이 Triple Crown 전략입니다. 구체적인 5단계 사이클, 팀 환경 배분 방법, 실전 예시는 [7-4. Triple Crown 전략](#)에서 자세히 다룹니다.

버전 이력

버전	주요 변경사항
v1.1.3.0	<code>/checkpoint</code> → <code>/context-save</code> + <code>/context-restore</code> 분리
v1.1.1.0	<code>/ship</code> 버전 드리프트 감지 및 수정
v1.1.0.0	로컬 HTML 렌더링 및 Retina 스크린샷 지원

버전	주요 변경사항
v1.0.0.0	V1 프롬프트 스타일 개선, /plan-tune 추가

현재 최신 버전: v1.1.3.0 (2026-04-19 기준)

6-2. superpowers — 스킬 기반 워크플로우 자동화

superpowers란?

superpowers는 Claude Code에 전문화된 워크플로우 스킬을 주입하는 플러그인 시스템입니다. Claude는 기본 상태에서도 다양한 작업을 수행할 수 있지만, **superpowers**를 설치하면 테스트 주도 개발, 체계적인 디버깅, 코드 리뷰, 브레인스토밍 등 검증된 개발 방법론을 그대로 따르도록 Claude를 훈련시킬 수 있습니다.

핵심 개념은 간단합니다. 각 스킬은 특정 상황에서 Claude가 취해야 할 정확한 행동 지침을 담고 있으며, Claude는 해당 상황이 되면 스킬을 불러와 그 지침을 따릅니다. "알아서 잘 해줘"가 아니라 "이 방법론대로 정확히 해줘"가 **superpowers**의 동작 방식입니다.

설치 방법

superpowers는 Claude Code의 글로벌 스킬 시스템으로 설치됩니다.

```
# Claude Code 세션에서 설치 시작
/init
```

또는 직접 스킬 저장소를 클론하는 방식으로 설치할 수 있습니다.

```
# 글로벌 스킬 디렉토리에 설치
git clone https://github.com/[superpowers-repo] ~/.claude/skills/superpowers
```

설치 후 스킬이 자동으로 인식됩니다. `~/.claude/CLAUDE.md` 파일에 **superpowers** 스킬 디렉토리가 참조되면 모든 Claude Code 세션에서 해당 스킬을 사용할 수 있습니다.

스킬 목록 확인

Claude Code 세션에서 현재 사용 가능한 스킬 목록은 시스템이 자동으로 세션 시작 시 알려줍니다. 주요 **superpowers** 스킬은 다음과 같습니다.

스킬 이름	용도
<code>superpowers:test-driven-development</code>	TDD 사이클(Red-Green-Refactor) 강제 적용

스킬 이름	용도
<code>superpowers:systematic-debugging</code>	과학적 방법론으로 버그 추적
<code>superpowers:brainstorming</code>	구조화된 아이디어 발산 세션
<code>superpowers:writing-plans</code>	실행 가능한 개발 계획서 작성
<code>superpowers:executing-plans</code>	계획서 기반 단계적 실행
<code>superpowers:dispatching-parallel-agents</code>	병렬 에이전트 작업 분배
<code>superpowers:code-reviewer</code>	심층 코드 리뷰 수행
<code>superpowers:verification-before-completion</code>	완료 전 검증 체크리스트
<code>superpowers:finishing-a-development-branch</code>	개발 브랜치 마무리 절차
<code>superpowers:using-git-worktrees</code>	Git Worktree 기반 병렬 작업

스킬 호출 방법

스킬을 호출하는 방법은 두 가지입니다.

방법 1: 직접 지시

작업을 요청할 때 스킬 이름을 명시합니다.

```
"superpowers:test-driven-development 스킬을 사용해서
사용자 로그인 기능을 구현해줘."
```

방법 2: 자동 감지 (권장)

superpowers의 `using-superpowers` 스킬이 활성화되어 있으면, Claude는 작업 내용을 보고 자동으로 적합한 스킬을 선택합니다. 예를 들어 "버그를 고쳐줘"라고 하면

`systematic-debugging` 스킬이, "새 기능을 만들어줘"라고 하면 `brainstorming` → `writing-plans` → `executing-plans` 순서로 스킬이 연계됩니다.

핵심 스킬 심층 소개

test-driven-development

TDD(테스트 주도 개발)를 강제 적용하는 스킬입니다. 이 스킬을 사용하면 Claude는 반드시 다음 순서를 지킵니다.

1. 실패하는 테스트 먼저 작성 (Red)
2. 테스트를 통과하는 최소한의 코드 작성 (Green)
3. 리팩토링 (Refactor)

단순히 "테스트도 써줘"라고 부탁하는 것과 달리, 이 스킬은 구현 전 테스트 작성을 보장합니다.

systematic-debugging

버그를 감(感)으로 고치는 것이 아니라 과학적 방법으로 추적하는 스킬입니다.

- 1단계: 현상 관찰 - 버그를 재현 가능하게 문서화
- 2단계: 가설 수립 - 가능한 원인 목록 작성
- 3단계: 가설 검증 - 하나씩 배제하며 원인 특정
- 4단계: 수정 및 검증 - 수정 후 회귀 테스트

brainstorming

새로운 기능이나 접근 방식을 탐색할 때 사용합니다. Claude가 즉시 구현에 달려들지 않고, 먼저 다양한 옵션을 발산하고 트레이드오프를 분석한 뒤 방향을 결정합니다.

커스텀 스킬 작성

superpowers 스킬은 마크다운 파일로 작성됩니다. 자신만의 스킬을 만들 수 있습니다.

```
---
name: my-deploy-checklist
description: 배포 전 체크리스트를 실행합니다
---

# 배포 전 체크리스트

다음 항목을 순서대로 확인하세요:

1. [ ] 테스트 전체 통과 확인
2. [ ] 환경 변수 설정 확인 (staging vs production)
```

- 3. [] 데이터베이스 마이그레이션 준비 여부
- 4. [] 롤백 계획 수립
- 5. [] 팀원 배포 승인 획득

이 파일을 `~/.claude/skills/` 디렉토리에 저장하면 바로 사용 가능합니다.

스킬 우선순위 규칙

여러 스킬이 동시에 적용될 수 있는 상황에서는 다음 순서를 따릅니다.

- 1순위: 프로세스 스킬 (brainstorming, systematic-debugging)
 - 작업 방식 자체를 결정
- 2순위: 구현 스킬 (test-driven-development, code-reviewer)
 - 실제 실행 방법을 안내

"기능을 만들어줘" → brainstorming 먼저 → 방향 결정 후 TDD로 구현

"버그를 고쳐줘" → systematic-debugging 먼저 → 원인 파악 후 수정

이 규칙을 이해하면 "Claude가 왜 바로 코드를 짜지 않고 먼저 질문하는가"에 대한 답을 얻을 수 있습니다. 그것이 바로 superpowers가 동작하는 방식입니다.

팀 환경에서의 활용

멀티에이전트 팀 환경에서는 각 팀원에게 역할에 맞는 스킬을 부여할 수 있습니다.

```
# 서연(개발자) CLAUDE.md에 설정
superpowers:test-driven-development
superpowers:systematic-debugging

# 태양(리뷰어) CLAUDE.md에 설정
superpowers:code-reviewer
superpowers:verification-before-completion

# 민준(PM) CLAUDE.md에 설정
superpowers:brainstorming
superpowers:writing-plans
```

각 팀원이 자신의 역할에 특화된 스킬을 사용하면, 팀 전체의 작업 품질이 일관되게 유지됩니다.

6-3. gsd — Get Stuff Done 프로젝트 관리

gsd란?

gsd(Get Stuff Done)는 Claude Code를 위한 구조화된 프로젝트 관리 프레임워크입니다. 단순히 "이걸 만들어줘"라고 요청하는 방식에서 벗어나, 프로젝트를 마일스톤 → 페이지 → 태스크로 분해하고 각 단계를 체계적으로 계획, 실행, 검증하는 워크플로우를 제공합니다.

gsd가 해결하는 핵심 문제는 컨텍스트 소실입니다. 복잡한 프로젝트를 단일 대화로 진행하면 Claude는 앞서 논의한 내용을 잃어버리거나 일관성 없는 결정을 내릴 수 있습니다. gsd는 프로젝트 상태를 파일로 저장하여 대화가 바뀌어도 작업을 이어갈 수 있게 합니다.

설치 방법

gsd는 superpowers 스킬 시스템의 일부로 함께 제공됩니다. superpowers가 설치되어 있다면 gsd 커맨드를 바로 사용할 수 있습니다.

설치 확인:

```
# Claude Code 세션에서
/gsd:help
```

사용 가능한 모든 gsd 커맨드 목록이 출력되면 정상적으로 설치된 것입니다.

프로젝트 구조

gsd는 다음 계층으로 프로젝트를 관리합니다.

```
프로젝트 (Project)
├── 마일스톤 (Milestone) ← 큰 목표 단위
│   ├── 페이지 (Phase) ← 구현 가능한 작업 묶음
│   │   └── 태스크 (Task) ← 개별 실행 단위
```

예를 들어 "쇼핑몰 앱"이라는 프로젝트가 있다면:

- 마일스톤 1: 기본 기능 구현 (회원가입, 상품 목록, 장바구니)
- 마일스톤 2: 결제 시스템 연동
- 마일스톤 3: 관리자 대시보드

각 마일스톤은 여러 페이지로 나뉘며, 페이지 단위로 계획과 실행이 이루어집니다.

새 프로젝트 시작

1단계: 프로젝트 생성

```
/gsd:new-project
```

프로젝트 이름, 목표, 기술 스택 등을 입력하면 초기 구조가 생성됩니다. 프로젝트 정보는 `.planning/` 디렉토리에 저장됩니다.

2단계: 마일스톤 생성

```
/gsd:new-milestone
```

프로젝트의 주요 목표를 마일스톤으로 정의합니다. Claude가 질문을 통해 마일스톤의 범위와 성공 기준을 함께 도출합니다.

3단계: 진행 상황 확인

```
/gsd:progress
```

현재 마일스톤의 완료율, 각 페이지의 상태, 다음 해야 할 일을 한눈에 볼 수 있습니다.

페이지 관리

페이지 계획 수립

```
/gsd:plan-phase 1
```

1번 페이지의 상세 구현 계획을 작성합니다. Claude가 코드베이스를 분석하고 필요한 변경사항을 태스크 목록으로 만들어냅니다.

페이지 실행

```
/gsd:execute-phase 1
```

계획서에 따라 실제 구현을 시작합니다. 각 태스크를 순서대로 실행하며 진행 상황을 업데이트합니다.

페이지 검증

```
/gsd:validate-phase 1  
/gsd:verify-work
```

페이지가 완료된 후 목표 달성 여부를 검증합니다. 단순히 태스크 완료 여부가 아니라, 실제로 기능이 의도대로 동작하는지를 목표 역방향으로 분석합니다.

실전 워크플로우 예시

일반적인 개발 사이클

```
아침: 어제 어디까지 했나?  
/gsd:progress  
→ "Phase 2 진행 중, Task 3/7 완료"  
  
작업 시작:  
/gsd:next  
→ "다음 태스크: 사용자 프로필 API 구현"  
  
구현 완료 후:  
/gsd:validate-phase 2  
→ 완료 기준 체크 및 결과 보고  
  
다음 단계로:  
/gsd:execute-phase 3
```

막혔을 때

```
/gsd:discuss-phase 2
```

현재 페이지의 접근 방식에 대해 Claude와 논의합니다. 대안적 구현 방법, 트레이드오프 분석, 위험 요소 점검 등을 수행합니다.

가정(Assumption) 확인

```
/gsd:list-phase-assumptions 2
```

페이지 계획에 포함된 가정들을 나열합니다. "이 라이브러리가 이 방식으로 동작할 것이다"와 같은 검증되지 않은 가정을 미리 확인하여 리스크를 줄입니다.

주요 커맨드 전체 목록

커맨드	설명
<code>/gsd:new-project</code>	새 프로젝트 생성
<code>/gsd:new-milestone</code>	새 마일스톤 추가
<code>/gsd:add-phase</code>	마일스톤에 페이지 추가
<code>/gsd:plan-phase [N]</code>	N번 페이지 상세 계획 수립
<code>/gsd:execute-phase [N]</code>	N번 페이지 실행
<code>/gsd:validate-phase [N]</code>	N번 페이지 검증
<code>/gsd:verify-work</code>	현재 작업의 목표 달성 여부 검증
<code>/gsd:progress</code>	전체 진행 상황 요약
<code>/gsd:next</code>	다음 수행할 태스크 안내
<code>/gsd:discuss-phase [N]</code>	페이지 접근 방식 토론
<code>/gsd:list-phase-assumptions</code>	페이지 가정 목록
<code>/gsd:add-todo</code>	백로그에 할 일 추가
<code>/gsd:review-backlog</code>	백로그 검토
<code>/gsd:milestone-summary</code>	마일스톤 요약 보고

커맨드	설명
<code>/gsd:complete-milestone</code>	마일스톤 완료 처리
<code>/gsd:audit-milestone</code>	마일스톤 품질 감사
<code>/gsd:health</code>	프로젝트 건강 상태 진단
<code>/gsd:help</code>	전체 커맨드 도움말

gstack과 함께 사용하기

gsd는 gstack, superpowers와 함께 사용할 때 가장 강력합니다.

```

비전 수립: /cso (gstack)
계획 수립: /gsd:plan-phase
실행: /gsd:execute-phase + superpowers:test-driven-development
검증: /gsd:verify-work + /qa (gstack)
완료: /ship (gstack)

```

단순한 AI 대화를 넘어, 실제 소프트웨어 팀이 사용하는 수준의 프로젝트 관리가 Claude Code 안에서 가능해집니다.

파일 시스템 구조

gsd는 프로젝트 루트의 `.planning/` 디렉토리에 모든 상태를 저장합니다.

```

.planning/
├── ROADMAP.md           # 전체 로드맵
├── milestones/
│   ├── M1/
│   │   ├── MILESTONE.md # 마일스톤 정의
│   │   ├── phase-1/
│   │   │   ├── PLAN.md  # 페이지 계획서
│   │   │   └── VERIFICATION.md # 검증 결과
│   │   └── phase-2/
│   └── M2/
└── research/           # 기술 조사 결과

```

이 구조 덕분에 Git으로 프로젝트 계획 자체도 버전 관리할 수 있습니다.

6-4. RTK — Rust Token Killer 토큰 최적화

RTK란?

RTK(Rust Token Killer)는 Claude Code에서 실행되는 CLI 명령어의 출력을 필터링하여 불필요한 토큰 소비를 줄여주는 프록시 도구입니다. Rust로 작성되어 처리 속도가 빠르며, Claude Code의 훅(hook) 시스템과 연동하여 개발자가 의식하지 않아도 자동으로 동작합니다.

왜 토큰 절약이 중요할까요? Claude Code는 실행한 명령어의 출력을 컨텍스트로 읽습니다. `ls` 명령어 하나가 수백 개의 파일 목록을 출력하거나, `git log` 명령어가 수천 줄의 히스토리를 쏟아내면 그 모두가 토큰을 소비합니다. RTK는 이런 출력을 스마트하게 요약하거나 필터링하여 같은 정보를 더 적은 토큰으로 전달합니다.

실제 사용 데이터에 따르면 RTK 적용 후 평균 **63%**의 토큰 절약이 이루어집니다.

설치 방법

사전 요구사항

- Rust 툴체인 (rustup)
- Claude Code 설치 및 실행 중

설치 과정

```
# RTK 저장소 클론
git clone https://github.com/[rtk-repo]/rtk
cd rtk

# 빌드 및 설치
cargo install --path .

# 또는 사전 빌드된 바이너리 직접 설치
curl -sSL https://[rtk-releases]/install.sh | bash
```

설치 후 바이너리는 `~/.local/bin/rtk` 경로에 위치합니다.

```
# 설치 확인
rtk --version

# 출력: rtk 0.27.2
```

```
which rtk
# 출력: /home/user/.local/bin/rtk
```

RTK 버전 확인

주의: `rtk` 라는 이름을 가진 다른 패키지(Rust Type Kit)가 존재합니다. `rtk gain` 명령어가 정상 동작하는지 확인하여 올바른 RTK가 설치되었는지 검증하세요.

Hook 기반 자동 적용

RTK의 가장 강력한 기능은 **Claude Code** hook과의 통합입니다. 한 번 설정하면 이후 모든 Bash 명령어가 자동으로 RTK를 통해 실행됩니다. 개발자가 `git status` 를 실행하면 Claude Code 내에서 `rtk git status` 로 투명하게 변환됩니다.

Claude Code의 `settings.json` 에 다음 hook을 추가합니다.

```
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Bash",
        "hooks": [
          {
            "type": "command",
            "command": "~/.local/bin/rtk-hook"
          }
        ]
      }
    ]
  }
}
```

hook이 설정되면 다음과 같이 동작합니다.

```
개발자 요청: "현재 파일 목록 보여줘"
Claude 실행: git status
  ↓ RTK hook 자동 개입
실제 실행: rtk git status
  ↓ 출력 필터링
Claude 수신: 최적화된 출력 (토큰 65% 절약)
```

주요 메타 명령어

RTK는 몇 가지 직접 실행하는 메타 명령어를 제공합니다.

rtk gain — 토큰 절약 분석

```
rtk gain
```

지금까지의 RTK 사용 현황을 보여줍니다.

RTK 토큰 절약 분석 결과

RTK Token Savings (Global Scope)

```
Total commands: 1271
Input tokens: 1.3M
Output tokens: 497.8K
Tokens saved: 850.1K (63.1%)
Total exec time: 29m44s (avg 1.4s)
Efficiency meter:  63.1%
```

By Command

#	Command	Count	Saved	Avg%
1.	rtk ls	431	323.6K	65.1%
2.	rtk read	330	137.9K	6.1%
3.	rtk git log ...	1	58.7K	99.9%

명령어별로 얼마나 절약했는지, 가장 효과가 큰 명령어는 무엇인지 한눈에 파악할 수 있습니다.

rtk gain --history — 히스토리 분석

```
rtk gain --history
```

시간 흐름에 따른 토큰 절약 추이를 확인합니다. 특정 기간 동안 어떤 작업에서 토큰을 많이 사용했는지 분석할 수 있습니다.

rtk discover — 최적화 기회 발굴

```
rtk discover
```


Claude Code 사용 히스토리를 분석하여 RTK를 아직 적용하지 않은 고비용 명령어를 찾아냅니다. "이 명령어도 RTK로 처리하면 X토큰을 추가로 절약할 수 있습니다"라는 형태로 개선 기회를 알려줍니다.

rtk proxy — 원시 실행 (디버깅용)

```
rtk proxy git log --oneline -20
```

RTK 필터링 없이 원래 명령어 그대로 실행합니다. RTK가 출력을 어떻게 변환하는지 비교하거나 필터링 로직을 디버깅할 때 사용합니다.

RTK가 최적화하는 명령어들

RTK는 각 명령어의 특성에 맞게 다른 필터링 전략을 적용합니다.

명령어	절약율	최적화 방식
<code>ls</code>	~65%	불필요한 메타데이터 제거, 핵심 파일명만 유지
<code>git log</code>	~99%	커밋 수 제한, 관련 없는 히스토리 요약
<code>curl</code>	~99%	대용량 응답 트리밍, 핵심 데이터만 추출
<code>read</code> (파일 읽기)	~6%	중복 공백/빈 줄 제거
<code>cat</code>	~10~40%	파일 크기에 따라 가변 압축

실전 활용 팁

큰 프로젝트에서 특히 효과적

파일 수가 많은 프로젝트에서 `ls` 나 `find` 명령어를 자주 실행할 때 RTK의 효과가 극대화됩니다. 수백 개의 파일 목록이 핵심 정보만 담긴 요약으로 변환됩니다.

Git 집중 작업 시

`git log`, `git diff`, `git blame` 등 Git 관련 명령어는 RTK를 통해 거의 99%에 가까운 절약이 가능합니다. 특히 전체 히스토리를 훑는 작업에서 효과가 큼니다.

API 응답 처리 시

외부 API를 `curl` 로 호출하고 결과를 Claude에게 전달할 때, RTK가 대용량 JSON 응답을 자동으로 정제합니다.

토큰 절약의 실제 가치

Claude Code API는 토큰 기반으로 과금됩니다. RTK로 63%의 토큰을 절약한다는 것은:

- 같은 비용으로 약 2.7배 더 많은 작업을 수행할 수 있음
- 또는 같은 양의 작업을 37%의 비용으로 처리할 수 있음

장기 프로젝트, 대용량 코드베이스, 반복적인 파일 탐색이 많은 작업일수록 RTK의 ROI는 높아집니다.

요약

RTK는 설치하고 후속을 연결하면 이후 별도의 조작 없이 자동으로 동작합니다. 개발자 입장에서는 투명한 도구이며, 실제로 절약되는 토큰의 양은 `rtk gain` 명령어로 언제든지 확인할 수 있습니다. Claude Code를 장기적으로 사용할 계획이라면 초기에 RTK를 설치하는 것이 비용 대비 가장 효율적인 투자 중 하나입니다.

7-1. 팀 지시 흐름 설계

지시 흐름의 원칙

팀 에이전트를 효과적으로 운용하기 위해서는 지시가 어떻게 흘러가는지를 명확히 설계해야 한다. 명확한 흐름이 없으면 팀원 간 작업이 중복되거나, 지시가 누락되거나, 결과가 사용자에게 돌아오지 않는 문제가 발생한다.

이 책에서 채택한 지시 흐름은 단일 진입점 패턴이다.

팀 지시 흐름도

왜 단일 진입점인가

사용자가 팀원에게 직접 지시를 내리는 직접 지시 패턴도 가능하지만, 다음과 같은 이유로 단일 진입점을 권장한다.

1. 컨텍스트 관리 — 팀장이 전체 작업 상황을 파악하고 있어 중복 방지
2. 우선순위 조정 — 여러 작업이 동시에 들어올 때 순서 조정 가능
3. 진행 상황 추적 — 모든 보고가 팀장에게 집중되므로 현황 파악이 용이
4. 사용자 부담 감소 — 사용자는 팀장에게만 지시하면 됨

지시 흐름 상세 설계

Phase 1: 지시 수신

사용자의 지시가 팀장에게 도달하는 경로는 세 가지다.

```
# → Pane 0에 전달

# 경로 2: Remote-Control
# 사용자가 claude.ai/code 또는 모바일 앱에서 직접 입력

# 경로 3: 로컬 터미널
# 사용자가 TMUX Pane 0에서 직접 입력
tmux select-pane -t team:0.0
```

Phase 2: 분석 및 분해

팀장은 받은 지시를 분석하여 작업 단위로 분해한다.

```
# 팀장의 내부 분석 프로세스
```

```
수신 지시: "결제 모듈을 PG사 변경에 대응할 수 있도록 리팩토링해줘"
```

```
분석 결과:
```

1. [리서치] 현재 결제 모듈 구조 파악 → 지훈
2. [설계] 새 PG사 대응 아키텍처 설계 → 민준
3. [구현] 설계에 따른 코드 리팩토링 → 서연
4. [리뷰] 리팩토링된 코드 품질 검토 → 태양

```
의존성:
```

- 2번은 1번 완료 후 시작
- 3번은 2번 완료 후 시작
- 4번은 3번 완료 후 시작

Phase 3: 팀원 배분

분해된 작업을 `tmux send-keys` 명령으로 각 팀원에게 전달한다.

```
# 팀장이 실행하는 명령들
```

```
# 1단계: 리서처에게 현황 분석 요청
```

```
tmux send-keys -t team:0.2 \  
"현재 /src/payment/ 디렉토리의 결제 모듈 구조를 분석해줘. \  
PG사 연동 부분의 인터페이스와 의존성을 파악해서 \  
/tmp/payment-analysis.md 에 정리해줘." Enter
```

```
# 2단계 이후는 1단계 완료 보고를 받은 뒤 진행
```

Phase 4: 진행 관리

팀장은 각 팀원의 작업 진행 상황을 모니터링한다.

```
# 팀원의 작업 상태 확인 (팀장이 수행)
```

```
# 각 파인의 최근 출력 확인
```

```
tmux capture-pane -t team:0.1 -p | tail -5 # PM 상태  
tmux capture-pane -t team:0.2 -p | tail -5 # 리서처 상태  
tmux capture-pane -t team:0.4 -p | tail -5 # 개발자 상태
```

Phase 5: 결과 보고

모든 작업이 완료되면 팀장이 결과를 종합하여 사용자에게 보고한다.

```
# Bot Mode로 수신한 경우 - 같은 채널로 응답
-m '🔗 결제 모듈 리팩토링 완료

- 아키텍처: Strategy 패턴 적용 (PG사별 어댑터)
- 변경 파일: 12개
- 테스트: 전체 통과
- 리뷰: 태양 승인 완료

상세 내용은 커밋 abc1234 참고'
```

지시 흐름 CLAUDE.md 설정

이 흐름이 자동으로 작동하려면 CLAUDE.md에 명확한 규칙을 정의해야 한다.

```
# 팀장 CLAUDE.md (Pane 0)

## 행동 원칙
- 직접 코드를 작성하거나 파일을 수정하지 않는다
- 지시를 수령하면 분석 → 분해 → 분배한다
- 각 팀원의 완료 보고를 수합하여 사용자에게 전달한다

## 팀원 배분 기준
| 작업 유형 | 담당 | 파인 |
|-----|-----|-----|
| 설계·계획 | 민준 | team:0.1 |
| 조사·분석 | 지훈 | team:0.2 |
| UI/UX      | 수아 | team:0.3 |
| 구현·수정 | 서연 | team:0.4 |
| 리뷰·검토 | 태양 | team:0.5 |

## 팀원에게 지시 전달 방법
tmux send-keys -t {파인} "{지시 내용}" Enter
```

병렬 실행과 순차 실행

작업 특성에 따라 병렬 또는 순차로 배분한다.

병렬 실행 가능한 경우

```
# 독립적인 조사 작업 - 동시 수행
tmux send-keys -t team:0.2 "프론트엔드 프레임워크 비교 조사" Enter
tmux send-keys -t team:0.1 "백엔드 API 스펙 설계" Enter

# 두 작업은 서로 의존하지 않으므로 동시에 진행
```

순차 실행이 필요한 경우

```
# 의존성이 있는 작업 - 순서대로 수행
# 1. 먼저 리서처가 조사
tmux send-keys -t team:0.2 "현재 인증 방식 조사해줘" Enter

# 2. 조사 완료 후 PM이 설계 (팀장이 완료 보고 수신 후 발행)
tmux send-keys -t team:0.1 "지훈의 조사 결과 기반으로 새 인증 설계해줘" Enter

# 3. 설계 완료 후 개발자가 구현
tmux send-keys -t team:0.4 "민준의 설계 기반으로 인증 모듈 구현해줘" Enter
```

흐름 장애 처리

팀원의 작업이 실패하거나 지연될 때의 대응 방안이다.

```
# 팀장의 장애 대응 규칙

## 작업 실패 시
1. 실패 원인을 팀원에게 확인
2. 가능하면 재시도 지시
3. 재시도 실패 시 다른 팀원에게 재배분 또는 사용자에게 보고

## 작업 지연 시
1. 팀원에게 현재 진행률 확인
2. 블로킹 요소가 있으면 해결을 위해 다른 팀원 투입
3. 사용자에게 중간 보고
```

핵심 정리: 모든 지시는 팀장(Pane 0)을 통해 흐른다. 수신 → 분석 → 분해 → 분배 → 수합 → 보고의 단일 진입점 패턴이 팀 운용의 기본이다.

7-2. Bot Mode 활용

Bot Mode란 무엇인가

Bot Mode는 Claude Code가 메시지 접두사를 기반으로 지시를 수신하고, 작업 완료 후 결과를 자동으로 보고하는 동작 방식이다. Remote-Control을 통해 모바일이나 웹에서 메시지를 보내면, 팀장(Pane 0)이 이를 수신하고 팀원에게 분배한 뒤 결과를 돌려보낸다.

일반 모드: 사용자 → Claude → 터미널에 출력
Bot Mode: 사용자 (모바일/웹) → Remote-Control → 팀장 → 팀원들 → 결과 보고

Bot Mode 동작 절차

1단계: Remote-Control로 지시 수신

모바일 앱 또는 `claude.ai/code`에서 메시지를 전송하면 Remote-Control을 통해 로컬 세션(Pane 0)에 실시간으로 전달된다.

수신 메시지: 결제 API 현황 알려줘
팀장(Pane 0)이 수신 → 분석 → 팀원 배분

2단계: 팀원에게 작업 배분

팀장은 `tmux send-keys` 로 각 팀원에게 지시를 전달한다.

```
# 팀장이 팀원들에게 작업 배분
tmux send-keys -t team:0.4 "결제 API 코드 현황 분석해서 보고해줘" Enter
tmux send-keys -t team:0.1 "현재 결제 모듈 아키텍처 문서 확인해줘" Enter
```

3단계: 결과 종합 및 보고

팀원이 작업 완료 후 팀장에게 보고하면, 팀장이 결과를 종합하여 Remote-Control을 통해 사용자에게 응답한다.

서연(Pane 4): 결제 API v2.3 구현 완료, 테스트 커버리지 87%
민준(Pane 1): 마이크로서비스 전환 설계 완료

팀장 → Remote-Control → 사용자에게 종합 보고 전달

CLAUDE.md에서 팀 역할 정의

팀장의 행동 원칙을 CLAUDE.md에 명시하면 일관된 팀 운용이 가능하다.

팀장 행동 원칙

- ****직접 작업 금지**** - 코드/파일 수정은 팀원에게 위임
- 역할: 지시 수령 → 분석 → 팀원 배분 → 결과 종합 → 보고
- 팀원 지시: `tmux send-keys -t team:0.{파인번호} "지시내용" Enter`

팀원 간 비동기 보고 패턴

팀원이 장시간 작업 후 완료를 알릴 때는 팀장 파인에 직접 메시지를 보낸다.

```
# 팀원(Pane 4 서연)이 팀장(Pane 0)에게 완료 보고
tmux send-keys -t team:0.0 "[서연] 결제 모듈 리팩토링 완료. PR #42 생성했습니다." Enter
```

팀장은 모든 팀원의 보고를 취합한 뒤 사용자에게 한 번에 결과를 전달한다.

7-3. 업무 분담 전략

역할 기반 분담의 원칙

6명의 에이전트로 구성된 팀에서 업무를 효율적으로 분담하려면 역할의 경계가 명확해야 한다. 역할이 모호하면 작업이 중복되거나, 아무도 맡지 않는 공백이 생긴다.

팀장 (쥘) — 지시 수령 · 분석 · 분배 · 보고

이름	역할	담당 업무
민준	PM · 아키텍트	설계 · 기획 · 문서
지훈	리서처	조사 · 분석 · 비교
수아	디자이너	UI/UX · 프로토타입
서연	개발자	구현 · 테스트 · 수정
태양	리뷰어	검토 · 리뷰 · 보안

각 역할의 상세 책임

팀장 (쥘 — Pane 0)

팀장은 오케스트레이터다. 직접 구현하지 않고, 작업의 흐름을 관리한다.

```
# 팀장 CLAUDE.md 핵심 규칙
```

```
## 해야 할 것
```

- 사용자 지시를 수령하고 분석
- 작업을 분해하여 적절한 팀원에게 배분
- 팀원의 완료 보고를 수합하여 사용자에게 전달
- 작업 간 의존성 관리 (순서 조정)

```
## 하지 말 것
```

- 직접 코드를 작성하거나 파일을 수정
- 직접 git commit/push 실행
- 팀원의 작업에 개입하여 직접 수정

PM·아키텍트 (민준 — Pane 1)

프로젝트의 구조와 방향을 설계한다.

```
# 민준 CLAUDE.md

## 담당 영역
- 시스템 아키텍처 설계
- 프로젝트 계획 수립
- 기술 문서 작성
- API 스펙 정의
- 데이터 모델 설계

## 산출물 예시
- /docs/architecture.md
- /docs/api-spec.md
- /docs/data-model.md

## 작업 방식
- 팀장의 지시에 따라 설계 작업 수행
- 설계 완료 시 팀장에게 보고
- 개발자(서연)가 참조할 수 있도록 문서를 명확히 작성
```

리서처 (지훈 — Pane 2)

정보 수집과 기술 조사를 담당한다.

```
# 지훈 CLAUDE.md

## 담당 영역
- 기술 조사 및 비교 분석
- 라이브러리·프레임워크 벤치마크
- 기존 코드 구조 분석
- 레퍼런스 사례 수집

## 산출물 예시
- /docs/research/framework-comparison.md
- /docs/research/current-architecture-analysis.md

## 작업 방식
- 웹 검색, 문서 읽기, 코드 분석 도구 활용
- 조사 결과를 구조화된 문서로 정리
- 결론과 추천을 포함하여 보고
```

디자이너 (수아 — Pane 3)

UI/UX와 사용자 경험을 설계한다.

```
# 수아 CLAUDE.md

## 담당 영역
- UI/UX 설계
- 사용자 흐름(User Flow) 기획
- 컴포넌트 구조 설계
- 스타일 가이드 정의
- 반응형 레이아웃 설계

## 산출물 예시
- /docs/design/user-flow.md
- /docs/design/component-spec.md
- HTML/CSS 프로토타입

## 작업 방식
- 사용자 관점에서 인터페이스 설계
- 개발자(서연)가 구현할 수 있도록 명확한 스펙 제공
```

개발자 (서연 — Pane 4)

실제 코드 구현을 담당한다.

```
# 서연 CLAUDE.md

## 담당 영역
- 기능 구현
- 버그 수정
- 테스트 코드 작성
- 리팩토링
- 빌드 및 배포 스크립트

## 작업 방식
- PM(민준)의 설계, 디자이너(수아)의 스펙을 기반으로 구현
- 구현 완료 시 팀장에게 보고
- 커밋 메시지는 규칙에 맞게 작성
- 아키텍처 변경이 필요하면 직접 변경하지 말고 팀장에게 보고
```

리뷰어 (태양 — Pane 5)

코드 품질과 보안을 검토한다.

태양 CLAUDE.md

담당 영역

- 코드 리뷰
- 품질 기준 준수 확인
- 보안 취약점 점검
- 성능 이슈 검토
- 테스트 커버리지 확인

리뷰 체크리스트

- [] 코드 컨벤션 준수
- [] 에러 핸들링 적절성
- [] 보안 취약점 (인젝션, XSS 등)
- [] 성능 저하 요소
- [] 테스트 커버리지

작업 방식

- 개발자(서연)의 코드를 리뷰
- 문제 발견 시 구체적인 개선안과 함께 보고
- 승인/반려 판단을 팀장에게 전달

업무 배분 기준표

팀장이 작업을 배분할 때 참조하는 기준표다.

작업 키워드	담당	예시
설계, 아키텍처, 계획, 스펙	민준 (PM)	"마이크로서비스 아키텍처 설계해줘"
조사, 비교, 분석, 리서치	지훈 (리서처)	"Redis vs Memcached 비교해줘"
UI, 디자인, 레이아웃, UX	수아 (디자이너)	"대시보드 화면 설계해줘"
구현, 코딩, 수정, 버그	서연 (개발자)	"로그인 API 구현해줘"
리뷰, 검토, 보안, 품질	태양 (리뷰어)	"PR #42 코드 리뷰해줘"
복합 작업	팀장이 분해 후 분배	"결제 시스템 만들어줘"

복합 작업의 분배 예시

실제 작업은 단일 역할로 끝나지 않는 경우가 많다. 복합 작업을 분배하는 실전 예시다.

예시 1: 새 기능 개발

지시: "사용자 알림 시스템을 만들어줘"

팀장의 분배:

Phase	실행	담당	작업
1	병렬	지훈	알림 시스템 기술 스택 조사
1	병렬	수아	알림 UI/UX 설계
2	순차	민준	조사 결과 기반 아키텍처 설계
3	순차	서연	설계 기반 구현
4	순차	태양	코드 리뷰

예시 2: 버그 수정

지시: "프로덕션에서 결제 오류 발생, 수정해줘"

팀장의 분배:

Phase	실행	담당	작업
1	긴급	지훈	에러 로그 분석, 원인 파악
2	긴급	서연	원인 기반 핫픽스 구현
3	긴급	태양	핫픽스 긴급 리뷰

긴급한 경우 PM과 디자이너를 건너뛰고 최소 인원으로 처리한다.

예시 3: 리팩토링

지시: "인증 모듈 리팩토링해줘"

팀장의 분배:

Phase	담당	작업
1	지훈	현재 인증 모듈 구조 분석
2	민준	새 아키텍처 설계

Phase	담당	작업
2	태양	현재 코드 품질 문제점 목록 작성
3	서연	리팩토링 구현
4	태양	리팩토링 결과 리뷰

분담 실패를 방지하는 규칙

실전에서 자주 발생하는 분담 실패 사례와 방지책이다.

실패 유형	원인	방지책
작업 중복	여러 팀원이 같은 파일 수정	팀장이 파일 레벨 담당 명시
공백 발생	아무도 안 맡은 작업	분배 시 체크리스트 사용
순서 무시	설계 전에 구현 시작	Phase 단위 순차 배분
보고 누락	팀원이 완료 보고 안 함	CLAUDE.md에 보고 의무 명시

핵심 정리: 6명의 에이전트는 각각 명확한 역할 경계를 가지며, 팀장이 작업 키워드와 의존성을 기반으로 적절한 팀원에게 분배한다. 복합 작업은 Phase로 분해하여 순차/병렬 조합으로 처리한다.

7-4. Triple Crown 전략 — gstack + GSD + Superpowers 통합 워크플로우

Triple Crown이란

6장에서 gstack, GSD, superpowers를 각각 소개했다. 이 세 도구는 독립적으로도 유용하지만, 셋을 하나의 파이프라인으로 연결할 때 진가를 발휘한다. 이것이 Triple Crown 전략이다.

순서	도구	역할	핵심 질문
①	gstack	전략·검증	"무엇을 왜"
②	GSD	관리·실행	"어떤 순서로"
③	Superpowers	품질·방법론	"어떻게 잘"

도구	역할	핵심 질문
gstack	전략 수립 + 최종 검증	이것을 만들 가치가 있는가? 완성도가 충분한가?
GSD	프로젝트 구조화 + 실행 관리	어떤 순서로 나누어 진행할 것인가?
Superpowers	구현 품질 보장	각 단계를 어떤 방법론으로 수행할 것인가?

세 도구 없이도 Claude Code는 코드를 작성할 수 있다. 하지만 "일단 코드부터" 접근은 프로젝트 규모가 커질수록 방향을 잃기 쉽다. Triple Crown은 전략 → 계획 → 실행 → 검증의 사이클을 강제하여 이 문제를 해결한다.

전체 흐름 개요

Triple Crown의 전체 사이클은 5단계로 구성된다.

```
Phase 1: 전략 수립 ————— gstack
  /cso → /autoplan
  |
Phase 2: 프로젝트 구조화 ————— GSD
  /gsd:new-project → /gsd:new-milestone → /gsd:plan-phase
  |
Phase 3: 구현 ————— GSD + Superpowers
```

```

/gsd:execute-phase
├─ superpowers:test-driven-development
├─ superpowers:systematic-debugging
└─ superpowers:dispatching-parallel-agents
    |
Phase 4: 검증 ───────────────────────────────── GSD + gstack
/gsd:validate-phase → /gsd:verify-work
/review → /qa
    |
Phase 5: 완료 ───────────────────────────────── gstack + GSD
/ship → /gsd:complete-milestone

```

각 단계를 구체적으로 살펴보자.

Phase 1: 전략 수립 (gstack)

코드를 한 줄도 작성하기 전에, 만들 가치가 있는지부터 확인한다.

/cso — CEO 관점 검토

사용자: "실시간 협업 에디터를 만들고 싶어."

/cso 실행 후 Claude의 분석:

비즈니스 가치: ★★★★★☆

- 기존 Google Docs, Notion 대비 차별점 필요
- 실시간 동기화는 기술적 복잡도 높음
- MVP 범위: 텍스트 편집 + 커서 공유로 축소 권장

우선순위 판단:

- P0: 텍스트 동기화 엔진 (CRDT)
- P1: 커서 위치 공유
- P2: 댓글/코멘트 기능
- P3: 버전 히스토리 → 2차 마일스톤으로 이연

/cso는 기능의 비즈니스 타당성을 검토하고, 범위를 적절히 조정한다. "다 만들어"가 아니라 "이것부터 먼저" 순서를 정하는 것이다.

/autoplan — 개발 계획 자동 생성

/cso에서 범위가 정해지면 /autoplan으로 실행 계획을 수립한다.

/autoplan 실행 후 출력:


```
=== 개발 계획 ===
```

Phase 1: CRDT 동기화 엔진 (3일)

- Yjs 라이브러리 조사 및 선정
- 기본 텍스트 동기화 구현
- WebSocket 서버 구현
- 동기화 충돌 해소 테스트

Phase 2: 커서 공유 (2일)

- 커서 위치 브로드캐스트
- 다중 사용자 커서 렌더링
- 네트워크 지연 처리

Phase 3: 통합 테스트 및 배포 (1일)

- E2E 테스트 시나리오 작성
- 성능 테스트 (동시 접속 50명)
- 배포 파이프라인 구성

/autoplan은 단순히 태스크를 나열하는 것이 아니다. CEO 리뷰, 설계 리뷰, 엔지니어링 리뷰를 자동으로 수행하여 계획의 완성도를 높인다.

Phase 2: 프로젝트 구조화 (GSD)

/autoplan의 결과를 GSD의 구조화된 프로젝트 관리 체계로 가져온다.

프로젝트와 마일스톤 생성

```
# 프로젝트 생성
/gsd:new-project
  → 이름: realtime-editor
  → 목표: 실시간 협업 텍스트 에디터 MVP
  → 스택: React, Node.js, Yjs, WebSocket

# 마일스톤 생성
/gsd:new-milestone
  → M1: MVP 출시 - 텍스트 동기화 + 커서 공유
```

페이지 계획 수립

/autoplan에서 도출된 Phase를 GSD의 페이지로 변환한다.

```
# Phase 1 상세 계획
/gsd:plan-phase 1
```

Claude의 계획 수립 과정:

1. 코드베이스 현재 상태 분석
2. 의존성 라이브러리 조사 (Yjs, y-websocket)
3. 태스크 분해:
 - Task 1: Yjs 문서 모델 초기화
 - Task 2: WebSocket 서버 세팅
 - Task 3: 클라이언트-서버 동기화 연결
 - Task 4: 오프라인 편집 + 재접속 동기화
 - Task 5: 동시 편집 충돌 테스트
4. 각 태스크의 완료 기준 정의
5. 가정(assumption) 목록 작성

GSD의 가장 큰 가치는 이 계획이 `.planning/` 디렉토리에 파일로 저장된다는 점이다. 대화가 끊기거나 세션이 바뀌어도 계획은 유지된다.

```
.planning/  
├── ROADMAP.md  
└── milestones/  
    ├── M1/  
    │   ├── MILESTONE.md  
    │   ├── phase-1/  
    │   │   └── PLAN.md      ← 여기에 상세 계획 저장  
    │   ├── phase-2/  
    │   │   └── PLAN.md  
    │   └── phase-3/  
    │       └── PLAN.md
```

Phase 3: 구현 (GSD + Superpowers)

여기서 Triple Crown의 핵심이 드러난다. GSD가 무엇을 할지 관리하고, Superpowers가 어떻게 할지를 강제한다.

GSD의 실행 관리

```
/gsd:execute-phase 1
```

GSD는 PLAN.md의 태스크를 순서대로 실행하며, 각 태스크의 진행 상태를 추적한다.

Superpowers의 품질 보장

GSD 실행 중 각 태스크에서 superpowers 스킬이 자동으로 개입한다.

시나리오 1: 새 기능 구현 시 — TDD 스킬

태스크: "WebSocket 서버 세팅"

superpowers:test-driven-development 적용:

Step 1 (Red): 실패하는 테스트 먼저 작성

```
test("클라이언트 연결 시 200 응답", () => {  
  const ws = new WebSocket("ws://localhost:3001");  
  expect(ws.readyState).toBe(WebSocket.OPEN);  
});
```

→ 테스트 실행: FAIL (서버 없음)

Step 2 (Green): 테스트 통과하는 최소 코드

```
const wss = new WebSocket.Server({ port: 3001 });  
wss.on("connection", (ws) => { /* 연결 수락 */ });
```

→ 테스트 실행: PASS

Step 3 (Refactor): 코드 정리

에러 핸들링, 연결 제한, 로깅 추가

→ 테스트 실행: PASS (리그레션 없음)

TDD 스킬이 없으면 Claude는 서버 코드를 먼저 작성하고 "테스트도 추가할까요?"라고 묻는다.
TDD 스킬이 있으면 반드시 테스트를 먼저 쓴다.

시나리오 2: 버그 발생 시 — 디버깅 스킬

문제: "두 클라이언트가 동시 편집 시 텍스트가 깨진다"

superpowers:systematic-debugging 적용:

Step 1 (관찰): 재현 조건 문서화

→ 클라이언트 A: "hello" 입력

→ 클라이언트 B: 동시에 "world" 입력

→ 결과: "hweolrlld" (기대: "helloworld" 또는 "worldhello")

Step 2 (가설): 원인 후보 나열

가설 1: Yjs 문서 동기화 지연

가설 2: WebSocket 메시지 순서 보장 실패

가설 3: 클라이언트 측 렌더링 타이밍

Step 3 (검증): 하나씩 테스트

가설 1: Yjs 로그 확인 → 동기화 정상

가설 2: WebSocket 메시지 로그 → 순서 뒤바뀜 확인 ✓

→ 원인: WebSocket 메시지 브로드캐스트 순서 미보장

Step 4 (수정): 시퀀스 넘버 추가 + 회귀 테스트

디버깅 스킬이 없으면 Claude는 코드를 보고 "이 부분이 문제인 것 같다"며 바로 수정을 시도한다. 디버깅 스킬이 있으면 가설을 세우고 검증하는 과정을 거친다. 감으로 코드를 고치다 다른 곳이 깨지는 악순환을 방지한다.

시나리오 3: 병렬 작업 시 — 에이전트 분배 스킬

팀 환경에서 여러 태스크를 동시에 진행할 때 사용한다.

superpowers:dispatching-parallel-agents 적용:

팀장 판단:

Task 3 (클라이언트-서버 연결) → 서연 (개발자)

Task 3의 UI 부분 (에디터 컴포넌트) → 수아 (디자이너)

→ 두 태스크는 인터페이스만 공유, 병렬 실행 가능

분배:

```
tmux send-keys -t team:0.4 \
```

```
"src/server/sync.ts 에서 WebSocket 동기화 핸들러를 구현해줘" Enter
```

```
tmux send-keys -t team:0.3 \
```

```
"src/components/Editor.tsx 에서 Yjs 연동 에디터 컴포넌트를 만들어줘" Enter
```

결합 조건:

서연의 서버 + 수아의 컴포넌트가 같은 프로토콜을 사용하도록

인터페이스 정의를 민준(PM)이 사전에 작성

Phase 4: 검증 (GSD + gstack)

구현이 끝나면 두 단계의 검증을 거친다.

4-1. GSD 검증 — 계획 대비 완성도

페이지 목표 달성 여부 확인

/gsd:validate-phase 1

출력:

Phase 1: CRDT 동기화 엔진

Task 1: Yjs 문서 모델 초기화 ☒ 완료

Task 2: WebSocket 서버 세팅 ☒ 완료

Task 3: 클라이언트-서버 동기화 연결 ☒ 완료

Task 4: 오프라인 편집 + 재접속  완료
Task 5: 동시 편집 충돌 테스트  완료

완료율: 5/5 (100%)

목표 달성:  텍스트 동기화 정상 동작 확인

목표 역방향 분석
/gsd:verify-work

출력:

목표: "실시간 텍스트 동기화"

검증 항목:

-  두 클라이언트 간 텍스트 동기화
-  동시 편집 시 충돌 해소
-  오프라인 편집 후 재접속 시 동기화
-  3명 이상 동시 편집 미테스트 (Phase 3에서 처리)

4-2. gstack 검증 — 코드 품질과 기능 완성도

코드 리뷰
/review

출력:

=== 코드 리뷰 ===

파일: src/server/sync.ts

-  코드 구조 양호
-  에러 핸들링: WebSocket 비정상 종료 시 cleanup 누락
-  보안: 연결 인증 없음 (Phase 2에서 추가 필요)

QA 검증
/qa

출력:

=== QA 결과 ===

테스트 시나리오 12건 실행

-  통과: 11건
-  실패: 1건
→ "100자 이상 동시 입력 시 1초 지연" (성능 임계값 초과)

권장 조치: 동기화 배치 간격 조정 (50ms → 100ms)

GSD 검증은 "계획한 것을 다 했는가"를 확인하고, gstack 검증은 "만든 것이 제대로 동작하는가"를 확인한다. 관점이 다르기 때문에 둘 다 필요하다.

Phase 5: 완료 (gstack + GSD)

모든 검증을 통과하면 작업을 마무리한다.

```
# 배포 준비
/ship

출력:
=== 배포 준비 ===
브랜치: feature/realtime-sync
변경 파일: 14개 (+1,245, -32)
테스트: 전체 통과 (47/47)

PR 생성:
Title: feat: add real-time text synchronization with CRDT
Body: [자동 생성된 변경 요약]

체크리스트:
✓ 코드 리뷰 완료
✓ QA 통과
✓ 충돌 없음
→ PR #15 생성 완료

# 마일스톤 상태 업데이트
/gsd:progress

출력:
M1: MVP 출시
 33% (Phase 1/3 완료)

다음: Phase 2 (커서 공유)
/gsd:plan-phase 2 로 시작
```

팀 환경에서의 Triple Crown 배분

6명의 팀 에이전트에게 Triple Crown의 각 단계를 분담하면 더 효율적이다.

Triple Crown 팀 배분

팀장의 Triple Crown 실행 스크립트

팀장이 전체 사이클을 오케스트레이션하는 예시이다.

```
#!/bin/bash
# triple-crown.sh - 기능 개발 전체 사이클
```

```

FEATURE="$1" # 예: "실시간 협업 에디터"

echo "=== Phase 1: 전략 수립 ==="
# 팀장이 /cso, /autoplan 실행 (Pane 0에서 직접)
echo "팀장: /cso와 /autoplan으로 전략 수립 완료"

echo "=== Phase 2: 구조화 ==="
# 민준에게 프로젝트 구조화 지시
tmux send-keys -t team:0.1 \
  "/gsd:new-project 으로 '$FEATURE' 프로젝트를 생성하고 \
  /gsd:plan-phase 1 까지 완료해줘." Enter

# 지훈에게 기술 조사 지시
tmux send-keys -t team:0.2 \
  "'$FEATURE'에 필요한 기술 스택을 조사하고 \
  /docs/research/ 에 정리해줘." Enter

echo "=== Phase 3: 구현 (Phase 2 완료 후) ==="
# 서연에게 구현 지시
tmux send-keys -t team:0.4 \
  "/gsd:execute-phase 1 을 실행해줘. \
  TDD로 진행하고, 각 태스크마다 테스트 먼저 작성해." Enter

# 수아에게 UI 구현 지시
tmux send-keys -t team:0.3 \
  "에디터 UI 컴포넌트를 구현해줘. \
  src/components/ 디렉토리에서 작업해." Enter

echo "=== Phase 4: 검증 (구현 완료 후) ==="
# 태양에게 검증 지시
tmux send-keys -t team:0.5 \
  "서연과 수아의 구현 결과를 /review 하고 \
  /qa 로 기능 검증해줘." Enter

# 민준에게 GSD 검증 지시
tmux send-keys -t team:0.1 \
  "/gsd:validate-phase 1 로 Phase 1 완료 여부를 확인해줘." Enter

echo "=== Phase 5: 완료 (검증 통과 후) ==="
echo "팀장: /ship 으로 배포 준비"

```

Triple Crown vs 단순 개발 비교

같은 기능을 Triple Crown 없이 만드는 경우와 비교한다.

단순 개발

Triple Crown 개발 파이프라인

소규모 작업(파일 하나 수정, 간단한 스크립트)에는 Triple Crown이 과하다. 하지만 여러 파일에 걸친 기능 개발, 팀 단위 프로젝트, 프로덕션 배포가 필요한 작업에서는 초기 투자 시간을 재작업 감소로 회수할 수 있다.

적용 기준

모든 작업에 Triple Crown을 적용할 필요는 없다. 상황에 따라 단계를 선택한다.

작업 규모	권장 조합	예시
소 (1시간 이내)	도구 없이 직접	오타 수정, 설정 변경
중 (반나절)	gstack만 (/review + /ship)	버그 수정, 작은 기능 추가
대 (1~3일)	gstack + Superpowers	새 기능 모듈, API 추가
특대 (1주 이상)	Triple Crown 전체	신규 서비스, 대규모 리팩토링

요약

Triple Crown 전략은 gstack(전략·검증), GSD(구조·실행), Superpowers(품질·방법론)를 하나의 파이프라인으로 연결하여 "무엇을 왜 → 어떤 순서로 → 어떻게 잘"의 질문에 순서대로 답하는 워크플로우다. 팀 환경에서는 각 Phase를 역할에 맞는 팀원에게 배분하여 병렬로 실행할 수 있다. 모든 작업에 적용할 필요는 없지만, 규모가 큰 프로젝트에서는 초기 구조화 비용을 재작업 감소로 충분히 회수한다.

7-5. 자동화 워크플로우 예시

워크플로우 자동화란

지금까지 팀 구성, Remote-Control, Bot Mode, 업무 분담을 각각 다루었다. 이 장에서는 이 모든 요소를 하나의 자동화된 워크플로우로 연결하는 실전 예시를 다룬다.

자동화 워크플로우의 핵심은 사용자의 한 마디 지시가 여러 단계의 작업으로 자동 전개되는 것이다.

사용자: "새 기능 만들어줘"

|



[자동] 조사 → 설계 → 구현 → 리뷰 → 보고

워크플로우 1: 기능 개발 파이프라인

가장 일반적인 워크플로우다. 새 기능을 처음부터 끝까지 개발하는 전체 과정이다.

사용자 지시

@cc 사용자 알림 시스템 만들어줘.
이메일과 푸시 알림을 지원하고,
알림 설정 화면도 필요해.

팀장의 워크플로우 실행

```
# =====  
# Phase 1: 조사 + 디자인 (병렬)  
# =====  
  
# 지훈에게 기술 조사 지시  
tmux send-keys -t team:0.2 \  
"알림 시스템 기술 스택을 조사해줘. \  
이메일(SMTP/SendGrid), 푸시 알림(FCM/APNs), \  
알림 큐(Redis/RabbitMQ) 각각 비교 분석하고 \  
/docs/research/notification-stack.md 에 정리해줘." Enter  
  
# 수아에게 UI 설계 지시  
tmux send-keys -t team:0.3 \  
"사용자 알림 설정 화면을 설계해줘. \  
"
```

```

이메일/푸시 알림 on/off 토글, 알림 종류별 설정, \
알림 이력 목록. \
/docs/design/notification-ui.md 에 정리해줘." Enter

# =====
# Phase 2: 아키텍처 설계 (조사 완료 후)
# =====

# 민준에게 설계 지시
tmux send-keys -t team:0.1 \
  "지훈의 조사(/docs/research/notification-stack.md)와 \
  수아의 UI 설계(/docs/design/notification-ui.md)를 참고해서 \
  알림 시스템 아키텍처를 설계해줘. \
  API 엔드포인트, 데이터 모델, 알림 전송 플로우 포함. \
  /docs/architecture/notification-system.md 에 정리해줘." Enter

# =====
# Phase 3: 구현 (설계 완료 후)
# =====

# 서연에게 구현 지시
tmux send-keys -t team:0.4 \
  "민준의 설계(/docs/architecture/notification-system.md)에 따라 \
  알림 시스템을 구현해줘. \
  1) 알림 서비스 모듈 \
  2) 이메일 전송 어댑터 \
  3) 푸시 알림 어댑터 \
  4) REST API 엔드포인트 \
  5) 알림 설정 UI 컴포넌트 \
  테스트 코드도 함께 작성해줘." Enter

# =====
# Phase 4: 리뷰 (구현 완료 후)
# =====

# 태양에게 리뷰 지시
tmux send-keys -t team:0.5 \
  "서연이 구현한 알림 시스템 코드를 리뷰해줘. \
  특히 보안(인젝션, 인증), 성능(대량 알림 처리), \
  에러 핸들링을 중점적으로 확인해줘." Enter

```

사용자에게 최종 보고

📋 작업 요약:

- 기술 스택: SendGrid(이메일) + FCM(푸시) + Redis(큐)
- API: POST /api/notifications, GET /api/notification-settings

- UI: 알림 설정 페이지 + 알림 이력 목록

📁 주요 파일:

- src/services/notification.ts
- src/adapters/email.ts, push.ts
- src/pages/NotificationSettings.tsx
- tests/notification.test.ts

✅ 리뷰: 태양 승인 (보안·성능 이슈 없음)

📝 커밋: feat: add notification system with email and push support'

워크플로우 2: 버그 핫픽스

프로덕션 장애 상황에서의 긴급 대응 워크플로우다.

사용자 지시

@cc 프로덕션 결제 오류 발생.
사용자가 결제 완료했는데 주문이 생성 안 됨.
긴급 수정해줘.

팀장의 긴급 워크플로우

```
# =====
# Phase 1: 원인 분석 (즉시)
# =====

# 지훈에게 긴급 분석 지시
tmux send-keys -t team:0.2 \
  "[긴급] 결제 완료 후 주문 미생성 버그 분석. \
  1) 최근 커밋 중 결제/주문 관련 변경 확인 \
  2) 에러 로그에서 관련 에러 검색 \
  3) 결제 → 주문 생성 코드 흐름 추적 \
  원인을 찾으면 즉시 보고해줘." Enter

# =====
# Phase 2: 핫픽스 구현 (원인 파악 후)
# =====

# 서연에게 수정 지시
tmux send-keys -t team:0.4 \
  "[긴급] 지훈이 분석한 결제-주문 버그 수정. \
  원인: {지훈의 분석 결과} \
  수정 후 관련 테스트 추가해줘. \
  브랜치명: hotfix/payment-order-creation" Enter
```

```
# =====
# Phase 3: 긴급 리뷰 (수정 후)
# =====

# 태양에게 긴급 리뷰
tmux send-keys -t team:0.5 \
  "[긴급] hotfix/payment-order-creation 브랜치 긴급 리뷰. \
  결제-주문 생성 버그 수정 건. \
  사이드 이펙트 없는지 집중 확인해줘." Enter
```

긴급 보고

🔍 원인: 결제 콜백에서 주문 생성 트랜잭션 커밋 누락
 🔧 수정: payment-callback.ts L45 - 트랜잭션 커밋 추가
 ✅ 리뷰: 태양 긴급 승인
 🌿 브랜치: hotfix/payment-order-creation

배포 승인해주시면 즉시 반영합니다.'

워크플로우 3: 코드 리뷰 자동화

PR이 올라올 때마다 자동으로 리뷰를 수행하는 워크플로우다.

사용자 지시

@cc PR #42 리뷰해줘

팀장의 리뷰 워크플로우

```
# =====
# Phase 1: 변경 사항 분석 (병렬)
# =====

# 지훈에게 변경 분석
tmux send-keys -t team:0.2 \
  "PR #42의 변경 사항을 분석해줘. \
  변경된 파일 목록, 변경 라인 수, \
  영향받는 모듈을 정리해줘." Enter

# 태양에게 코드 리뷰
tmux send-keys -t team:0.5 \
```

```
"PR #42 코드 리뷰 수행. \
gh pr diff 42 로 변경 사항 확인하고 \
코드 품질, 보안, 성능 관점에서 리뷰해줘. \
개선점이 있으면 구체적인 코드와 함께 제안해줘." Enter
```

```
# =====
# Phase 2: 결과 종합 및 보고
# =====
```



변경 요약:

- 변경 파일: 8개 (+245, -89)
- 영향 모듈: auth, user-profile



리뷰 결과:



코드 컨벤션: 준수



보안: 이슈 없음



성능: user-profile 쿼리에 N+1 가능성

→ 제안: eager loading 적용



코멘트 2건 등록 완료'

워크플로우 4: 일일 팀 보고

매일 정해진 시간에 각 팀원의 작업 현황을 수집하여 보고하는 워크플로우다.

사용자 지시

```
@cc 오늘 팀 현황 정리해줘
```

팀장의 현황 수집

```
# 모든 팀원에게 현황 보고 요청 (브로드캐스트)
tmux send-keys -t team:0.1 "오늘 작업 현황을 한 줄로 보고해줘" Enter
tmux send-keys -t team:0.2 "오늘 작업 현황을 한 줄로 보고해줘" Enter
tmux send-keys -t team:0.3 "오늘 작업 현황을 한 줄로 보고해줘" Enter
tmux send-keys -t team:0.4 "오늘 작업 현황을 한 줄로 보고해줘" Enter
tmux send-keys -t team:0.5 "오늘 작업 현황을 한 줄로 보고해줘" Enter
```

종합 보고



민준 (PM): 알림 시스템 API 스펙 문서 작성 완료

- 👤 지훈 (리서처): Redis vs RabbitMQ 비교 분석 완료
- 👤 수아 (디자이너): 알림 설정 UI 와이어프레임 완성
- 👤 서연 (개발자): 알림 서비스 모듈 구현 중 (70%)
- 👤 태양 (리뷰어): PR #41 리뷰 완료, #42 대기 중

📌 내일 예정:

- 서연: 알림 서비스 구현 완료 → 태양 리뷰
- 민준: 배포 계획 수립'

워크플로우 5: 셋업 스크립트 자동화

팀 환경 자체를 스크립트로 자동 구성하는 워크플로우다.

```
#!/bin/bash
# setup-team.sh - 팀 환경 원클릭 셋업

SESSION="team"

# 1. TMUX 세션 생성
tmux new-session -d -s $SESSION -x 200 -y 50

# 2. 파인 분할
tmux split-window -h -t $SESSION:0
tmux split-window -v -t $SESSION:0.1
tmux split-window -v -t $SESSION:0.2
tmux split-window -v -t $SESSION:0.3
tmux split-window -v -t $SESSION:0.4

# 3. 레이아웃 설정
tmux select-layout -t $SESSION main-vertical

# 4. 파인 타이틀 설정
tmux select-pane -t $SESSION:0.0 -T "쑤"
tmux select-pane -t $SESSION:0.1 -T "민준 PM·아키텍트"
tmux select-pane -t $SESSION:0.2 -T "지훈 리서처"
tmux select-pane -t $SESSION:0.3 -T "디자이너 수아"
tmux select-pane -t $SESSION:0.4 -T "서연 개발자"
tmux select-pane -t $SESSION:0.5 -T "태양 리뷰어"

# 5. 각 파인에서 Claude Code 실행
for i in 0 1 2 3 4 5; do
    tmux send-keys -t $SESSION:0.$i "claude" Enter
done

# 6. 파인 테두리에 타이틀 표시
tmux set-option -t $SESSION pane-border-status top
```

```
echo "팀 환경 셋업 완료! tmux attach -t $SESSION 으로 접속하세요."
```

이 스크립트를 실행하면 6개의 파인에 각각 Claude Code가 실행된 팀 환경이 자동으로 구성된다.

워크플로우 커스터마이징 팁

1. 반복 작업은 셸 함수로

```
# ~/.bashrc에 추가
team-send() {
    local pane=$1
    shift
    tmux send-keys -t team:0.$pane "$*" Enter
}

# 사용 예시
team-send 4 "로그인 버그 수정해줘" # 서연에게
team-send 5 "PR #42 리뷰해줘"     # 태양에게
```

2. 워크플로우 템플릿 저장

자주 사용하는 워크플로우를 스크립트로 저장해두면 재사용이 편리하다.

```
# ~/workflows/feature-dev.sh
#!/bin/bash
FEATURE_NAME=$1

team-send 2 "[$FEATURE_NAME] 기술 조사 시작해줘"
team-send 3 "[$FEATURE_NAME] UI 설계 시작해줘"
echo "Phase 1 시작: 조사 + 디자인 병렬 진행"
```

3. 팀원 상태 모니터링

```
# 모든 팀원의 마지막 출력 확인
for i in 1 2 3 4 5; do
    echo "=== Pane $i ==="
    tmux capture-pane -t team:0.$i -p | tail -3
    echo ""
done
```

핵심 정리: 자동화 워크플로우는 사용자의 단일 지시를 다단계 팀 작업으로 자동 전개한다. 기능 개발, 버그 수정, 코드 리뷰, 일일 보고 등 반복되는 패턴을 워크플로우로 정형화하면 팀 운용 효율이 크게 향상된다.

8-1. Remote-Control 인증 오류 해결

Remote Control을 사용하다 보면 인증 관련 오류를 마주칠 때가 있다. 대부분의 문제는 토큰 만료, 계정 불일치, 네트워크 설정에서 발생한다. 이 절에서는 자주 발생하는 인증 오류와 해결 방법을 정리한다.

증상별 진단 가이드

1. "Session not found" — 세션을 찾을 수 없음

모바일에서 세션 목록이 비어있거나 연결 시 "Session not found" 오류가 표시된다.

원인과 해결:

원인	확인 방법	해결
Claude Code가 실행 중이 아님	<code>pgrep -f claude</code>	Claude Code 재실행
서버 모드가 아님	터미널에서 Remote Control 상태 확인	<code>--remote-control</code> 플래그로 재시작
다른 Anthropic 계정	모바일과 서버의 계정 비교	동일 계정으로 로그인
세션 이름 불일치	<code>claude sessions list</code>	올바른 세션 이름으로 접속

```
# Claude Code 실행 상태 확인
pgrep -f claude
# 출력 없으면 실행 중이 아님

# Claude Code를 서버 모드로 재시작
claude --remote-control
```

2. "Authentication failed" — 인증 실패

연결 시도 시 인증 실패 오류가 발생한다.

해결 과정:

```
# 1. 현재 로그인된 계정 확인
claude auth status
```

```
# 2. 인증 토큰이 만료되었으면 재인증
claude auth login
```

```
# 3. 브라우저에서 Anthropic 계정으로 로그인 후
#   터미널에 표시되는 인증 코드 입력
```

3. "Connection timeout" — 연결 시간 초과

모바일에서 세션을 선택했지만 연결이 되지 않고 타임아웃이 발생한다.

원인과 해결:

```
# 네트워크 연결 확인
ping -c 3 api.anthropic.com

# DNS 확인
nslookup api.anthropic.com

# 방화벽이 HTTPS 아웃바운드를 차단하는 경우
sudo ufw allow out 443/tcp
```

WSL2 환경에서는 Windows 방화벽이 아웃바운드 연결을 차단하는 경우가 있다. Windows Defender 방화벽 설정에서 WSL2 관련 규칙을 확인한다.

4. "Permission denied" — 권한 거부

세션에는 연결되지만 도구 실행 시 "Permission denied"가 반복된다.

```
# settings.json의 deny 규칙 확인
cat ~/.claude/settings.json | grep -A 20 "deny"

# 프로젝트별 설정도 확인
cat .claude/settings.json | grep -A 20 "deny"
```

과도한 `deny` 규칙이 정상 명령어까지 차단하고 있을 수 있다. 패턴을 좁히거나 불필요한 규칙을 제거한다.

인증 토큰 관리

토큰 만료 주기

Anthropic 인증 토큰은 일정 기간이 지나면 만료된다. 장기간 실행 중인 Claude Code 세션에서 갑자기 인증 오류가 발생한다면 토큰 만료를 의심한다.

```
# 인증 상태 확인
claude auth status

# 출력 예시:
# Logged in as: user@example.com
# Token expires: 2026-05-15T00:00:00Z

# 토큰 갱신
claude auth login
```

여러 계정 사용 시

팀 환경에서 여러 Anthropic 계정을 사용하는 경우 계정 전환에 주의한다.

```
# 현재 계정 확인
claude auth whoami

# 로그아웃 후 다른 계정으로 전환
claude auth logout
claude auth login
```

TMUX 팀 환경에서의 인증 문제

모든 파인이 동시에 인증 실패하는 경우

팀 세션의 모든 파인이 같은 계정을 사용한다면, 토큰 만료 시 전체가 동시에 실패한다.

```
# 모든 파인에서 재인증 (브로드캐스트)
for i in 0 1 2 3 4 5; do
    tmux send-keys -t team:0.$i "/login" Enter
done
```

특정 파인만 인증 실패하는 경우

하나의 파인만 문제가 있다면 해당 파인의 Claude Code 프로세스를 개별적으로 재시작한다.

```
# 문제 파인의 Claude Code 재시작
tmux send-keys -t team:0.3 "/exit" Enter
sleep 2
tmux send-keys -t team:0.3 "claude" Enter
```

진단 명령어 모음

빠른 진단을 위한 명령어를 정리한다.

```
# 1. Claude Code 프로세스 확인
pgrep -af claude

# 2. 인증 상태 확인
claude auth status

# 3. 네트워크 연결 확인
curl -s -o /dev/null -w "%{http_code}" https://api.anthropic.com/v1/messages
# 200 또는 401이면 네트워크 정상 (401은 인증만 실패)

# 4. 로그 확인
ls -lt ~/.claude/logs/ | head -5
tail -50 ~/.claude/logs/latest.log

# 5. 설정 파일 유효성 확인
python3 -c "import json; json.load(open('$HOME/.claude/settings.json'))" \
    && echo "유효한 JSON" \
    || echo "JSON 구문 오류"
```

요약

Remote Control 인증 문제의 90%는 토큰 만료, 계정 불일치, 네트워크 차단 중 하나다. `claude auth status` 로 인증 상태를 확인하고, `pgrep` 으로 프로세스 생존을 확인하고, `curl` 로 네트워크 연결을 확인하는 세 단계 진단만으로 대부분의 문제를 해결할 수 있다.

8-2. TMUX 세션 복구 및 재연결

TMUX 기반 팀 환경을 운용하다 보면 세션이 끊기거나, 파인이 사라지거나, 레이아웃이 깨지는 상황이 발생한다. 이 절에서는 팀 세션을 빠르게 복구하는 방법을 다룬다.

흔한 문제 상황

상황	원인	위험도
SSH 연결 끊김	네트워크 불안정	낮음 — 세션은 살아있음
특정 파인의 Claude 종료	OOM, 수동 종료	중간 — 해당 파인만 복구
TMUX 서버 크래시	시스템 리소스 부족	높음 — 전체 재구성 필요
시스템 재부팅	업데이트, 정전	높음 — 전체 재구성 필요

상황 1: SSH 연결이 끊어졌을 때

가장 흔하고 가장 안전한 상황이다. TMUX 세션은 SSH 연결과 독립적으로 유지된다.

```
# SSH 재접속 후 세션 확인
tmux ls
# 출력: team: 1 windows (created Mon Apr 28 10:00:00 2026)

# 세션에 재연결
tmux attach -t team
```

연결만 끊어진 것이므로 모든 파인의 Claude Code는 그대로 실행 중이다. 아무런 데이터 손실 없이 그대로 이어서 작업할 수 있다.

상황 2: 특정 파인의 Claude가 종료되었을 때

한 팀원의 Claude Code가 종료되었지만 파인 자체는 살아있는 경우이다.

진단

```
# 각 파인의 상태 확인
for i in 0 1 2 3 4 5; do
    echo -n "Pane $i: "
    tmux send-keys -t team:0.$i "" 2>/dev/null \
        && echo "활성" \
        || echo "비활성"
done
```

파인에서 Claude Code가 종료되면 일반 셸 프롬프트(`$`)가 표시된다.

복구

```
# 해당 파인에서 Claude Code 재실행
tmux send-keys -t team:0.3 "claude" Enter
```

Claude Code는 이전 대화 히스토리를 로컬에 저장하므로, 재실행 후 `/resume` 명령으로 이전 세션을 이어갈 수 있다.

```
# Claude Code 재실행 후 이전 세션 복구
tmux send-keys -t team:0.3 "/resume" Enter
```

상황 3: 파인 자체가 사라졌을 때

TMUX 파인이 닫히면 복구할 수 없다. 새 파인을 생성하고 재설정해야 한다.

현재 파인 구조 확인

```
tmux list-panes -t team:0 -F "#{pane_index}: #{pane_title} (#{pane_pid})"
# 출력 예시:
# 0: 쏘 (1234)
# 1: 민준 PM·아키텍트 (1235)
# 2: 지훈 리서처 (1236)
# 4: 서연 개발자 (1238)
# 5: 태양 리뷰어 (1239)
# → 파인 3(수아)이 없음
```

파일 재생성

```
# 기존 파일 옆에 새 파일 분할
tmux split-window -v -t team:0.2

# 새 파일에 타이틀 설정
tmux select-pane -t team:0.3 -T "디자이너 수아"

# Claude Code 실행
tmux send-keys -t team:0.3 "claude" Enter

# 레이아웃 재정렬
tmux select-layout -t team:0 main-vertical
```

상황 4: TMUX 세션 전체가 소실되었을 때

시스템 재부팅이나 TMUX 서버 크래시로 세션이 완전히 사라진 경우이다. 셋업 스크립트로 전체를 재구성한다.

```
# 세션 존재 여부 확인
tmux ls 2>/dev/null || echo "세션 없음"

# 셋업 스크립트로 재구성
bash /mnt/c/work/Team/setup-team.sh

# 세션에 접속
tmux attach -t team
```

셋업 스크립트는 TMUX 세션 생성, 파일 분할, 타이틀 설정, Claude Code 실행을 자동으로 수행한다.

자동 복구 스크립트

수동 복구가 번거롭다면 상태를 점검하고 자동 복구하는 스크립트를 작성할 수 있다.

```
#!/bin/bash
# check-team.sh - 팀 세션 상태 점검 및 복구

SESSION="team"
EXPECTED_PANES=6
TITLES=("쫘" "민준 PM·아키텍트" "지훈 리서처" "디자이너 수아" "서연 개발자" "태양 리뷰어")
```

```

# 1. 세션 존재 확인
if ! tmux has-session -t $SESSION 2>/dev/null; then
    echo "세션이 없습니다. 재구성합니다..."
    bash /mnt/c/work/Team/setup-team.sh
    exit 0
fi

# 2. 파인 수 확인
CURRENT_PANES=$(tmux list-panes -t $SESSION:0 | wc -l)
if [ "$CURRENT_PANES" -ne "$EXPECTED_PANES" ]; then
    echo "파인 수 불일치: $CURRENT_PANES / $EXPECTED_PANES"
    echo "전체 재구성을 권장합니다."
    exit 1
fi

# 3. 각 파인에서 Claude 실행 상태 확인
for i in 0 1 2 3 4 5; do
    PANE_CMD=$(tmux list-panes -t $SESSION:0 -F "#{pane_index}
#{pane_current_command}" \
    | grep "^$i " | awk '{print $2}')

    if [ "$PANE_CMD" != "claude" ] && [ "$PANE_CMD" != "node" ]; then
        echo "Pane $i (${TITLES[$i]}): Claude 미실행 - 재시작합니다"
        tmux send-keys -t $SESSION:0.$i "claude" Enter
    else
        echo "Pane $i (${TITLES[$i]}): 정상"
    fi
done

echo "점검 완료"

```

이 스크립트를 cron에 등록하면 주기적으로 팀 상태를 점검할 수 있다.

```

# 매 10분마다 팀 상태 점검
crontab -e
# 추가:
# */10 * * * * /home/user/check-team.sh >> /tmp/team-check.log 2>&1

```

레이아웃 복구

파인 수는 맞지만 레이아웃이 깨진 경우이다.


```
# main-vertical 레이아웃 재적용
tmux select-layout -t team:0 main-vertical

# Pane 0 너비를 158로 고정
tmux resize-pane -t team:0.0 -x 158

# 파인 테두리에 타이틀 표시 재설정
tmux set-option -t team pane-border-status top
```

예방 조치

TMUX 자동 저장 플러그인

tmux-resurrect 플러그인을 사용하면 세션 상태를 파일로 저장하고 복원할 수 있다.

```
# tmux-resurrect 설치
git clone https://github.com/tmux-plugins/tmux-resurrect \
  ~/.tmux/plugins/tmux-resurrect

# ~/.tmux.conf에 추가
# run-shell ~/.tmux/plugins/tmux-resurrect/resurrect.tmux
```

키 바인딩	동작
<code>Ctrl-b</code> + <code>Ctrl-s</code>	현재 세션 상태 저장
<code>Ctrl-b</code> + <code>Ctrl-r</code>	마지막 저장된 상태 복원

시스템 리소스 모니터링

Claude Code는 메모리를 많이 사용한다. 6개 인스턴스를 동시에 실행하면 상당한 메모리가 필요하다.

```
# Claude Code 프로세스별 메모리 사용량 확인
ps aux | grep claude | grep -v grep | awk '{print $6/1024 "MB", $0}' | sort -rn
```

메모리 부족으로 Claude가 OOM(Out of Memory)으로 종료되는 것을 방지하려면 충분한 RAM(16GB 이상 권장)을 확보하거나, 동시 실행 파인 수를 줄이는 것을 고려한다.

요약

TMUX 세션 복구의 핵심은 단계별 진단이다. SSH 끊김은 재연결만 하면 되고, 파인 내 Claude 종료는 재실행으로 해결되고, 파인 소실은 재생성이 필요하고, 세션 전체 소실은 셋업 스크립트로 재구성한다. [check-team.sh](https://github.com/leventhe03/check-team.sh) 같은 자동 점검 스크립트와 `tmux-resurrect` 플러그인을 활용하면 복구 시간을 크게 줄일 수 있다.

8-3. 토큰 최적화 심화

6장에서 RTK를 통한 기본적인 토큰 절약을 다루었다. 이 절에서는 RTK를 넘어서 Claude Code 자체의 설정과 사용 습관을 통해 토큰 소비를 최소화하는 심화 전략을 다룬다.

토큰이 소비되는 곳

Claude Code에서 토큰이 소비되는 주요 경로를 이해해야 최적화 포인트를 찾을 수 있다.

토큰 소비 구조

[입력 토큰]

- 시스템 프롬프트 (CLAUDE.md 등)
- 대화 히스토리 (이전 메시지)
- 도구 결과 (Bash 출력, 파일 내용)
- 컨텍스트 파일 (자동 로드)

[출력 토큰]

- Claude의 응답 텍스트
- 도구 호출 요청
- 사고 과정 (thinking)

이 중 가장 큰 비중을 차지하는 것은 도구 결과와 대화 히스토리다.

전략 1: CLAUDE.md 최적화

CLAUDE.md는 매 요청마다 시스템 프롬프트에 포함된다. 내용이 길수록 매번 더 많은 토큰이 소비된다.

비효율적인 예

```
# CLAUDE.md
이 프로젝트는 2026년 1월에 시작된 웹 애플리케이션입니다.
React와 TypeScript를 사용하며, 백엔드는 Node.js Express입니다.
데이터베이스는 PostgreSQL을 사용합니다.
테스트는 Jest와 React Testing Library를 사용합니다.
린트는 ESLint, 포맷터는 Prettier를 사용합니다.
CI/CD는 GitHub Actions를 사용합니다.
배포 대상은 AWS ECS입니다.
```

...

(50줄의 일반적인 설명)

최적화된 예

```
# CLAUDE.md
## 스택
React 19 + TS, Express, PostgreSQL, Jest

## 규칙
- 테스트: `npm test -- --watch`
- 린트: `npm run lint`
- 커밋 전 `npm run check` 필수
```

프레임워크 이름이나 라이브러리 목록은 `package.json` 을 읽으면 알 수 있다. CLAUDE.md에는 코드에서 알 수 없는 규칙과 관례만 기록한다.

전략 2: 대화 세션 관리

대화가 길어질수록 히스토리가 누적되어 입력 토큰이 증가한다.

작업 단위로 세션 분리

```
# 나쁜 예: 하나의 세션에서 모든 작업
claude
> 기능 A 구현해줘
> ...100턴의 대화...
> 기능 B도 해줘
> ...또 100턴...

# 좋은 예: 작업별 세션 분리
claude  # 세션 1: 기능 A
> 기능 A 구현해줘
> ...완료...
> /exit

claude  # 세션 2: 기능 B
> 기능 B 구현해줘
```

/compact 명령어 활용

대화 중간에 컨텍스트를 압축하여 토큰 사용량을 줄일 수 있다.

```
> /compact
# Claude가 이전 대화를 요약하여 컨텍스트를 압축한다
# 이후 요청부터 입력 토큰이 줄어든다
```

긴 작업 중간에 주기적으로 `/compact` 를 실행하면 누적 토큰을 억제할 수 있다.

전략 3: 도구 출력 최적화

도구 실행 결과는 입력 토큰의 가장 큰 부분을 차지한다.

파일 읽기 범위 지정

```
# 나쁜 예: 전체 파일 읽기 (2000줄)
# Claude가 Read 도구로 전체 파일을 읽음

# 좋은 예: 필요한 부분만 읽기
# "main.py의 45~60번 줄을 확인해줘"라고 지시
```

지시를 구체적으로 하면 Claude가 파일의 필요한 부분만 읽는다.

검색 범위 제한

```
# 나쁜 예: 전체 프로젝트 검색
grep -r "function" .

# 좋은 예: 디렉토리 및 파일 형식 제한
grep -r "function" src/services/ --include="*.ts"
```

.claudeignore 활용

불필요한 파일을 Claude의 탐색 범위에서 제외한다.

```
# .claudeignore
node_modules/
dist/
build/
coverage/
*.min.js
*.map
*.lock
```

특히 `node_modules/`, `dist/` 같은 대용량 디렉토리를 제외하면 `ls` 나 `find` 명령어의 출력이 크게 줄어든다.

전략 4: 프롬프트 캐싱 활용

Anthropic API는 프롬프트 캐싱을 지원한다. Claude Code에서는 자동으로 적용되지만, 그 원리를 이해하면 더 효과적으로 활용할 수 있다.

캐시 적중 조건:

시스템 프롬프트가 동일 + 대화 시작 부분이 동일
→ 캐시된 토큰은 비용 90% 할인

캐시가 깨지는 조건:

CLAUDE.md 수정 → 시스템 프롬프트 변경 → 캐시 무효화

CLAUDE.md를 자주 수정하면 캐시가 무효화되어 비용이 증가한다. 안정적인 내용은 CLAUDE.md에, 자주 변하는 지시는 대화에서 직접 전달하는 것이 유리하다.

전략 5: 팀 환경 토큰 관리

6개의 Claude Code 인스턴스를 동시 실행하면 토큰 소비가 6배로 늘어난다. 팀 환경에서의 토큰 관리 전략이다.

필요한 파인만 활성화

모든 팀원이 항상 필요한 것은 아니다.

```
# 현재 작업에 불필요한 파인의 Claude Code를 일시 중지
# 파인 자체는 유지하되 Claude를 종료
tmux send-keys -t team:0.3 "/exit" Enter
```

역할별 **CLAUDE.md** 최적화

각 팀원의 CLAUDE.md를 역할에 맞게 최소화한다.

```
# 리뷰어 전용 CLAUDE.md (짧게)
## 역할
코드 리뷰어. 보안·성능·코드 품질 중심 리뷰.
```

```
## 명령어
- `gh pr diff {번호}` - PR 변경 사항 확인
```

PM에게는 아키텍처 문서 경로만, 개발자에게는 빌드 명령어만 포함하는 식으로 각 역할에 필요한 최소한의 정보만 담는다.

RTK 팀 전체 적용

RTK 혹은 각 파인에서 독립적으로 동작한다. 모든 파인에 RTK가 적용되어 있는지 확인한다.

```
# 각 파인의 RTK 설정 확인
for i in 0 1 2 3 4 5; do
  echo "=== Pane $i ==="
  tmux send-keys -t team:0.$i "rtk --version" Enter
done
```

토큰 사용량 모니터링

RTK gain으로 확인

```
rtk gain
# 명령어별 토큰 절약량 확인

rtk gain --history
# 시간대별 추이 확인
```

Claude Code 내장 사용량

대화 중 `/cost` 명령어로 현재 세션의 토큰 사용량을 확인할 수 있다.

```
> /cost
Input tokens: 45,230
Output tokens: 12,100
Cache read: 38,000 (84% hit rate)
Estimated cost: $0.42
```

최적화 효과 비교

전략	절약률	적용 난이도
RTK 적용	~63%	낮음 (설치 후 자동)
.claudeignore 설정	~20-40%	낮음
CLAUDE.md 간소화	~10-15%	낮음
세션 분리 + /compact	~30-50%	중간 (습관 필요)
구체적 지시 습관	~20-30%	중간 (습관 필요)
불필요 파인 비활성화	해당 파인 100%	낮음

이 전략들은 중첩 적용이 가능하다. RTK + .claudeignore + CLAUDE.md 간소화만 적용해도 전체 토큰 소비를 절반 이하로 줄일 수 있다.

요약

토큰 최적화는 RTK 같은 도구 적용만으로 끝나지 않는다. CLAUDE.md를 간결하게 유지하고, 대화 세션을 작업 단위로 분리하고, 도구 출력 범위를 제한하고, .claudeignore로 불필요한 파일을 제외하는 종합적인 접근이 필요하다. 팀 환경에서는 필요한 파인만 활성화하고 역할별로 최적화된 설정을 적용하면 6배의 토큰 소비를 효율적으로 관리할 수 있다.

8-4. 멀티에이전트 충돌 방지

여러 Claude Code 인스턴스가 동시에 같은 코드베이스에서 작업하면 충돌이 발생할 수 있다. 두 팀원이 같은 파일을 동시에 수정하거나, 한 팀원의 변경이 다른 팀원의 작업을 무효화하는 상황이다. 이 절에서는 멀티에이전트 환경에서 충돌을 예방하고 관리하는 전략을 다룬다.

충돌이 발생하는 유형

유형 1: 파일 동시 수정

서연 (Pane 4): `src/api/users.ts` 수정 중
↓ 동시에
태양 (Pane 5): `src/api/users.ts` 리뷰 후 직접 수정
→ 한쪽의 변경이 덮어씌워짐

유형 2: 의존성 충돌

서연: 새 패키지 추가 (`npm install axios`)
↓ 동시에
민준: `package.json` 수동 수정 중
→ `package.json` / `package-lock.json` 충돌

유형 3: Git 충돌

서연: `feature/notification` 브랜치에서 커밋
↓ 동시에
지훈: 같은 브랜치에서 다른 파일 커밋
→ push 실패 또는 머지 충돌

전략 1: 영역 분리

가장 효과적인 충돌 방지 방법은 작업 영역을 물리적으로 분리하는 것이다.

디렉토리 기반 분리

각 팀원에게 담당 디렉토리를 배정한다.

프로젝트 구조:

```
├─ src/
│   ├─ api/          ← 서연 담당
│   ├─ components/   ← 수아 담당
│   ├─ services/      ← 민준 담당
│   └─ utils/         ← 공용 (팀장 승인 후 수정)
├─ tests/             ← 태양 담당
└─ docs/              ← 지훈 담당
```

팀장이 지시를 내릴 때 담당 영역을 명시한다.

```
# 명확한 영역 지정
tmux send-keys -t team:0.4 \
  "src/api/ 디렉토리에서 알림 API 엔드포인트를 구현해줘. \
  다른 디렉토리의 파일은 수정하지 마." Enter
```

브랜치 기반 분리

각 팀원이 독립된 Git 브랜치에서 작업한다.

```
# 팀원별 브랜치 생성
tmux send-keys -t team:0.4 \
  "git checkout -b feature/notification-api 에서 작업해줘" Enter

tmux send-keys -t team:0.3 \
  "git checkout -b feature/notification-ui 에서 작업해줘" Enter
```

브랜치가 분리되면 파일 수준의 충돌이 커밋 시점에서는 발생하지 않는다. 충돌은 머지 시점에만 발생하며, 팀장이 통제할 수 있다.

Git Worktree 활용

같은 저장소의 다른 브랜치를 별도 디렉토리로 체크아웃하는 Git Worktree를 사용하면 브랜치 전환 없이 병렬 작업이 가능하다.

```
# 팀원별 worktree 생성
git worktree add ../project-suyeon feature/notification-api
git worktree add ../project-sua feature/notification-ui

# 서연은 ../project-suyeon/ 에서 작업
# 수아는 ../project-sua/ 에서 작업
```

각 팀원의 Claude Code가 다른 디렉토리에서 실행되므로 파일 시스템 수준에서 완전히 격리된다.

전략 2: 잠금 규칙

영역 분리가 불가능한 경우 잠금 규칙을 도입한다.

파일 잠금 컨벤션

```
# 잠금 파일 생성
echo "서연 작업중 $(date)" > src/api/users.ts.lock

# 작업 완료 후 잠금 해제
rm src/api/users.ts.lock
```

이 컨벤션을 CLAUDE.md에 명시한다.

```
# CLAUDE.md 잠금 규칙
## 파일 잠금
- 파일 수정 전 `.lock` 파일 존재 여부 확인
- `.lock` 파일이 있으면 해당 파일 수정 금지
- 수정 시작 시 `.lock` 파일 생성, 완료 시 삭제
```

공유 파일 수정 프로토콜

여러 팀원이 수정할 수 있는 공유 파일(라우터 설정, 타입 정의 등)은 팀장을 통해 순차적으로 수정한다.

```
# 나쁜 예: 두 팀원이 동시에 routes.ts 수정
# → 충돌 발생

# 좋은 예: 순차 실행
# 1단계: 서연이 API 라우트 추가
tmux send-keys -t team:0.4 \
  "src/routes.ts에 알림 API 라우트를 추가해줘" Enter
# 서연 완료 확인 후...

# 2단계: 수아가 UI 라우트 추가
tmux send-keys -t team:0.3 \
  "src/routes.ts에 알림 설정 페이지 라우트를 추가해줘" Enter
```

전략 3: 충돌 감지 및 해소

예방에도 불구하고 충돌이 발생할 수 있다. 빠른 감지와 해소가 중요하다.

정기적 상태 점검

```
# 팀장이 주기적으로 실행하는 충돌 점검
#!/bin/bash

# 수정 중인 파일 목록 수집
echo "=== 현재 수정 중인 파일 ==="
git diff --name-only
echo ""

echo "=== 스테이지되지 않은 변경 ==="
git status --short
echo ""

# 같은 파일을 여러 곳에서 수정했는지 확인
MODIFIED=$(git diff --name-only)
for file in $MODIFIED; do
    COUNT=$(echo "$MODIFIED" | grep -c "$file")
    if [ "$COUNT" -gt 1 ]; then
        echo "⚠️ 충돌 위험: $file ($COUNT곳에서 수정)"
    fi
done
```

Git 충돌 해소

머지 시 충돌이 발생하면 팀장이 판단하여 해소한다.

```
# 브랜치 머지 시도
git merge feature/notification-api
# CONFLICT: src/routes.ts

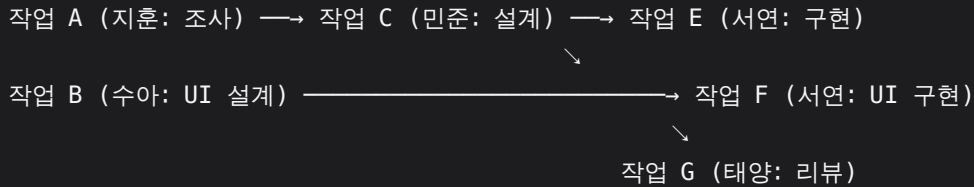
# 충돌 내용 확인
git diff --name-only --diff-filter=U
# src/routes.ts

# 팀원에게 충돌 해소 지시
tmux send-keys -t team:0.4 \
    "src/routes.ts에서 머지 충돌이 발생했어. \
    두 브랜치의 변경을 모두 유지하는 방향으로 해소해줘." Enter
```

전략 4: 작업 순서 설계

팀장이 작업을 배분할 때 충돌 가능성을 고려하여 순서를 설계한다.

의존성 그래프 기반 배분



병렬 작업(A와 B)은 서로 다른 영역을 다루도록 설계하고, 의존성이 있는 작업(C→E, B→F)은 순차 실행한다.

페이지 기반 실행

```
# Phase 1: 병렬 (서로 영역이 다름)
# 지훈 → 기술 조사
# 수아 → UI 설계
# (동시 실행 가능, 충돌 없음)

# Phase 2: 순차 (이전 결과에 의존)
# 민준 → 아키텍처 설계 (지훈+수아의 결과 필요)

# Phase 3: 병렬 (브랜치로 분리)
# 서연 → API 구현 (feature/api 브랜치)
# 수아 → UI 구현 (feature/ui 브랜치)

# Phase 4: 순차 (머지 후 리뷰)
# 팀장 → 브랜치 머지
# 태양 → 통합 리뷰
```

CLAUDE.md에 충돌 방지 규칙 명시

팀 전체에 적용되는 충돌 방지 규칙을 CLAUDE.md에 기록한다.

```
## 충돌 방지 규칙
1. 지시받은 디렉토리/파일 범위 안에서만 작업할 것
2. 공유 파일(routes.ts, types.ts) 수정 전 팀장에게 보고
3. 새 패키지 설치 시 팀장 승인 후 진행
```

4. 작업 완료 후 반드시 `git add + commit`
5. 다른 팀원의 브랜치를 직접 수정하지 않을 것

요약

멀티에이전트 충돌 방지책의 핵심은 영역 분리와 순서 설계다. 디렉토리, 브랜치, 또는 Git Worktree로 작업 영역을 물리적으로 분리하고, 공유 리소스는 순차적으로 접근하도록 설계한다. CLAUDE.md에 충돌 방지 규칙을 명시하고, 팀장이 작업 배분 시 의존성 그래프를 고려하면 대부분의 충돌을 사전에 예방할 수 있다.

9-1. 앞으로의 발전 방향

이 책에서 다룬 TMUX + Claude Code + Remote Control 팀 에이전트 구성은 현재 시점에서 가능한 하나의 조합이다. AI 코딩 도구와 에이전트 기술은 빠르게 발전하고 있으며, 여기서 소개한 방법론의 각 구성 요소도 진화할 것이다. 이 절에서는 앞으로 주목할 발전 방향을 전망한다.

Claude Code의 진화

에이전트 간 직접 통신

현재 팀 에이전트 간 통신은 `tmux send-keys` 를 통한 텍스트 전달 방식이다. 향후 Claude Code가 에이전트 간 직접 통신 프로토콜을 지원하면 더 구조화된 협업이 가능해진다.

현재:

팀장 → `tmux send-keys` → 팀원 (텍스트 기반)

미래:

팀장 → Agent Protocol → 팀원 (구조화된 메시지)

팀원 → Agent Protocol → 팀장 (구조화된 결과)

MCP (Model Context Protocol) 확장

MCP는 Claude Code가 외부 도구와 데이터 소스에 접근하는 표준 프로토콜이다. MCP 서버가 더 풍부해지면 각 팀원이 자신의 역할에 맞는 전문 도구를 직접 활용할 수 있다.

역할	현재	MCP 확장 후
PM	파일 기반 문서 관리	Jira/Linear MCP로 직접 티켓 관리
리서처	웹 검색 + 정리	사내 위키/Confluence MCP로 지식 검색
디자이너	마크다운 와이어프레임	Figma MCP로 디자인 파일 직접 접근
개발자	코드 편집 + 테스트	DB MCP로 스키마 직접 관리
리뷰어	PR diff 읽기	SonarQube MCP로 정적 분석 연동

Headless 모드 고도화

Claude Code의 비대화형 실행(`-p` 플래그, Stream JSON)이 더 성숙해지면 CI/CD 파이프라인과의 통합이 자연스러워진다.

GitHub PR 생성

- CI 트리거
- Claude Code 자동 실행 (코드 리뷰)
- 결과를 PR 코멘트로 게시
- 리뷰어에게 알림

멀티에이전트 프레임워크의 등장

현재 우리는 TMUX 파인으로 에이전트를 물리적으로 분리하고, 셸 스크립트로 조율하고 있다. 전용 멀티에이전트 프레임워크가 등장하면 이 과정이 더 체계적으로 변할 수 있다.

기대되는 기능

- 역할 정의 **DSL**: 코드로 팀 구조와 역할을 정의
- 작업 큐: 팀원 간 작업을 구조화된 큐로 전달
- 상태 추적: 각 에이전트의 작업 상태를 대시보드로 확인
- 자동 조율: 의존성 기반으로 작업 순서를 자동 결정

현재 접근 방식의 가치

프레임워크가 등장하더라도 이 책에서 익힌 개념은 유효하다.

이 책에서 배운 것	프레임워크에서의 대응
TMUX 파인 = 에이전트 격리	컨테이너/프로세스 격리
CLAUDE.md = 역할 정의	에이전트 설정 파일
tmux send-keys = 지시 전달	메시지 큐 / API 호출
Bot Mode = 외부 연동	웹훅 / 이벤트 시스템

도구가 바뀌어도 역할 분리, 작업 분배, 충돌 관리, 결과 종합이라는 팀 운용의 본질은 동일하다.

Remote Control의 확장

멀티 디바이스 협업

현재 Remote Control은 단일 사용자가 모바일에서 접속하는 모델이다. 향후 여러 사용자가 동시에 같은 세션을 관찰하거나 제어할 수 있다면, 실시간 팀 협업 도구로 진화할 수 있다.

```
PM (태블릿) → 아키텍처 문서 확인
↓ 동일 세션
개발자 (데스크톱) → 코드 구현
↓
리뷰어 (모바일) → 실시간 리뷰 코멘트
```

음성 제어

모바일 환경에서 텍스트 입력 대신 음성으로 지시를 전달하는 인터페이스가 추가되면, 이동 중이나 회의 중에도 팀을 제어할 수 있다.

우리가 준비할 것

기술이 발전해도 기본기가 중요하다. 이 책을 통해 익힌 다음 능력은 어떤 도구에서도 통용된다.

1. 역할 설계: 작업을 분해하고 적절한 역할에 배정하는 능력
2. 지시 작성: 모호하지 않고 실행 가능한 지시를 작성하는 능력
3. 충돌 관리: 병렬 작업에서 발생하는 충돌을 예방하고 해소하는 능력
4. 결과 검증: 에이전트의 출력물을 평가하고 피드백하는 능력
5. 비용 인식: 토큰 소비를 이해하고 최적화하는 습관

이것들은 특정 도구에 종속되지 않는 에이전트 운용 리터러시다.

요약

AI 에이전트 기술은 MCP 확장, 에이전트 간 직접 통신, 전용 멀티에이전트 프레임워크 등의 방향으로 발전하고 있다. 도구와 프로토콜은 바뀌지만, 이 책에서 다룬 역할 분리, 작업 분배, 충돌 관리의 원칙은 변하지 않는다. 현재의 TMUX + Claude Code 조합으로 실전 경험을 쌓는 것이 미래의 어떤 도구에서도 팀 에이전트를 효과적으로 운용하는 최선의 준비다.

9-2. 커뮤니티 참여

혼자 배우는 것보다 커뮤니티에서 경험을 공유하고 피드백을 받는 것이 훨씬 빠르다. Claude Code와 AI 에이전트 분야는 활발하게 발전 중이며, 관련 커뮤니티도 빠르게 성장하고 있다. 이 절에서는 주요 커뮤니티와 참여 방법을 안내한다.

Claude Code 공식 채널

GitHub 저장소

```
https://github.com/anthropics/claude-code
```

Claude Code의 공식 저장소다. 이슈 트래커에서 버그 리포트, 기능 요청, 질문을 할 수 있다.

활동	방법
버그 리포트	Issues → New Issue → Bug Report
기능 요청	Issues → New Issue → Feature Request
질문	Issues → New Issue → Question
코드 기여	Fork → 수정 → Pull Request

Anthropic 공식 문서

```
https://docs.anthropic.com/en/docs/claude-code
```

Claude Code의 공식 문서다. Remote Control, 설정, 플러그, 권한 등 모든 기능의 상세 문서가 있다. 새 버전이 출시될 때마다 업데이트된다.

Anthropic Discord

Anthropic 공식 Discord 서버에서 Claude Code 관련 채널을 찾을 수 있다. 실시간으로 질문하고 다른 사용자의 경험을 확인할 수 있다.

관련 도구 커뮤니티

이 책에서 다룬 도구들의 개별 커뮤니티도 유용하다.

TMUX

```
https://github.com/tmux/tmux
```

TMUX의 공식 저장소이자 커뮤니티. 고급 설정, 플러그인 개발, 성능 최적화 관련 이슈와 위키가 풍부하다.

RTK (Rust Token Killer)

RTK 저장소의 이슈 트래커에서 새로운 명령어 필터 요청, 절약을 개선 아이디어, 혹은 통합 관련 논의에 참여할 수 있다.

gstack / superpowers / GSD

Claude Code 플러그인 생태계의 도구들이다. 각 저장소에서 플러그인 사용법, 커스텀 스킬 작성, 워크플로우 공유가 이루어진다.

경험 공유 방법

블로그 포스트

자신만의 팀 에이전트 구성과 운용 경험을 블로그에 기록하면 커뮤니티에 큰 도움이 된다. 특히 다음 주제가 인기 있다.

- 실제 프로젝트에서의 팀 에이전트 적용 사례
- 토큰 최적화 전/후 비용 비교
- 특정 워크플로우 자동화 레시피
- 충돌 관리 실패와 교훈

셋업 스크립트 공유

팀 환경 셋업 스크립트를 공개 저장소에 공유하면 다른 사용자가 빠르게 시작할 수 있다.

```
# 예시: 셋업 스크립트 저장소 구조
team-agent-starter/
├─ setup-team.sh      # 팀 환경 구성
├─ check-team.sh      # 상태 점검
```

```

├─ claude-md/
│  ├─ leader.md      # 팀장 CLAUDE.md
│  ├─ pm.md          # PM CLAUDE.md
│  ├─ developer.md   # 개발자 CLAUDE.md
│  └─ reviewer.md    # 리뷰어 CLAUDE.md
├─ workflows/
│  ├─ feature-dev.sh  # 기능 개발 워크플로우
│  ├─ hotfix.sh       # 핫픽스 워크플로우
│  └─ code-review.sh  # 코드 리뷰 워크플로우
└─ README.md

```

CLAUDE.md 템플릿 공유

역할별로 검증된 CLAUDE.md 템플릿을 공유하면 다른 팀이 시행착오를 줄일 수 있다. 어떤 규칙이 효과적이었는지, 어떤 규칙이 불필요했는지 경험을 포함하면 더 가치있다.

학습 리소스

공식 자료

자료	내용
Anthropic 문서	Claude Code 전체 기능 레퍼런스
Claude Code 릴리즈 노트	버전별 변경 사항
Anthropic 블로그	새 기능 소개, 사용 사례

실습 방법

이 책의 내용을 실습할 때 권장하는 순서다.

```

1단계: 기본 환경 구성
├─ Ubuntu + Claude Code + TMUX 설치 (2~3장)

2단계: 2인 팀으로 시작
├─ 팀장(Pane 0) + 개발자(Pane 1) 구성
├─ 간단한 작업 지시 → 결과 확인 반복

3단계: 역할 확장
├─ PM, 리서처, 리뷰어 추가
├─ 워크플로우 자동화 실습

4단계: Remote Control 연동

```

—— 모바일에서 팀 제어 실습

5단계: 실전 프로젝트 적용

—— 실제 프로젝트에 팀 에이전트 투입

2인 팀부터 시작하여 점진적으로 확장하는 것이 학습 곡선을 완만하게 한다.

피드백과 개선

이 책의 내용에 대한 피드백, 오류 제보, 개선 제안도 환영한다. 실제 운용 경험에서 발견한 더 나은 방법이 있다면 공유해주길 바란다.

AI 에이전트 팀 운용은 아직 초기 단계다. 정답이 확립되지 않은 분야이기에 각자의 시행착오와 발견이 모두에게 가치 있다. 커뮤니티에 참여하여 함께 이 분야를 발전시켜 나가길 바란다.

요약

Claude Code와 AI 에이전트 커뮤니티는 GitHub, Discord, 블로그 등 여러 채널에서 활발하게 운영되고 있다. 자신의 경험을 공유하고, 다른 사용자의 사례에서 배우고, 셋업 스크립트와 CLAUDE.md 템플릿을 교환하면 개인과 커뮤니티 모두가 성장한다. 2인 팀부터 시작하여 점진적으로 확장하는 것이 가장 효과적인 학습 경로다.